



ResearchOS is an ethical AI research assistant that supports researchers through the full publishing workflow, from choosing a topic to preparing a paper for submission.

It helps users:

generate novel or improvement-based research ideas,
review and compare research papers,
identify research gaps,
organize manuscript sections,
convert content into LaTeX,
create architecture or workflow diagrams,
check originality and AI-use risk,
and recommend suitable conferences or journals.

The main idea is assist, not replace: it should use grounded paper-based evidence, keep citations visible, and avoid unsupported or plagiarized content. It also adds optional features like a novelty score, reviewer simulation, and dataset recommendations.

In short, ResearchOS is a research copilot for students and researchers that makes the literature review and publication process faster, more structured, and more ethical.

based on the summary and synopsis give me a detailed roadmap to create the project and how to approach it

Yes — based on your summary and synopsis, ResearchOS should be approached as a **modular research intelligence platform**, not as a single chatbot. The strongest implementation path is to build a citation-grounded pipeline first, then layer higher-level features like gap detection, draft generation, reviewer simulation, and venue recommendation on top of that evidence layer.^[1]

Product framing

Your synopsis defines ResearchOS as an ethical, end-to-end assistant that helps with paper collection, parsing, structured extraction, comparison, research-gap discovery, citation-grounded drafting, and integrity validation. That means your core differentiator is not “AI writes papers,” but “AI organizes and reasons over papers with visible evidence.”^[1]

So your north-star architecture should be:

1. Ingest papers.
2. Parse and structure them.
3. Retrieve grounded evidence.

4. Generate only from retrieved evidence.
5. Validate outputs for citation support and academic integrity. ^[1]

Build strategy

The best way to approach this as a major project is to split it into **three layers**:

Layer	Purpose	What it includes
Evidence layer	Make papers machine-usable	PDF parsing, metadata extraction, chunking, embeddings, vector DB, citation mapping ^[1]
Intelligence layer	Produce research insights	Summaries, paper comparison, gap detection, novelty hints, dataset suggestions, manuscript section planning ^[1]
Trust layer	Keep it ethical and useful	Grounded citations, unsupported-claim detection, originality checks, reviewer simulation, AI-use risk flags ^[1]

This structure maps directly to the phased methodology in your synopsis, especially collection, preprocessing, retrieval, comparison, draft generation, and integrity validation. ^[1]

System roadmap

Phase 0: Scope freeze

Before coding, convert the synopsis into a strict MVP. Your synopsis lists many features, but for a final-year project you should define “must-have,” “good-to-have,” and “demo-only” features so the project remains finishable. ^[1]

Use this cut:

- Must-have:
 - Paper upload and parsing
 - Metadata + section extraction
 - Semantic search over papers
 - Structured cross-paper comparison
 - Citation-grounded answer generation
 - Research gap identification
 - Manuscript section organizer
 - Reviewer simulation basics ^[1]
- Good-to-have:
 - LaTeX conversion
 - Architecture/workflow diagram generation
 - Dataset recommendation
 - Conference/journal recommendation

- Novelty score^[1]
- Optional/demo:
 - AI-use risk scoring
 - Originality heuristic checker
 - Fine-grained citation consistency validator^[1]

A good final demo is better than an over-scoped incomplete product.

Phase 1: Problem and users

Your users are students and early-stage researchers who struggle with literature review, comparison, gap detection, and draft organization because current tools are fragmented and often not citation-transparent. Build every screen around 4 user jobs:^[1]

- “I need to understand 10–20 papers fast.”
- “I need to compare methods/datasets/results.”
- “I need to find a research gap I can actually work on.”
- “I need help drafting a paper section without fake claims.”^[1]

From this, define the key workflow:

Upload papers -> parse -> ask query -> compare papers -> identify gaps -> outline draft -> generate grounded section -> run review simulation.^[1]

Phase 2: Data model design

This part is critical. Do not start with prompting first. Start with a schema.

Your minimum entities should be:

- Paper
 - id, title, authors, year, abstract, venue, keywords, doi, references
- Section
 - paper_id, section_title, text
- Chunk
 - section_id, text, page_no, citation_anchor, embedding
- ExtractedFact
 - paper_id, category, value, evidence_chunk_ids, confidence
- ComparisonTable
 - topic, compared_paper_ids, fields
- DraftSection
 - title, content, source_chunk_ids
- ReviewReport

- novelty, clarity, methodology, limitations, unsupported_claims^[1]

For extracted categories, align with your synopsis:

- methodology
- model architecture
- dataset
- baseline
- evaluation metric
- experimental setup
- limitations
- future work
- findings^[1]

This schema gives you explainability because every insight can point back to source chunks.

Phase 3: Core architecture

A clean architecture for ResearchOS would be:

- Frontend: React or Next.js
- Backend API: FastAPI
- Async workers: Celery or FastAPI background tasks
- Database: PostgreSQL
- Vector database: Chroma, Qdrant, or FAISS
- Object storage: local/S3 for PDFs
- LLM provider: OpenAI-compatible API or open-source model
- Parsing stack: PyMuPDF + GROBID-style heuristics or structured PDF extraction
- Embeddings: sentence-transformers or API embeddings^[1]

Recommended backend modules:

- ingestion-service
- parsing-service
- indexing-service
- retrieval-service
- extraction-service
- generation-service
- validation-service
- recommendation-service^[1]

Approach it as a pipeline with independent modules, because your synopsis itself is modular and phased. ^[1]

Feature-by-feature implementation

1. Paper collection and ingestion

Your synopsis starts with paper collection from uploaded PDFs or public repositories plus metadata extraction. So first implement: ^[1]

- PDF upload
- metadata extraction
- reference parsing
- storage of raw files and extracted text
- paper status tracking: uploaded, parsed, indexed, analyzed ^[1]

Good practice:

- Keep the original PDF.
- Store page-wise text.
- Track parser confidence.
- Allow manual metadata correction in UI.

2. Parsing and preprocessing

This is the backbone of the whole project because poor parsing breaks retrieval and citation grounding. Your synopsis explicitly depends on chunking, embedding, and machine-readable representation creation. ^[1]

Implement:

- PDF text extraction
- heading detection
- section segmentation
- chunking by semantic section, not fixed tokens only
- chunk metadata: page number, section name, chunk index
- cleaned citation references and bibliography mapping ^[1]

Best approach:

- Chunk by section with overlap.
- Preserve page numbers.
- Preserve inline reference markers if possible.
- Add `paper_title`, `section_title`, and `year` into chunk metadata.

3. Semantic search and RAG

Your synopsis positions RAG as the factual grounding layer. This should be your first intelligent feature, and it must work well before fancy generation features. ^[1]

Pipeline:

- User query
- retrieve top-k chunks
- rerank chunks
- feed top evidence to LLM
- return answer with visible citations ^[1]

Important design rule:

Never let the LLM answer from memory if evidence is missing. Instead return:

- “insufficient evidence in uploaded corpus”
- “needs more papers”
- “low confidence due to limited retrieval” ^[1]

That ethical fallback will strengthen your viva/demo.

4. Structured information extraction

Your synopsis says the system should extract methodologies, architectures, datasets, baselines, metrics, limitations, and future directions. This should not be one big free-text summary. Use extraction templates. ^[1]

For each paper, run extraction into JSON like:

- objective
- problem addressed
- proposed method
- architecture/model
- dataset used
- baselines
- evaluation metrics
- best reported result
- limitations
- future work
- domain/task ^[1]

Use an LLM with a fixed output schema and save source chunk IDs for each extracted field. That traceability is your biggest academic strength.

5. Cross-paper comparison

This is one of your highest-value features because it directly addresses the literature review pain point mentioned in the synopsis. ^[1]

Build comparison views like:

Field	Paper A	Paper B	Paper C
Task	...	...	...
Method	...	...	...
Dataset	...	...	...
Baselines	...	...	...
Metric	...	...	...
Best result	...	...	...
Limitation	...	...	...

Also support:

- compare by methodology
- compare by dataset
- compare by performance metric
- compare by limitations and future work ^[1]

This is more valuable than generic summarization because it creates structured state-of-the-art views.

6. Research gap identification

Your synopsis explicitly includes identifying unexplored areas and emerging trends by analyzing similarities, limitations, and future directions across papers. This feature should be implemented as evidence synthesis, not hallucinated novelty. ^[1]

A strong way to do it:

- Aggregate repeated limitations across papers.
- Find tasks/datasets with low coverage.
- Detect where many papers optimize one metric but ignore another.
- Detect underexplored combinations: method X not tested on dataset Y.
- Extract “future work” statements across papers and cluster them. ^[1]

Output style:

- observed trend
- evidence papers

- gap statement
- why it matters
- suggested project directions
- confidence level^[1]

Do not label every weakly supported idea as “novel.” Use “potential gap,” “underexplored direction,” or “improvement opportunity.”

7. Idea generation and novelty support

The summary you gave adds idea generation, novelty scoring, and improvement-based suggestions. This fits well if grounded on extracted literature evidence rather than pure brainstorming.^[1]

Implement idea generation in 3 modes:

- Improvement-based: improve limitation in existing method
- Transfer-based: apply method from adjacent domain
- Combination-based: combine complementary strengths of papers^[1]

Novelty score should be heuristic, not absolute. Score based on:

- overlap with existing extracted methods
- similarity to prior abstracts
- uniqueness of method-dataset pairing
- recurrence of same idea in retrieved papers
- presence of explicit related work overlap^[1]

Call it “novelty heuristic” in UI. That wording is safer and academically honest.

8. Manuscript organizer

Your synopsis includes evidence-based manuscript drafting and structured academic content generation. Start smaller: first build a manuscript planner before full text generation.^[1]

Features:

- suggest section structure
- map notes and evidence to sections
- generate bullet outline for:
 - abstract
 - introduction
 - related work
 - methodology
 - experiments

- conclusion^[1]

This will make the drafting feature much more controllable than direct long-form generation.

9. Citation-grounded drafting

Only after retrieval and extraction are solid should you add section generation. Your synopsis requires every generated claim to be linked to supporting citations.^[1]

Drafting flow:

- User selects topic/goal.
- System retrieves supporting chunks.
- LLM produces section text.
- Each sentence is tagged with source references.
- Validator checks unsupported claims.
- User edits before export.^[1]

Rules to enforce:

- No unsupported sentence.
- No citation-free generated paragraph.
- No invented metric or result.
- If evidence is weak, output bullets instead of polished prose.

That “assist, not replace” philosophy matches your project vision directly.^[1]

10. LaTeX conversion

This is a very demo-friendly feature. Once section content is structured, convert it into LaTeX templates:

- title
- abstract
- sections
- subsection formatting
- bibliography placeholders
- figure/table blocks^[1]

Do not try to solve full bibliography generation initially. For MVP, export:

- clean LaTeX section text
- citation placeholders
- BibTeX-ready reference list

11. Diagram generation

Since your summary includes architecture or workflow diagrams, implement this using structured JSON -> Mermaid conversion.

Useful diagram types:

- system architecture
- model pipeline
- workflow diagram
- experiment flow
- literature taxonomy map^[1]

This is practical because:

- it looks strong in demo,
- it is deterministic,
- and it avoids hallucinated images.

12. Reviewer simulation

Your synopsis explicitly includes peer-review simulation for novelty, clarity, methodology, and weaknesses. Make this a rubric-based evaluator rather than a vague chatbot opinion.^[1]

Reviewer dimensions:

- problem clarity
- literature coverage
- method justification
- experiment completeness
- citation support
- novelty heuristic
- limitations acknowledged
- reproducibility readiness^[1]

Output format:

- score
- concern
- evidence/rationale
- improvement suggestion^[1]

This feels much more like real peer review.

13. Originality and AI-use risk

Your summary includes originality and AI-use risk checks. Since full plagiarism detection is hard, keep this ethical and limited.

You can implement:

- unsupported-claim detector
- overly generic academic language detector
- citation density checker
- repeated phrasing detector across generated sections
- similarity warning against retrieved chunks
- “AI-like verbosity” heuristic^[1]

Phrase it as:

- originality support
- citation support risk
- generic writing risk
- AI-overuse warning

Avoid claiming “plagiarism detector” unless you actually have validated plagiarism infrastructure.

14. Venue recommendation

This can be built from structured metadata:

- domain
- task
- methodology
- dataset scale
- paper maturity
- application area^[1]

Recommend venues by:

- topic similarity
- accepted scope match
- empirical/theoretical style
- workshop vs conference fit
- journal vs conference maturity^[1]

Keep this as recommendation support, not authoritative advice.

Recommended MVP

For a major project, I would strongly recommend this MVP:

- Upload 5–20 papers
- Parse and index them
- Ask grounded questions over corpus
- Auto-extract structured fields
- Compare papers in table form
- Detect repeated limitations/future work
- Generate a literature review outline with citations
- Simulate reviewer feedback
- Export structured content to LaTeX^[1]

This MVP already fully aligns with your synopsis objectives of retrieval, organization, extraction, comparison, gap identification, citation-grounded drafting, and review support.^[1]

Suggested tech stack

Given your background, this stack is practical:

- Frontend: Next.js or React + Tailwind
- Backend: FastAPI
- Task queue: Celery + Redis
- Database: PostgreSQL
- Vector DB: Qdrant or Chroma
- PDF parsing: PyMuPDF, pdfplumber, Grobid-inspired sectioning
- Embeddings: bge-small or all-MiniLM for MVP, stronger model later
- Reranking: cross-encoder reranker
- LLM tasks:
 - extraction
 - comparison synthesis
 - grounded generation
 - reviewer simulation^[1]

Why this works:

- modular,
- demoable,
- academically defensible,
- and scalable enough for future extension.

Development plan

Week 1–2: Requirements and architecture

- Freeze scope and MVP.
- Create use cases and workflow diagrams.
- Design DB schema and pipeline architecture.
- Define all extraction fields and output JSON contracts.^[1]

Week 3–4: Ingestion pipeline

- PDF upload
- metadata extraction
- section parsing
- chunking
- store raw text and metadata^[1]

Week 5–6: Embeddings and retrieval

- vector indexing
- semantic search
- reranking
- cited answer generation^[1]

Week 7–8: Structured extraction

- per-paper field extraction
- evidence linking
- confidence scoring
- UI for extracted summaries^[1]

Week 9–10: Comparison and gap detection

- comparison tables
- recurring limitation mining
- trend/gap synthesis
- evidence-backed idea generation^[1]

Week 11–12: Drafting and LaTeX

- manuscript outline generator
- section drafting with citations
- unsupported-claim validator
- LaTeX export^[1]

Week 13: Reviewer simulation

- rubric design
- scoring engine
- critique generation
- improvement suggestions^[1]

Week 14: UI polishing and evaluation

- dashboard cleanup
- add diagrams
- add venue recommendations if time remains
- prepare demo dataset and test cases^[1]

UI modules

A strong UI for ResearchOS should include these pages:

- Dashboard
- Paper Library
- Paper Detail View
- Semantic Search / Ask ResearchOS
- Comparison Workspace
- Gap Explorer
- Manuscript Planner
- Draft Generator
- Reviewer Simulation
- Export Center^[1]

Best interaction pattern:

- left sidebar: papers/workspaces
- center: response/comparison/draft view
- right panel: citations/evidence/source chunks

That right-side evidence panel will make your product stand out.

Evaluation plan

For academic evaluation, do not just say “it works.” Measure it.

Suggested evaluation metrics:

- retrieval relevance: precision@k or human-rated relevance
- extraction quality: field-level accuracy on sampled papers
- citation grounding rate: percentage of generated claims with evidence
- unsupported claim reduction
- time saved in literature review task
- user satisfaction from students/research peers^[1]

Run a small user study:

- 5–10 users
- task: compare papers or draft related-work section
- compare baseline manual workflow vs ResearchOS-assisted workflow

This will make your final report much stronger.

Risks and how to handle them

Parsing quality risk

Academic PDFs are messy. Reduce risk by:^[1]

- supporting uploaded PDFs first,
- storing page-level fallback text,
- allowing manual correction.

Hallucination risk

Your synopsis is explicitly built to reduce hallucinations with citation grounding. Enforce:^[1]

- generation only from retrieved context,
- low-confidence fallback,
- unsupported-claim warnings.

Over-scoping risk

Your feature list is large. Solve it by shipping MVP first and presenting extras as extensible modules.^[1]

Evaluation risk

If you do not define benchmarks, the project may look like “just another RAG app.” Your structured extraction, comparison tables, reviewer simulation, and integrity validation are what make it distinct. ^[1]

Practical approach

The smartest way to approach the build is:

- First, build the paper intelligence engine.
- Second, make outputs structured and traceable.
- Third, add generation only where citations can be shown.
- Fourth, make ethics visible in the interface through evidence panels, confidence labels, and unsupported-claim checks. ^[1]

In other words, treat ResearchOS as:

PDF Intelligence + Retrieval + Structured Knowledge + Ethical Writing Support, not as a generic academic chatbot. ^[1]

Final architecture recommendation

If I were turning your synopsis into a strong final-year build, I would implement this sequence:

1. Paper upload + parser.
2. Chunking + embeddings + vector search.
3. Structured extractor for methods/datasets/metrics/limitations.
4. Cross-paper comparison tables.
5. Gap detector from clustered limitations and future work.
6. Grounded outline and section drafting.
7. Reviewer simulation and integrity checks.
8. LaTeX export and optional venue recommendation. ^[1]

That sequence matches the methodology in your synopsis almost exactly, which will make your implementation, report, and presentation highly consistent. ^[1]

Would you like me to turn this into a **module-wise architecture + database schema + API endpoint roadmap** next?

✱

can you explain what modules our project will have based on the objectives given in the synopsis and give an overview of what it will look like

Yes — based on the objectives and methodology in your synopsis, ResearchOS will look like a **multi-module research workspace** where each module supports one stage of the research workflow, from paper collection to final manuscript review. Instead of behaving like a generic chatbot, it should feel like a dashboard with connected tools for literature exploration, evidence extraction, comparison, drafting, and validation.^[2]

Core modules

These are the main modules your project should have if you map them directly from the synopsis objectives.^[2]

Module	What it does	Derived from synopsis
Paper Ingestion Module	Collects uploaded papers or papers from research sources and stores metadata	Paper collection and data acquisition phase ^[2]
Parsing and Preprocessing Module	Extracts text from PDFs, cleans it, splits it into chunks, and prepares embeddings	Document parsing and preprocessing phase ^[2]
Retrieval and Search Module	Lets users query the paper corpus using semantic search and RAG	Information retrieval objective and methodology ^[2]
Knowledge Extraction Module	Pulls structured fields like methods, datasets, metrics, baselines, limitations, and future work	Structured information extraction objective ^[2]
Comparison and Analysis Module	Compares multiple papers and shows trends, similarities, differences, and benchmark views	Cross-paper comparison and research analysis objective ^[2]
Research Gap Module	Identifies unexplored areas, recurring limitations, and improvement opportunities	Research gap identification objective ^[2]
Manuscript Assistance Module	Helps organize sections and generate citation-grounded draft content	Citation-grounded manuscript generation objective ^[2]
Review Simulation Module	Simulates reviewer feedback for novelty, clarity, completeness, and weaknesses	Peer-review simulation objective ^[2]
Integrity Validation Module	Flags unsupported claims, citation mismatch, and overly generic or risky content	Academic integrity and citation consistency objective ^[2]
Export and Presentation Module	Converts output into usable formats such as LaTeX and possibly diagrams	Manuscript assistance and structured workflow support ^[2]

These modules together match the synopsis goal of creating a unified platform instead of many disconnected tools.^[2]

What each module means

1. Paper ingestion module

This is the entry point of the system. According to the methodology, the system begins by collecting papers from uploaded PDFs or public repositories and extracting metadata such as title, authors, abstract, year, keywords, and references. ^[2]

So in the UI, this module would look like:

- an upload area for PDFs,
- a paper library,
- metadata cards for each paper,
- and status labels like uploaded, parsed, indexed, or analyzed. ^[2]

2. Parsing and preprocessing module

After upload, ResearchOS must parse PDF text, clean it, split it into meaningful chunks, and convert it into embeddings for storage in a vector database. This module is not very visible to the user, but it is the technical backbone of the whole system. ^[2]

Internally, this module handles:

- PDF text extraction,
- section identification,
- chunk creation,
- embedding generation,
- and linking chunks back to source pages for citation traceability. ^[2]

3. Retrieval and search module

This module is the “ask questions over papers” part of the system. The synopsis says the system should use RAG to retrieve relevant document sections based on user queries and then generate grounded responses from those sections. ^[2]

In the product, this would look like:

- a search bar or chat interface,
- retrieved evidence snippets,
- answer panels with visible citations,
- and maybe filters by paper, year, author, or topic. ^[2]

This module is what makes the platform feel interactive and useful for fast literature review.

4. Knowledge extraction module

Your synopsis strongly emphasizes automatic extraction of methodologies, model architectures, datasets, baselines, evaluation metrics, limitations, and future research directions. This means you need a dedicated module that converts raw paper text into structured research facts.^[2]

This module would produce outputs like:

- “Method used: Transformer-based classifier”
- “Dataset: CIFAR-10”
- “Metric: Accuracy, F1-score”
- “Limitation: small dataset, no real-world testing”^[2]

Visually, this may look like structured cards or tables on each paper’s detail page.

5. Comparison and analysis module

This module is one of the biggest academic strengths of your project. The synopsis says ResearchOS should enable structured comparisons of methodologies, datasets, results, and performance metrics across papers.^[2]

So this module would likely show:

- side-by-side comparison tables,
- metric comparison charts,
- summary of common methods,
- recurring datasets,
- and trend views across multiple papers.^[2]

This is the module that helps users answer, “How do these papers differ?” instead of reading each paper manually.

6. Research gap module

The synopsis explicitly says the system should identify research gaps and emerging trends by analyzing similarities, limitations, and unexplored areas across papers. So this should be a separate reasoning module, not just a side effect of summarization.^[2]

It would analyze:

- repeated limitations across papers,
- missing experiments,
- underexplored datasets,
- weak evaluation settings,
- and future work statements across multiple papers.^[2]

The output might look like:

- potential research gap,
- supporting papers,
- reason for the gap,
- and suggested improvement direction. ^[2]

7. Manuscript assistance module

This module comes from the objective of generating citation-grounded manuscript drafts using retrieved evidence. It should not blindly generate full papers; instead, it should help organize sections and draft content with visible supporting references. ^[2]

This module would likely include:

- section organizer for abstract, introduction, related work, methodology, results, and conclusion,
- note-to-section mapping,
- grounded paragraph generation,
- and citation insertion support. ^[2]

So the user experience would feel more like a research writing assistant than an automatic paper writer.

8. Review simulation module

The synopsis clearly includes AI-powered reviewer simulation to evaluate novelty, clarity, methodological completeness, and weaknesses before submission. This module acts like a mock conference reviewer. ^[2]

It can produce:

- clarity feedback,
- missing experiment warnings,
- novelty concerns,
- baseline comparison issues,
- and suggestions to strengthen the paper. ^[2]

In the UI, this can look like a rubric report with sections such as strengths, weaknesses, major concerns, and improvement suggestions.

9. Integrity validation module

This is part of what makes the project ethical. The synopsis says ResearchOS should detect unsupported claims, verify citation consistency, and encourage evidence-based research practices.^[2]

So this module should check:

- whether generated claims have evidence,
- whether citations match the text,
- whether content is too generic,
- and whether any paragraph seems insufficiently grounded.^[2]

This can appear as a “trust panel” or “validation report” next to generated drafts.

10. Export and presentation module

Since your broader summary mentions LaTeX conversion and diagram creation, the final module should help transform structured outputs into submission-ready forms. This is especially useful because users may want to move from analysis into actual writing artifacts.^[2]

This module may include:

- LaTeX export,
- formatted literature review export,
- architecture/workflow diagram generation,
- and downloadable citation-backed summaries.^[2]

How the whole system will look

At a high level, the project will probably look like a **research dashboard** with multiple connected panels rather than a plain chat screen. The synopsis repeatedly describes it as a unified platform that integrates literature exploration, knowledge extraction, comparative analysis, manuscript assistance, and research quality assessment in one workflow.^[2]

A practical interface could look like this:

- Left sidebar: Papers, Workspaces, Drafts, Review Reports.
- Main center panel: current task, such as search results, comparison table, or manuscript draft.
- Right evidence panel: citations, retrieved chunks, source paper links, confidence or validation notes.^[2]

That layout would make the “citation-grounded” aspect visible at all times, which is central to your project vision.^[2]

User workflow view

If we convert the synopsis into a user journey, the project will look something like this: upload papers, process them into a searchable knowledge base, extract structured insights, compare papers, identify gaps, create a draft outline, generate citation-backed writing, and finally run review and integrity checks.

So the workflow is:

- 1. Add papers to corpus.
- 2. Parse and index them.
- 3. Ask questions or search themes.
- 4. View extracted methods, datasets, metrics, and limitations.
- 5. Compare papers side by side.
- 6. See possible research gaps and trends.
- 7. Organize manuscript sections and draft with citations.
- 8. Run reviewer simulation and integrity validation before export.

This sequence follows your methodology phase by phase almost exactly.

Best way to present it

For your project report or viva, the cleanest explanation is to present ResearchOS as five major layers:

Layer	Modules inside
Input layer	Paper ingestion, metadata acquisition
Processing layer	Parsing, chunking, embeddings, vector storage
Intelligence layer	Retrieval, extraction, comparison, gap analysis
Writing layer	Manuscript assistance, LaTeX generation
Trust layer	Reviewer simulation, integrity validation, citation checks

This makes the system easy to explain technically and also aligns tightly with the objectives in the synopsis.

Recommended final phrasing

In simple words, your project will look like a **research operating system** where a user uploads papers, explores them through search and structured extraction, compares them to find trends and gaps, drafts manuscript sections with citations, and validates quality before submission. The modules are connected in sequence, so each one builds on the output of the previous stage rather than functioning as isolated tools.

Would you like me to turn this into a **module architecture diagram description** you can directly use in your report or presentation?

okay so for the first phase of presentation of the project we need to show around 50% of the work how much of it can be done (given we have 3-4 days left)

Yes — in 3 to 4 days, you can realistically show a **strong 40–55% project prototype** if you focus only on the core pipeline and not the full end-to-end platform. Since your synopsis is modular and phased, the best first presentation is to demonstrate that the foundation works: paper ingestion, parsing, semantic retrieval, structured extraction, and basic comparison. ^[3]

What counts as 50%

From the synopsis, the full system includes paper collection, preprocessing, retrieval, knowledge extraction, comparison, gap identification, manuscript drafting, and review/integrity validation. In only 3–4 days, “50% done” should mean you have completed the **foundation + one visible intelligence layer**, not all modules equally. ^[3]

A realistic definition of 50% for phase-1 presentation is:

Area	Full project scope	What you should finish now
Ingestion	Upload and store papers	Complete ^[3]
Parsing	Extract text, chunk, metadata	Complete ^[3]
Retrieval	Semantic search / RAG base	Complete basic version ^[3]
Extraction	Methods, datasets, metrics, limitations	Complete partial structured extraction ^[3]
Comparison	Cross-paper comparison	Complete basic table view ^[3]
Gap analysis	Identify unexplored areas	Only simple heuristic demo ^[3]
Draft generation	Citation-grounded writing	Optional mock/demo only ^[3]
Review simulation	Reviewer feedback	Not necessary for first phase ^[3]
Integrity validation	Unsupported claims / citation checks	Not necessary for first phase ^[3]

So for the first review, you do **not** need a complete manuscript generator or reviewer simulator. Those belong to later phases in your own methodology. ^[3]

What you can complete

In 3–4 days, you can reasonably build and present these 5 things:

1. Paper upload and corpus creation

You can definitely complete a PDF upload flow where users add research papers and the system stores title, authors, abstract, year, and raw file metadata. This directly matches Phase 1 of your methodology.^[3]

What to show:

- upload PDF,
- paper card list,
- extracted metadata,
- paper library screen.^[3]

2. PDF parsing and chunking

You can also finish basic text extraction, section-wise parsing, and chunk storage for retrieval. This is Phase 2 in the synopsis and is one of the most important technical milestones.^[3]

What to show:

- extracted sections like Abstract, Introduction, Methodology,
- chunk count,
- maybe page-wise parsed preview,
- “indexed successfully” status.^[3]

3. Semantic search over papers

A working semantic search or question-answer interface over the uploaded papers is very possible in 3–4 days if you keep it simple. This aligns with the RAG-based retrieval described in the synopsis.^[3]

What to show:

- search query,
- top relevant chunks,
- source paper title and page,
- simple answer generated from retrieved chunks.^[3]

Even if the answer generation is basic, retrieval itself will already make the project feel real.

4. Structured extraction for a few fields

You probably cannot build every extraction category robustly in 3–4 days, but you can absolutely show structured extraction for 4–6 fields such as:

- methodology,
- dataset,

- metric,
- baseline,
- limitation,
- future work.^[3]

What to show:

- each paper gets an “extracted insights” panel,
- fields are displayed in cards or JSON-like format,
- each field maps to supporting text.^[3]

That is enough to prove the knowledge extraction module is working.

5. Basic comparison view

You can likely build one simple cross-paper comparison table for 2–5 papers. This directly supports the objective of comparing methods, datasets, and experimental results.^[3]

What to show:

- Paper A vs Paper B vs Paper C
- method used
- dataset used
- metric reported
- key limitation^[3]

This will look impressive in presentation even if the backend logic is still basic.

What you should avoid right now

Do not spend these 3–4 days trying to fully build:

- manuscript drafting,
- LaTeX export,
- reviewer simulation,
- novelty score,
- conference recommendation,
- full originality checker.^[3]

These are part of the broader project vision, but your methodology places draft generation and review/integrity validation in later phases after retrieval, extraction, and comparison are established. If you try to build everything now, you may end up with many unfinished features and no stable demo.^[3]

Best 4-day target

If I were planning your first phase strictly for presentation, I would target this as the “50% version”:

Must-build

- Upload PDFs. [3]
- Extract metadata and text. [3]
- Chunk and embed paper content. [3]
- Store searchable corpus. [3]
- Search/query over uploaded papers. [3]
- Extract structured fields from papers. [3]
- Show comparison table for selected papers. [3]

Nice-to-have

- very simple “possible gap” suggestion from repeated limitations,
- one citation-grounded summary block,
- architecture diagram in the UI or slides. [3]

That is enough to honestly say: “We have completed the core research intelligence pipeline and one analysis layer.” [3]

Realistic completion percentage

If you finish the items above, your progress can be presented like this:

Component	Weight	Realistic status in 3–4 days
Paper ingestion	15%	100% [3]
Parsing and chunking	15%	80–100% [3]
Semantic retrieval	15%	70–90% [3]
Structured extraction	15%	60–80% [3]
Comparison view	10%	70–90% [3]
Gap detection	10%	20–40% [3]
Draft generation	10%	0–20% [3]
Reviewer simulation	5%	0–10% [3]
Integrity validation	5%	0–10% [3]

That means your practical progress could be shown as around 45% to 55% of the meaningful system, even if later modules are still pending. [3]

What to say in presentation

For phase 1, frame your work like this:

- “We completed the core knowledge ingestion and retrieval pipeline.”^[3]
- “The system can now parse research papers, build a searchable corpus, extract key fields, and compare papers in structured form.”^[3]
- “Higher-level modules such as research gap reasoning, manuscript drafting, and reviewer simulation are planned for the next phase.”^[3]

That statement is fully aligned with your methodology because the synopsis itself builds the platform in phases from acquisition to validation.^[3]

Recommended 3–4 day plan

Day 1

- PDF upload
- metadata extraction
- paper library UI^[3]

Day 2

- text parsing
- chunking
- embeddings
- vector storage^[3]

Day 3

- search interface
- retrieved evidence display
- simple grounded answer generation^[3]

Day 4

- structured extraction
- comparison table
- polish demo and slides^[3]

If time remains, add a simple research-gap heuristic from common limitations.^[3]

Best demo flow

Your presentation demo should be:

1. Upload 2–3 papers. ^[3]
2. Show metadata extraction. ^[3]
3. Show parsed sections/chunks. ^[3]
4. Ask a query like “What datasets are used across these papers?” ^[3]
5. Show retrieved evidence and answer. ^[3]
6. Open extracted insights panel. ^[3]
7. Show cross-paper comparison table. ^[3]
8. End by saying next phase includes gap detection, drafting, and review simulation. ^[3]

That will look polished and clearly demonstrate substantial progress.

My recommendation

In 3–4 days, aim for a **solid backend pipeline with 2 polished frontend screens**, not a broad but shallow product. If ingestion, retrieval, extraction, and comparison work reliably, you can confidently present that as roughly half of the core project because the later modules depend on this foundation anyway. ^[3]

Would you like me to break this into a **task-by-task 4-day execution checklist** for your team of 3, so each person knows exactly what to build?

✱✱

okay we will aim for 5 modules completions (frontend + backend) lets start with the implementation tell me how to go about it (things to be installed,files to be made) step to step in detail

Yes — for 5 completed modules in 3–4 days, you should build a **tight MVP** around the first four phases of your synopsis: paper collection, parsing/preprocessing, retrieval/knowledge extraction, and research analysis/comparison. Those phases are explicitly defined in the methodology and are enough to present a meaningful half-built system. ^[4]

Scope first

The 5 modules you should implement now are:

Module	Why this one
1. Authentication + Project Workspace	Gives the system a real product feel and organizes users/papers.
2. Paper Upload + Metadata Extraction	Matches Phase 1 paper collection and structured corpus creation. ^[4]

Module	Why this one
3. PDF Parsing + Chunking + Embedding	Matches Phase 2 preprocessing and vectorization. ^[4]
4. Semantic Search + Grounded QA	Matches Phase 3 retrieval using RAG. ^[4]
5. Structured Extraction + Comparison View	Matches Phase 3–4 extraction and cross-paper comparison. ^[4]

Do **not** start with reviewer simulation or manuscript drafting yet, because the synopsis places those after retrieval and analysis, and they depend on the first modules being stable. ^[4]

Tech stack

Use the fastest stack that is still clean enough to demo:

Backend

- FastAPI
- SQLAlchemy
- PostgreSQL
- Qdrant or ChromaDB
- PyMuPDF
- sentence-transformers
- OpenAI-compatible LLM API for extraction/QA
- Pydantic
- Uvicorn^[4]

Frontend

- Next.js
- TypeScript
- Tailwind CSS
- Axios
- React Query or simple fetch hooks
- Zustand or simple local state

Optional but useful

- Docker + docker-compose
- Alembic for migrations
- Redis only if you need async jobs later

For 3–4 days, I recommend:

- PostgreSQL for metadata

- ChromaDB for embeddings
- local file storage for PDFs
- no Celery for now

That keeps setup simple and still aligns with the searchable structured knowledge base in the synopsis.^[4]

What to install

Backend install

Create a backend folder and install:

```
mkdir researchos
cd researchos
mkdir backend frontend
```

Inside backend:

```
cd backend
python -m venv venv
source venv/bin/activate
pip install fastapi uvicorn sqlalchemy psycpg2-binary python-multipart pydantic-set
```

Also install:

```
pip install python-dotenv
```

Frontend install

Inside frontend:

```
npx create-next-app@latest . --typescript --tailwind --eslint --app
npm install axios
npm install @tanstack/react-query
npm install zustand
npm install lucide-react
```

Folder structure

Make the project look like this:

```
researchos/
├─ backend/
│   └─ app/
│       └─ api/
│           └─ routes/
```

```
├── auth.py
│   ├── projects.py
│   ├── papers.py
│   ├── search.py
│   ├── extraction.py
│   └── comparison.py
├── core/
│   ├── config.py
│   ├── database.py
│   └── security.py
├── models/
│   ├── user.py
│   ├── project.py
│   ├── paper.py
│   ├── chunk.py
│   └── extraction.py
├── schemas/
│   ├── user.py
│   ├── project.py
│   ├── paper.py
│   ├── search.py
│   ├── extraction.py
│   └── comparison.py
├── services/
│   ├── pdf_parser.py
│   ├── chunker.py
│   ├── embedder.py
│   ├── vector_store.py
│   ├── rag_service.py
│   ├── extraction_service.py
│   └── comparison_service.py
├── utils/
│   └── text_cleaning.py
├── main.py
├── uploads/
├── requirements.txt
├── .env
├── frontend/
│   ├── app/
│   │   ├── dashboard/
│   │   ├── projects/[id]/
│   │   ├── papers/[id]/
│   │   ├── search/
│   │   └── compare/
│   ├── components/
│   │   ├── layout/
│   │   ├── papers/
│   │   ├── search/
│   │   └── comparison/
│   ├── lib/
│   │   ├── api.ts
│   │   └── query-client.ts
│   └── .env.local
├── README.md
```

Database design

Since the synopsis emphasizes structured knowledge, searchable corpus, and comparison, your DB must preserve paper structure and extracted facts.^[4]

Create these tables first:

1. users

- id
- name
- email
- password_hash
- created_at

2. projects

- id
- user_id
- title
- description
- created_at

3. papers

- id
- project_id
- title
- authors
- year
- abstract
- keywords
- file_path
- status
- uploaded_at

4. sections

- id
- paper_id
- section_title

- content

5. chunks

- id
- paper_id
- section_id
- chunk_text
- page_no
- chunk_index
- vector_id

6. extracted_fields

- id
- paper_id
- field_name
- field_value
- evidence_chunk_ids
- confidence

That schema directly supports retrieval, extraction, and cross-paper analysis from your objectives.^[4]

Backend implementation order

Step 1: backend bootstrap

Create `app/main.py`:

- FastAPI app
- CORS middleware
- include routers
- health check route

Create:

- `core/config.py`
- `core/database.py`

In `.env`:

```
DATABASE_URL=postgresql://postgres:postgres@localhost:5432/researchos
OPENAI_API_KEY=your_key
```

```
UPLOAD_DIR=uploads
CHROMA_DIR=./chroma_db
```

Create DB connection and Base model setup.

Step 2: create models

Create model files:

- `user.py`
- `project.py`
- `paper.py`
- `chunk.py`
- `extraction.py`

Start only with the essential relationships:

- `user -> projects`
- `project -> papers`
- `paper -> chunks`
- `paper -> extracted_fields`

Do not over-model everything initially.

Step 3: create API routers

Make these files:

`auth.py`

For now, keep it minimal:

- `signup`
- `login`
- `dummy token or simple JWT if time permits`

`projects.py`

Routes:

- `POST /projects`
- `GET /projects`
- `GET /projects/{id}`

`papers.py`

Routes:

- POST `/papers/upload`
- GET `/papers/{id}`
- GET `/projects/{project_id}/papers`

`search.py`

Routes:

- POST `/search/query`

`extraction.py`

Routes:

- POST `/papers/{paper_id}/extract`
- GET `/papers/{paper_id}/extractions`

`comparison.py`

Routes:

- POST `/compare`

Service implementation

Now build the intelligence pipeline one service at a time.

Module 1: Authentication + Workspace

This module is not from the research methodology directly, but it is important for product structure.

Backend tasks

Create:

- `models/user.py`
- `models/project.py`
- `schemas/user.py`
- `schemas/project.py`
- `routes/auth.py`
- `routes/projects.py`

Features:

- user signup/login
- create research project/workspace
- list user projects

Frontend tasks

Create pages:

- /
- /dashboard
- /projects/[id]

Components:

- login form
- create project modal
- project cards

What to show

- user logs in
- creates project “Deepfake Detection Review”
- sees empty project workspace

Module 2: Paper upload + metadata extraction

This is directly from Phase 1: collect papers and extract metadata into a structured corpus. ^[4]

Backend tasks

Create:

- routes/papers.py
- services/pdf_parser.py

Functions:

- save uploaded file in uploads/
- open PDF with PyMuPDF
- read first page and abstract area
- extract rough metadata:
 - title
 - authors
 - abstract
 - maybe year if available

Store result in papers table.

Files to create

- `schemas/paper.py`
- `models/paper.py`
- `services/pdf_parser.py`

Inside `pdf_parser.py`, implement:

- `extract_full_text(pdf_path)`
- `extract_basic_metadata(pdf_path)`
- `extract_sections(pdf_text)`

Frontend tasks

Create:

- upload button
- paper list table
- paper detail page

Components:

- `PaperUpload.tsx`
- `PaperCard.tsx`
- `PaperList.tsx`

What to show

- upload 2–3 PDFs
- paper cards appear
- extracted metadata visible

Module 3: Parsing + chunking + embeddings

This matches Phase 2: parse PDFs, clean text, chunk it, convert it into embeddings, and store them in a vector DB.^[4]

Backend tasks

Create:

- `services/chunker.py`
- `services/embedder.py`
- `services/vector_store.py`

- `models/chunk.py`

Implement pipeline:

1. Parse whole PDF text.
2. Detect sections using simple heading heuristics.
3. Split into chunks of 400–700 words with overlap.
4. Generate embeddings using sentence-transformers.
5. Store in Chroma with metadata:
 - `paper_id`
 - `section_title`
 - `page_no`
 - `chunk_index`

Files to create

- `services/chunker.py`
- `services/embedder.py`
- `services/vector_store.py`
- `schemas/search.py`

Functions

- `clean_text(text)`
- `split_into_sections(text)`
- `chunk_text(section_text, chunk_size=500, overlap=80)`
- `generate_embedding(texts)`
- `index_chunks(chunks)`

Frontend tasks

On paper detail page, show:

- parsing status
- number of chunks created
- section preview
- indexed status badge

What to show

- “This paper has 37 searchable chunks”
- user can inspect chunked content

Module 4: Semantic search + grounded QA

This matches Phase 3 RAG retrieval in your synopsis. ^[4]

Backend tasks

Create:

- `services/rag_service.py`
- `routes/search.py`

Flow:

1. User asks a question.
2. Embed the question.
3. Retrieve top-k similar chunks from Chroma.
4. Build context.
5. Send context + question to LLM.
6. Return:
 - answer
 - cited chunks
 - paper titles
 - section names

Files to create

- `services/rag_service.py`
- `schemas/search.py`
- `routes/search.py`

Response format

Return JSON like:

```
{
  "answer": "Most papers use benchmark datasets such as ...",
  "sources": [
    {
      "paper_id": 1,
      "paper_title": "Paper A",
      "section_title": "Methodology",
```

```
    "chunk_text": "..."  
  }  
]  
}
```

Frontend tasks

Create search page:

- query input
- answer card
- source evidence cards
- maybe filters by project/paper

Components:

- `SearchBox.tsx`
- `AnswerPanel.tsx`
- `EvidencePanel.tsx`

What to show

- Ask: “What datasets are most used in these papers?”
- Show grounded answer plus source snippets

Module 5: Structured extraction + comparison

This covers the extraction and comparison objectives from the synopsis. ^[4]

Backend tasks

Create:

- `services/extraction_service.py`
- `services/comparison_service.py`
- `routes/extraction.py`
- `routes/comparison.py`

For extraction, do not try to extract everything. Start with:

- methodology
- dataset
- evaluation metric
- limitation
- future work ^[4]

Flow:

1. Retrieve relevant chunks for each field.
2. Ask LLM to return JSON.
3. Save JSON into `extracted_fields`.

For comparison:

- take selected paper IDs
- fetch extracted fields
- align them into one comparison structure

Files to create

- `models/extraction.py`
- `schemas/extraction.py`
- `schemas/comparison.py`
- `services/extraction_service.py`
- `services/comparison_service.py`

Frontend tasks

Create:

- paper insights panel
- compare papers page
- comparison table component

Components:

- `ExtractionCard.tsx`
- `ComparisonTable.tsx`
- `PaperSelector.tsx`

What to show

- click “Extract Insights”
- extracted fields appear
- select 2–3 papers
- compare them side by side

Frontend page flow

Your frontend should look like this for demo:

1. Dashboard

- project cards
- create project

2. Project workspace

- upload papers
- list papers
- quick stats:
 - total papers
 - total chunks
 - indexed papers

3. Paper detail

- metadata
- parsed sections
- extracted insights

4. Search page

- ask question over project papers
- grounded answer
- source evidence

5. Compare page

- choose papers
- see structured comparison

This gives the unified workflow feel described in the synopsis. ^[4]

Detailed implementation sequence

Day 1

Backend

- setup FastAPI
- setup PostgreSQL
- create models
- create upload API
- save paper metadata

Frontend

- setup Next.js
- create dashboard
- create project page
- create upload UI

Deliverable

- upload papers into a project

Day 2

Backend

- parse PDFs
- create section/chunk pipeline
- store chunks
- integrate Chroma

Frontend

- show paper details
- show parse/index status

Deliverable

- indexed searchable paper corpus

Day 3

Backend

- semantic query endpoint
- grounded answer endpoint
- extraction service

Frontend

- search page
- answer + source evidence cards
- extraction trigger button

Deliverable

- ask questions + extract fields

Day 4

Backend

- comparison endpoint
- cleanup JSON formats
- basic error handling

Frontend

- comparison page
- UI polish
- project stats
- demo testing

Deliverable

- full 5-module demo

Example APIs

Use these endpoints:

- POST /auth/signup
- POST /auth/login
- POST /projects
- GET /projects
- POST /papers/upload

- GET /projects/{id}/papers
- GET /papers/{id}
- POST /papers/{id}/parse
- POST /papers/{id}/index
- POST /search/query
- POST /papers/{id}/extract
- GET /papers/{id}/extractions
- POST /compare

Important shortcuts

To finish fast, use these shortcuts:

- Skip full auth if needed; use one demo user.
- Use local uploads instead of cloud storage.
- Use Chroma instead of a hosted vector DB.
- Use simple regex/heuristic section splitting.
- Extract only 5 structured fields, not 12.
- Compare only extracted summaries, not raw full-text reasoning.

These shortcuts still stay faithful to the synopsis because the objectives focus on structured retrieval, extraction, and comparison, not production cloud complexity. ^[4]

Key implementation rule

Build the system as a **pipeline**, not disconnected pages:

upload -> parse -> chunk -> embed -> search -> extract -> compare ^[4]

That flow is exactly how your methodology is written, so it will help both your implementation and your presentation. ^[4]

Best starting point

Start coding in this exact order:

1. Create project and paper tables.
2. Build PDF upload.
3. Build parser and metadata extraction.
4. Build chunking and vector indexing.
5. Build semantic search.
6. Build structured extraction.
7. Build comparison UI.

That is the fastest path to a convincing 5-module MVP. ^[4]

Would you like me to give you the **exact code skeleton for backend files first** so you can start implementing immediately?



can we use an alternative instead of sql wouldnt mongodb be a better choice?

Yes — MongoDB is a valid choice for your MVP, and for a 3–4 day build it may actually be faster than SQL if your team is more comfortable with document-style storage. Your synopsis needs a structured and searchable knowledge base, extracted fields, and cross-paper comparison, but it does **not** require a relational database specifically, so MongoDB is acceptable as long as your data remains well organized and traceable. ^[5]

Should you switch?

For this project, MongoDB is better if your immediate goal is speed of implementation and flexibility for storing parsed paper content, extracted JSON fields, chunk metadata, and evolving schemas. The synopsis emphasizes paper metadata, parsed text, extracted methodologies/datasets/metrics, and cross-paper analysis, all of which fit naturally into document-oriented storage. ^[5]

SQL is better if you expect complex joins, strict normalization, and highly relational analytics from day one, but for your first 5 modules that is not the main bottleneck. In your current phase, the real challenge is parsing, retrieval, extraction, and comparison, not relational consistency. ^[5]

My recommendation

For your first presentation, I would recommend this combination:

Storage need	Best choice
User/project/paper metadata	MongoDB
Parsed chunks and extracted fields	MongoDB
Vector search	ChromaDB or Qdrant
PDF files	Local uploads folder

That means:

- MongoDB stores the paper record and extracted structured knowledge.
- Chroma stores embeddings and chunk vectors.
- File system stores PDFs.

This maps cleanly to the synopsis flow of collecting papers, parsing them, creating embeddings, retrieving evidence, and extracting structured fields. ^[5]

Why MongoDB fits well

1. Paper data is naturally document-shaped

Each paper has:

- metadata,
- sections,
- chunks,
- extracted insights,
- status fields,
- maybe citations,
- maybe future comparison summaries.

That is a nested object, which MongoDB handles very naturally. This aligns with your synopsis goal of transforming unstructured papers into organized knowledge objects. ^[5]

2. Extraction schemas may change

Today you may extract:

- methodology,
- dataset,
- metric,
- limitation,
- future work.

Tomorrow you may add:

- baselines,
- architecture,
- experiment settings,
- novelty hints.

MongoDB handles this schema evolution much more easily than SQL, which is useful because your synopsis includes many categories of structured information and you may refine them during implementation. ^[5]

3. Faster MVP development

For a short timeline, MongoDB reduces time spent on:

- joins,
- migrations,
- strict schema redesign,
- multiple related tables.

That helps you focus on the core research pipeline from the methodology phases instead of DB overhead.^[5]

Where SQL is still stronger

MongoDB is not automatically “better”; it is just better for **this phase**. SQL would still be stronger for:

- many-to-many author-paper relationships,
- complex analytics across projects,
- strict citation/reference normalization,
- long-term production reporting.

But your current MVP is not at that level yet. Your current milestone is a structured research corpus with retrieval and comparison.^[5]

Best architecture with MongoDB

Use this:

- FastAPI backend
- MongoDB for app data
- ChromaDB for vector storage
- PyMuPDF for parsing
- sentence-transformers for embeddings
- Next.js frontend^[5]

This keeps your system modular and still faithful to the synopsis phases of acquisition, preprocessing, retrieval, and comparison.^[5]

MongoDB collections

If you switch to MongoDB, use these collections.

1. users

```
{
  "_id": "user_id",
  "name": "Vaishnavi",
  "email": "example@mail.com",
  "password_hash": "...",
  "created_at": "..."
}
```

2. projects

```
{
  "_id": "project_id",
  "user_id": "user_id",
  "title": "ResearchOS Demo",
  "description": "Literature review workspace",
  "created_at": "..."
}
```

3. papers

This is your main collection.

```
{
  "_id": "paper_id",
  "project_id": "project_id",
  "title": "Paper title",
  "authors": ["Author 1", "Author 2"],
  "year": 2024,
  "abstract": "...",
  "keywords": ["rag", "llm", "literature review"],
  "file_path": "uploads/paper1.pdf",
  "status": {
    "uploaded": true,
    "parsed": true,
    "indexed": true,
    "extracted": false
  },
  "sections": [
    {
      "section_title": "Introduction",
      "content": "..."
    }
  ],
  "chunks": [
    {
      "chunk_id": "paper1_chunk_1",
      "section_title": "Introduction",
      "page_no": 2,
      "chunk_index": 1,
      "text": "..."
    }
  ]
}
```

```

    }
  ],
  "extractions": {
    "methodology": {
      "value": "...",
      "evidence_chunk_ids": ["paper1_chunk_3"],
      "confidence": 0.89
    },
    "dataset": {
      "value": "...",
      "evidence_chunk_ids": ["paper1_chunk_7"],
      "confidence": 0.91
    }
  },
  "created_at": "..."
}

```

This structure matches the synopsis very well because it stores metadata, parsed content, extracted knowledge, and evidence linkage together. ^[5]

4. comparisons

Optional for caching comparison results.

```

{
  "_id": "comparison_id",
  "project_id": "project_id",
  "paper_ids": ["p1", "p2", "p3"],
  "comparison_table": {
    "methodology": [...],
    "dataset": [...],
    "metrics": [...]
  },
  "created_at": "..."
}

```

When to embed chunks inside papers

For your MVP, embedding chunks inside the papers collection is fine if the number of papers is small. That will likely be enough for a first-phase demo. ^[5]

If chunk arrays become too large, create a separate chunks collection:

```

{
  "_id": "chunk_id",
  "paper_id": "paper_id",
  "project_id": "project_id",
  "section_title": "Methodology",
  "page_no": 4,
  "chunk_index": 12,
  "text": "...",

```

```
"vector_id": "chromadb_vector_id"
}
```

For 3–4 days, this separate collection is actually cleaner because retrieval and chunk lookup become easier.

Recommended MongoDB design

Use this balance:

Collection	Keep separate?
users	Yes
projects	Yes
papers	Yes
chunks	Yes
comparisons	Optional
extracted summaries	inside paper or separate, both okay

My recommendation:

- keep papers for metadata,
- keep chunks as a separate collection,
- keep extractions inside papers.

That avoids giant paper documents while keeping extracted insights easy to fetch.

Install with MongoDB

Backend install

Use:

```
pip install fastapi uvicorn pymongo motor python-multipart pydantic-settings pymupdf
```

If you want async Mongo usage, use `motor`. That is the best choice with FastAPI.

Files to change

If you use MongoDB, your backend changes like this:

Replace

- SQLAlchemy models
- Alembic migrations
- relational database config

With

- `core/mongodb.py`
- collection helper functions
- Pydantic schemas for validation

Suggested backend structure with MongoDB

```
backend/
├── app/
│   ├── api/
│   │   └── routes/
│   │       ├── auth.py
│   │       ├── projects.py
│   │       ├── papers.py
│   │       ├── search.py
│   │       ├── extraction.py
│   │       └── comparison.py
│   ├── core/
│   │   ├── config.py
│   │   └── mongodb.py
│   ├── schemas/
│   │   ├── user.py
│   │   ├── project.py
│   │   ├── paper.py
│   │   ├── chunk.py
│   │   ├── extraction.py
│   │   └── search.py
│   ├── services/
│   │   ├── pdf_parser.py
│   │   ├── chunker.py
│   │   ├── embedder.py
│   │   ├── vector_store.py
│   │   ├── rag_service.py
│   │   ├── extraction_service.py
│   │   └── comparison_service.py
│   └── main.py
├── uploads/
└── .env
```

MongoDB connection file

Your `core/mongodb.py` should do:

- load `MONGODB_URL`
- create async client
- expose:
 - `db.users`
 - `db.projects`
 - `db.papers`
 - `db.chunks`

That is much simpler than ORM setup.

Query patterns you will need

MongoDB can easily support your first 5 modules with these operations:

- find papers by project
- insert parsed chunks
- update paper status
- save extracted JSON fields
- fetch papers for comparison
- fetch chunks by paper id
- fetch chunks by chunk ids for evidence

These are all straightforward and enough for the structured searchable platform described in the synopsis. ^[5]

Potential downside

The only thing you must be careful about is **not becoming too unstructured**. Since the synopsis emphasizes ethical, explainable, and citation-grounded outputs, your stored data must still be consistent and traceable. ^[5]

So even in MongoDB:

- define clear Pydantic schemas,
- keep fixed extraction field names,
- always store evidence chunk IDs,
- always keep `project_id` and `paper_id` references,
- keep timestamps and status fields.

In other words, use MongoDB flexibly, but not carelessly.

Final answer

So yes — MongoDB is a good choice for your current MVP, and in your case it may even be the better one because your data is document-heavy, nested, and still evolving as you implement parsing, extraction, and comparison. That fits the synopsis requirement of a structured, searchable, citation-grounded research corpus without forcing you into relational complexity too early. ^[5]

My practical recommendation is:

- MongoDB for users, projects, papers, chunks, and extracted knowledge.
- ChromaDB for embeddings and semantic retrieval.
- Local file storage for PDFs. ^[5]

If you want, I can now rewrite the entire implementation plan using MongoDB instead of SQL, including exact files, install commands, schema definitions, and step-by-step setup.

✱

do we start with backend or frontend first

Start with the **backend first**, then build the frontend in parallel once the first 2–3 APIs are stable. Your synopsis is pipeline-driven: collection, parsing, retrieval, extraction, then comparison, so the backend creates the actual research intelligence foundation that every visible feature depends on. ^[6]

Why backend first

ResearchOS is not a UI-first product; its value comes from collecting papers, parsing them, converting them into embeddings, retrieving grounded evidence, and extracting structured fields for analysis. Those steps are explicitly the foundation for later comparison and manuscript support in your methodology, so without backend pipeline outputs, the frontend will just be static screens. ^[6]

Also, your first presentation needs to prove that the system can build a structured research corpus and perform retrieval/comparison, which are backend-heavy capabilities. The synopsis even states that paper collection and preprocessing serve as the foundation for all subsequent analysis. ^[6]

Best approach

Do **backend first, frontend alongside**, but not 100% backend-only for days. The practical order should be:

1. Backend skeleton
2. Upload API
3. Parse/index API
4. Search API

5. Then connect minimal frontend screens to those APIs^[6]

This gives you real progress quickly and avoids wasting time polishing UI before the intelligence layer exists.

What to build first

Phase 1

Start with these backend pieces first:

- project creation
- paper upload
- metadata storage
- PDF parsing
- chunk creation^[6]

These directly map to Phase 1 and Phase 2 of your methodology, which are the base for the rest of the system.^[6]

Phase 2

Then implement:

- embeddings
- vector storage
- semantic retrieval
- grounded response API^[6]

That gets you to the RAG stage described in Phase 3.^[6]

Phase 3

After that, add:

- structured extraction
- comparison endpoint^[6]

That matches the transition from Phase 3 to Phase 4 in the synopsis.^[6]

Frontend timing

Once your upload, paper list, and search APIs are working, start frontend immediately. The frontend can then be built as thin wrappers around working endpoints:

- dashboard for projects,
- upload page for papers,
- paper detail view,

- search page,
- comparison page.^[6]

So the answer is not “backend for 3 days, frontend last minute.” It is:

- **backend first for the foundation**
- **frontend from day 2 onward for integration**

Best team split

If you are 3 people, do this:

Person	Main work
Person 1	Backend APIs, MongoDB, file upload, parsing
Person 2	Embeddings, vector search, extraction/comparison logic
Person 3	Frontend pages and API integration

This works well because the synopsis itself separates data acquisition, preprocessing, retrieval, and analysis into modular stages.^[6]

Recommended order

Use this exact order:

1. Set up backend project.
2. Connect MongoDB.
3. Build project and paper upload APIs.
4. Build parser and chunk pipeline.
5. Build vector indexing and search.
6. Build extraction/comparison APIs.
7. Build frontend around finished APIs.^[6]

Short answer

So yes: **start with backend first**, because ResearchOS depends on a working paper-processing and retrieval pipeline, and your synopsis clearly makes those early phases the foundation for all later modules. But as soon as upload and search endpoints are stable, start frontend integration immediately so you have a visible demo by the end.^[6]

Would you like me to give you the exact day-wise order for backend-first development with MongoDB next?

okay lets do it give me the required file and folder structure for backend

Yes — and since you're going with a backend-first approach for a modular pipeline, your folder structure should reflect the phases in the synopsis: paper collection, preprocessing, retrieval/extraction, and comparison. That way, your codebase will match the project methodology and be easier to explain in review.^[7]

Recommended backend structure

Use this structure for a FastAPI + MongoDB + ChromaDB backend:

```
backend/
├── app/
│   ├── main.py
│   ├── api/
│   │   ├── deps.py
│   │   └── routes/
│   │       ├── auth.py
│   │       ├── projects.py
│   │       ├── papers.py
│   │       ├── parsing.py
│   │       ├── search.py
│   │       ├── extraction.py
│   │       └── comparison.py
│   ├── core/
│   │   ├── config.py
│   │   ├── mongodb.py
│   │   ├── security.py
│   │   └── constants.py
│   ├── schemas/
│   │   ├── auth.py
│   │   ├── common.py
│   │   ├── project.py
│   │   ├── paper.py
│   │   ├── parsing.py
│   │   ├── chunk.py
│   │   ├── search.py
│   │   ├── extraction.py
│   │   └── comparison.py
│   ├── models/
│   │   ├── user_model.py
│   │   ├── project_model.py
│   │   ├── paper_model.py
│   │   ├── chunk_model.py
│   │   └── comparison_model.py
│   └── services/
│       └── auth_service.py
```

```
|
|
|   ├── project_service.py
|   ├── paper_service.py
|   ├── pdf_parser_service.py
|   ├── metadata_service.py
|   ├── chunking_service.py
|   ├── embedding_service.py
|   ├── vector_store_service.py
|   ├── rag_service.py
|   ├── extraction_service.py
|   └── comparison_service.py
|
|   ├── db/
|   |   ├── collections.py
|   |   └── init_db.py
|   |
|   ├── utils/
|   |   ├── file_handler.py
|   |   ├── text_cleaner.py
|   |   ├── section_splitter.py
|   |   ├── id_generator.py
|   |   ├── logger.py
|   |   └── helpers.py
|   |
|   └── prompts/
|       ├── extraction_prompts.py
|       └── rag_prompts.py
|
|   ├── uploads/
|   |   ├── raw_pdfs/
|   |   └── parsed_json/
|   |
|   ├── chroma_db/
|   |
|   ├── tests/
|   |   ├── test_auth.py
|   |   ├── test_projects.py
|   |   ├── test_papers.py
|   |   ├── test_search.py
|   |   └── test_extraction.py
|   |
|   ├── requirements.txt
|   ├── .env
|   ├── .env.example
|   ├── run.py
|   └── README.md
```

Why this structure works

This structure is good because it separates:

- API routes,
- DB/config logic,
- business logic,

- schemas,
- parsing/retrieval services,
- and reusable utilities.

That fits the synopsis well because ResearchOS is described as a modular, phased system rather than one monolithic chatbot flow. The methodology also explicitly moves from acquisition to preprocessing to retrieval to comparison, so your service layer should mirror that order.^[7]

Folder-by-folder explanation

1. `app/main.py`

This is the backend entry point.

It should:

- create the FastAPI app,
- register CORS,
- include all route files,
- expose a health route like `/health`,
- optionally run startup DB initialization.

Keep this file very small.

2. `app/api/routes/`

This folder contains your HTTP endpoints.

Recommended files:

- `auth.py` — signup/login or simple demo auth
- `projects.py` — create/list project workspaces
- `papers.py` — upload papers, fetch paper details
- `parsing.py` — trigger parsing/chunking/indexing
- `search.py` — semantic search and grounded QA
- `extraction.py` — extract structured fields
- `comparison.py` — compare selected papers

These route groups map directly to the core functions mentioned in the synopsis: retrieve, parse, organize, extract, compare.^[7]

3. app/core/

This is for application-level configuration.

`config.py`

Stores:

- MongoDB URL
- database name
- OpenAI key or model provider key
- upload folder path
- Chroma path
- allowed origins

`mongodb.py`

Creates and exposes MongoDB client connection.

`security.py`

Password hashing, JWT utilities, or simple auth helpers.

`constants.py`

Common constants like:

- extraction field names,
- paper status values,
- default chunk size,
- overlap size.

This helps keep your pipeline consistent, which matters because the synopsis stresses structured and explainable outputs. ^[7]

4. app/schemas/

These are Pydantic schemas for request/response validation.

Recommended schema files:

- `auth.py`
- `project.py`
- `paper.py`
- `parsing.py`
- `chunk.py`

- `search.py`
- `extraction.py`
- `comparison.py`

Examples:

- `PaperUploadResponse`
- `ParsePaperResponse`
- `SearchRequest`
- `SearchResponse`
- `ExtractionResponse`
- `ComparisonRequest`

Since MongoDB is schema-flexible, these Pydantic schemas are important so your app still stays disciplined and traceable.^[7]

5. `app/models/`

Even with MongoDB, keep “model” files as document shape definitions or helper constructors.

Recommended files:

- `user_model.py`
- `project_model.py`
- `paper_model.py`
- `chunk_model.py`
- `comparison_model.py`

These are not ORM models like SQLAlchemy models. Instead, they define document-building functions or typed model classes for consistent inserts.

For example:

- `build_paper_document()`
- `build_chunk_document()`
- `build_project_document()`

6. `app/services/`

This is the most important folder. Put all business logic here.

This is where your real project lives.

auth_service.py

- create user
- verify password
- login helper

project_service.py

- create workspace
- list projects

paper_service.py

- save upload
- create paper record
- update paper status

pdf_parser_service.py

- extract raw text from PDF
- identify page text
- section extraction

metadata_service.py

- title extraction
- author extraction
- abstract detection
- keyword extraction

chunking_service.py

- clean text
- split by section
- chunk with overlap

embedding_service.py

- load embedding model
- generate chunk embeddings
- generate query embedding

`vector_store_service.py`

- insert chunk vectors into ChromaDB
- search top-k chunks
- delete vectors if paper deleted

`rag_service.py`

- retrieve relevant chunks
- build context
- call LLM
- return answer with sources

`extraction_service.py`

- extract methodology
- extract dataset
- extract metrics
- extract limitations
- store evidence chunk ids

`comparison_service.py`

- gather structured fields from multiple papers
- align them into a comparison table

These services directly reflect the synopsis phases of preprocessing, retrieval, knowledge extraction, and research comparison.^[7]

7. app/db/

This keeps raw database access logic separate from services.

`collections.py`

Create helpers like:

- `get_users_collection()`
- `get_projects_collection()`
- `get_papers_collection()`
- `get_chunks_collection()`

`init_db.py`

Can create indexes like:

- `projects.user_id`
- `papers.project_id`
- `chunks.paper_id`
- `chunks.project_id`

Even with MongoDB, indexes matter because you'll query papers and chunks often.

8. `app/utils/`

This is for helper functions that should not be mixed into services.

`file_handler.py`

- save file
- validate extension
- create upload path

`text_cleaner.py`

- remove extra whitespace
- normalize unicode
- fix broken line joins

`section_splitter.py`

- detect headings like Abstract, Introduction, Methodology, Conclusion

`id_generator.py`

- create chunk ids
- create paper-safe ids

`logger.py`

- custom logging config

`helpers.py`

- shared utility functions

This keeps your services clean and easier to debug.

9. `app/prompts/`

Since your project involves RAG and structured extraction, keep prompts separate from service logic.

`extraction_prompts.py`

Store prompts for:

- methodology extraction
- dataset extraction
- metric extraction
- limitation extraction

`rag_prompts.py`

Store prompts for:

- grounded answer generation
- evidence-based response format

This is a very good practice because your synopsis stresses ethical, citation-grounded outputs, so prompt design should remain explicit and controlled. ^[7]

10. `uploads/`

This holds uploaded and generated local files.

Recommended subfolders:

- `uploads/raw_pdfs/` — original uploaded PDFs
- `uploads/parsed_json/` — optional parsed paper JSON dumps

Since Phase 1 starts with uploaded PDF documents and metadata extraction, this folder is part of the foundation. ^[7]

11. `chroma_db/`

This holds the persistent Chroma vector database.

The synopsis explicitly says embeddings are stored in a vector database for semantic retrieval, so keep this separate and obvious in your architecture. ^[7]

12. `tests/`

Even if you do only light testing, keep a test folder.

Minimum test files:

- `test_papers.py`

- test_search.py
- test_extraction.py

If your demo fails, small tests for upload/search/extraction will save you.

Minimal version

If you want an even smaller structure for the first 3–4 days, use this reduced version:

```

backend/
├── app/
│   ├── main.py
│   ├── api/
│   │   └── routes/
│   │       ├── projects.py
│   │       ├── papers.py
│   │       ├── search.py
│   │       ├── extraction.py
│   │       └── comparison.py
│   ├── core/
│   │   ├── config.py
│   │   └── mongodb.py
│   ├── schemas/
│   │   ├── project.py
│   │   ├── paper.py
│   │   ├── search.py
│   │   ├── extraction.py
│   │   └── comparison.py
│   ├── services/
│   │   ├── paper_service.py
│   │   ├── pdf_parser_service.py
│   │   ├── chunking_service.py
│   │   ├── embedding_service.py
│   │   ├── vector_store_service.py
│   │   ├── rag_service.py
│   │   ├── extraction_service.py
│   │   └── comparison_service.py
│   ├── utils/
│   │   ├── file_handler.py
│   │   ├── text_cleaner.py
│   │   └── section_splitter.py
│   └── prompts/
│       ├── extraction_prompts.py
│       └── rag_prompts.py
├── uploads/
│   └── raw_pdfs/
├── chroma_db/
├── .env
├── requirements.txt
└── run.py
  
```

This reduced structure is enough for your first 5 modules.

Best file creation order

Create files in this order:

1. main.py
2. core/config.py
3. core/mongodb.py
4. schemas/project.py
5. schemas/paper.py
6. api/routes/projects.py
7. api/routes/papers.py
8. services/paper_service.py
9. services/pdf_parser_service.py
10. services/chunking_service.py
11. services/embedding_service.py
12. services/vector_store_service.py
13. api/routes/search.py
14. services/rag_service.py
15. api/routes/extraction.py
16. services/extraction_service.py
17. api/routes/comparison.py
18. services/comparison_service.py

This order follows the research workflow in your synopsis: collection -> preprocessing -> retrieval -> extraction -> comparison. ^[7]

Naming suggestion

For presentation clarity, name your internal modules like this:

- Paper Acquisition Module
- Document Parsing Module
- Semantic Retrieval Module
- Knowledge Extraction Module
- Comparison Module

These names are easy to map to both code folders and synopsis objectives. ^[7]

Recommendation

Use the **reduced structure first**, not the full enterprise structure. Your real goal in the next few days is to make 5 modules work end to end, not to over-engineer the backend. The synopsis already gives you a natural modular breakdown, so keep the folder structure clean, phase-aligned, and easy to demo.^[7]

Would you like me to give you the **exact code for** `main.py`, `config.py`, **and** `mongodb.py` **next** so you can start creating the backend immediately?



is this just for phase 1

Not exactly – the structure I gave is mainly for your **phase-1 implementation target**, but it already covers more than just Phase 1 of the synopsis. It is designed to support **Phase 1 to Phase 4** of your methodology: paper collection, preprocessing, retrieval/extraction, and comparison.^[8]

What it currently covers

Your synopsis defines the workflow in six phases, starting with paper collection and ending with review simulation and integrity validation. The backend structure I suggested is meant to help you finish the early, foundational phases first, because those are the ones you can realistically demonstrate now and they serve as the base for all later features.^[8]

So right now, that structure supports:

Synopsis phase	Covered by current backend structure?
Phase 1: Paper Collection and Data Acquisition	Yes ^[8]
Phase 2: Document Parsing and Preprocessing	Yes ^[8]
Phase 3: Information Retrieval and Knowledge Extraction	Yes ^[8]
Phase 4: Research Analysis and Comparison	Yes ^[8]
Phase 5: Citation-Grounded Draft Generation	Not fully yet ^[8]
Phase 6: Review Simulation and Integrity Validation	Not fully yet ^[8]

So what is it for?

In simple terms:

- The **current folder structure** is your MVP backend.
- It is enough for the first major demo.
- It is also extensible later for full project completion.^[8]

That means you are not making “throwaway code for only Phase 1.” You are building the **foundation architecture** of the full ResearchOS system, but only implementing the first 4

functional layers now because the synopsis itself makes those the groundwork for all subsequent analysis and assistance.^[8]

What Phase 1 alone would mean

If you were doing **only** Phase 1 from the methodology, then the backend would be much smaller and would include only:

- project creation
- paper upload
- metadata extraction
- file storage
- paper listing^[8]

That would not include:

- chunking,
- embeddings,
- semantic search,
- extraction,
- comparison.^[8]

So the structure I gave is already bigger than just Phase 1.

How to think about it

The best way to think about your backend is:

- **Core base architecture** = long-term full project
- **Implemented modules now** = first 4 phases
- **Future modules later** = draft generation + review/integrity checks^[8]

So yes, the structure is suitable for your first presentation, but it is also the same backend you can continue using later when you add:

- manuscript drafting,
- LaTeX export,
- reviewer simulation,
- unsupported claim detection,
- citation consistency checks.^[8]

What gets added later

When you move to later phases, you will extend the same backend with extra route and service files such as:

For Phase 5

- `api/routes/drafting.py`
- `services/drafting_service.py`
- `schemas/drafting.py`
- `prompts/drafting_prompts.py`

This would support citation-grounded manuscript section generation, which is explicitly described in Phase 5.^[8]

For Phase 6

- `api/routes/review.py`
- `api/routes/integrity.py`
- `services/review_service.py`
- `services/integrity_service.py`
- `schemas/review.py`
- `schemas/integrity.py`

This would support reviewer simulation, unsupported-claim detection, and citation consistency validation, which are described in Phase 6.^[8]

Practical answer

So the answer is:

- **No, it is not just for Phase 1.**
- It is for your **phase-1 presentation milestone**, but technically it covers the architecture needed for **Phase 1 to Phase 4** of the full synopsis.^[8]
- Later, you will **extend** the same structure for Phase 5 and Phase 6 instead of redesigning everything.^[8]

Best decision

That is actually the correct strategy, because your synopsis says Phase 1 and Phase 2 create the foundation for all subsequent analysis, and Phase 3 and Phase 4 create the knowledge layer used before drafting and review. So if you build these early modules well now, the later features can plug into the same backend cleanly.^[8]

Would you like me to now give you **two versions of the folder structure**:

1. **strict MVP for current demo**, and

2. extended final-year version for full project?



yes

Yes — here are the two versions: a **strict MVP backend structure for your current demo**, and an **extended final backend structure for the complete ResearchOS project**. The split follows your synopsis well because the first version covers the early workflow phases, while the second extends into manuscript generation and integrity/review support. ^[9]

MVP backend

This version is for your **current 5-module target** and phase-1 presentation. It supports paper collection, parsing, retrieval, structured extraction, and comparison, which correspond to the first four phases in your methodology. ^[9]

```
backend/
├── app/
│   ├── main.py
│   └── api/
│       ├── routes/
│       │   ├── projects.py
│       │   ├── papers.py
│       │   ├── parsing.py
│       │   ├── search.py
│       │   ├── extraction.py
│       │   └── comparison.py
│       ├── core/
│       │   ├── config.py
│       │   ├── mongodb.py
│       │   └── constants.py
│       ├── schemas/
│       │   ├── project.py
│       │   ├── paper.py
│       │   ├── parsing.py
│       │   ├── search.py
│       │   ├── extraction.py
│       │   └── comparison.py
│       └── services/
│           ├── project_service.py
│           ├── paper_service.py
│           ├── pdf_parser_service.py
│           ├── chunking_service.py
│           ├── embedding_service.py
│           ├── vector_store_service.py
│           ├── rag_service.py
│           ├── extraction_service.py
│           └── comparison_service.py
```

```
|
|
|  └─ utils/
|      └─ file_handler.py
|      └─ text_cleaner.py
|      └─ section_splitter.py
|      └─ helpers.py
|
|  └─ prompts/
|      └─ extraction_prompts.py
|      └─ rag_prompts.py
|
|  └─ uploads/
|      └─ raw_pdfs/
|
|  └─ chroma_db/
|
|  └─ requirements.txt
|  └─ .env
|  └─ .env.example
|  └─ run.py
```

What this MVP covers

This structure is enough for:

- creating projects/workspaces,
- uploading PDFs,
- extracting metadata,
- parsing and chunking papers,
- generating embeddings and storing them in a vector DB,
- semantic search with grounded responses,
- extracting structured fields like methodology, dataset, metric, and limitation,
- and comparing papers side by side. ^[9]

That directly matches the synopsis goals of building a structured, searchable research corpus and enabling cross-paper analysis. ^[9]

Why this is enough now

The methodology says paper collection and preprocessing create the foundation, retrieval/extraction form the knowledge layer, and comparison enables deeper research analysis. That means this MVP already implements the core intelligence path of ResearchOS, even before later writing and review modules are added. ^[9]

Final project backend

This is the **extended version** for the full project after your first milestone. It includes the later phases from the synopsis, especially citation-grounded draft generation and review/integrity validation.^[9]

```
backend/
├── app/
│   ├── main.py
│   └── api/
│       ├── deps.py
│       └── routes/
│           ├── auth.py
│           ├── users.py
│           ├── projects.py
│           ├── papers.py
│           ├── parsing.py
│           ├── search.py
│           ├── extraction.py
│           ├── comparison.py
│           ├── gaps.py
│           ├── drafting.py
│           ├── latex.py
│           ├── diagrams.py
│           ├── review.py
│           ├── integrity.py
│           ├── originality.py
│           └── recommendations.py
├── core/
│   ├── config.py
│   ├── mongodb.py
│   ├── security.py
│   ├── constants.py
│   └── logging_config.py
├── schemas/
│   ├── auth.py
│   ├── common.py
│   ├── user.py
│   ├── project.py
│   ├── paper.py
│   ├── parsing.py
│   ├── chunk.py
│   ├── search.py
│   ├── extraction.py
│   ├── comparison.py
│   ├── gap.py
│   ├── drafting.py
│   ├── latex.py
│   ├── diagram.py
│   ├── review.py
│   ├── integrity.py
│   └── originality.py
```

```
└─ recommendation.py

└─ models/
  └─ user_model.py
  └─ project_model.py
  └─ paper_model.py
  └─ chunk_model.py
  └─ draft_model.py
  └─ review_model.py
  └─ recommendation_model.py

└─ services/
  └─ auth_service.py
  └─ user_service.py
  └─ project_service.py
  └─ paper_service.py
  └─ pdf_parser_service.py
  └─ metadata_service.py
  └─ citation_service.py
  └─ chunking_service.py
  └─ embedding_service.py
  └─ vector_store_service.py
  └─ reranker_service.py
  └─ rag_service.py
  └─ extraction_service.py
  └─ comparison_service.py
  └─ gap_analysis_service.py
  └─ drafting_service.py
  └─ latex_service.py
  └─ diagram_service.py
  └─ review_service.py
  └─ integrity_service.py
  └─ originality_service.py
  └─ recommendation_service.py

└─ db/
  └─ collections.py
  └─ indexes.py
  └─ init_db.py

└─ utils/
  └─ file_handler.py
  └─ text_cleaner.py
  └─ section_splitter.py
  └─ citation_mapper.py
  └─ id_generator.py
  └─ logger.py
  └─ helpers.py
  └─ validators.py

└─ prompts/
  └─ extraction_prompts.py
  └─ rag_prompts.py
  └─ gap_prompts.py
  └─ drafting_prompts.py
  └─ review_prompts.py
```

```
| | | integrity_prompts.py
| | |
| | | workers/
| | |   | queue.py
| | |   | indexing_worker.py
| | |   | extraction_worker.py
| | |   | drafting_worker.py
| | |
| | uploads/
| |   | raw_pdfs/
| |   | parsed_json/
| |   | generated_drafts/
| |   | exports/
| |
| | chroma_db/
| |
| | tests/
| |   | test_auth.py
| |   | test_projects.py
| |   | test_papers.py
| |   | test_search.py
| |   | test_extraction.py
| |   | test_comparison.py
| |   | test_drafting.py
| |   | test_review.py
| |
| | requirements.txt
| | .env
| | .env.example
| | docker-compose.yml
| | Dockerfile
| | run.py
| | README.md
```

How the two versions differ

The MVP version is built for:

- data acquisition,
- parsing,
- retrieval,
- extraction,
- comparison.^[9]

The final version adds modules for:

- research gap analysis,
- grounded draft generation,
- LaTeX export,
- diagram generation,

- reviewer simulation,
- originality and citation consistency checks,
- and recommendation features like journals/conferences. ^[9]

So the difference is basically:

Version	Phases supported
MVP backend	Phase 1 to Phase 4 ^[9]
Final backend	Phase 1 to Phase 6 ^[9]

Which one you should use now

You should **start with the MVP backend structure now**, because it is small enough to finish in time and still aligns directly with the main research workflow in the synopsis. Later, when you move beyond the first presentation, you can extend the exact same structure instead of rebuilding it. ^[9]

Practical recommendation

Use this strategy:

- Create the **MVP structure physically right now**.
- Keep a note of the **final structure** for future expansion.
- Do not create empty files for late-phase features yet, because they will just create noise during current development.

So today, only create folders/files needed for:

- projects,
- papers,
- parsing,
- search,
- extraction,
- comparison. ^[9]

Best next step

The cleanest next step is to create these first files immediately:

- `main.py`
- `core/config.py`
- `core/mongodb.py`
- `api/routes/projects.py`
- `api/routes/papers.py`

- `schemas/project.py`
- `schemas/paper.py`
- `services/paper_service.py`
- `services/pdf_parser_service.py`^[9]

That starts Phase 1 and Phase 2 implementation directly.^[9]

Would you like me to now give you the **exact MVP backend tree** with what each file should contain in one line, so you can create everything quickly?



i have created the mvp backend folder structure

how to proceed further?

Proceed by building the backend in the same order as your synopsis workflow: **ingestion -> parsing -> chunking/embedding -> retrieval -> extraction -> comparison API**. That order matches your project methodology, where paper collection and preprocessing come first, then retrieval and structured knowledge extraction, and only after that the higher-level analysis modules.^[10]

Next milestone

Right now, your goal is not “finish all endpoints,” but “make one paper successfully travel through the pipeline into searchable structured data.” Your synopsis says the system must first collect papers and metadata, then parse PDFs, create machine-readable chunks and embeddings, and use those as the base for retrieval and extraction.^[10]

So the next concrete milestone is:

- upload 1 PDF,
- save metadata,
- extract text by section,
- create chunks,
- store embeddings,
- run one query,
- return cited evidence.^[10]

If that works end-to-end, your backend foundation is strong enough for the first presentation.

Build order

Use this exact backend sequence:

Step	What to build now	Why first
1	Config + DB connection	Everything depends on it
2	Paper upload API	Matches paper collection phase [10]
3	PDF parser service	Matches parsing phase [10]
4	Chunking service	Needed before embeddings [10]
5	Embedding + vector storage	Needed for semantic retrieval [10]
6	Retrieval API	First visible intelligence feature [10]
7	Structured extraction API	Matches knowledge extraction phase [10]
8	Comparison API	Matches cross-paper analysis goal [10]

Do not start with reviewer simulation or draft generation yet, because your own methodology places those later, after retrieval and extraction are ready. [\[10\]](#)

Immediate folder plan

If you already have an MVP backend folder structure, fill it with these modules first:

- `app/main.py`
- `app/core/config.py`
- `app/db/session.py`
- `app/models/`
- `app/schemas/`
- `app/api/routes/`
- `app/services/ingestion.py`
- `app/services/parser.py`
- `app/services/chunker.py`
- `app/services/embedder.py`
- `app/services/retriever.py`
- `app/services/extractor.py`
- `app/services/comparator.py`
- `app/utils/`
- `app/workers/` if async [\[10\]](#)

A clean split is important because your project is inherently modular: collection, preprocessing, retrieval, extraction, comparison, then validation. [\[10\]](#)

Database first

Before writing many APIs, define the minimum schema. Your synopsis repeatedly emphasizes a structured and searchable knowledge base with extracted metadata and traceable evidence. [\[10\]](#)

Start with these tables or collections:

`papers`

- `id`
- `title`
- `authors`
- `year`
- `abstract`
- `filename`
- `uploaded_at`
- `status`

`sections`

- `id`
- `paper_id`
- `section_name`
- `content`
- `page_start`
- `page_end`

`chunks`

- `id`
- `paper_id`
- `section_id`
- `chunk_text`
- `chunk_index`
- `page_no`
- `embedding_id`

extractions

- id
- paper_id
- field_name
- field_value
- evidence_chunk_ids
- confidence

queries **optional**

- id
- query_text
- response
- retrieved_chunk_ids ^[10]

This schema is enough for your first working backend.

First APIs to implement

Start with only 5 backend APIs. That is enough for a serious MVP demo.

1. Upload paper

POST /papers/upload

Input:

- PDF file

Output:

- paper id
- upload status
- filename ^[10]

This supports the paper collection phase from your methodology. ^[10]

2. Parse paper

POST /papers/{paper_id}/parse

This should:

- extract raw text,
- detect sections,
- store metadata,

- update status to parsed. ^[10]

3. Index paper

POST /papers/{paper_id}/index

This should:

- chunk text,
- create embeddings,
- push to vector DB,
- update status to indexed. ^[10]

4. Search/query

POST /search/query

Input:

- question
- optional paper ids

Output:

- top chunks
- source paper names
- page numbers
- answer summary ^[10]

This demonstrates the retrieval-augmented evidence layer described in the synopsis. ^[10]

5. Extract structured insights

POST /papers/{paper_id}/extract

This should pull:

- methodology
- dataset
- metrics
- baselines
- limitations
- future work ^[10]

That aligns directly with the extraction goals in your synopsis. ^[10]

What to code first

1. Make the upload flow work

Do this before anything else:

- accept PDF with FastAPI,
- save to `/storage/papers/`,
- create a DB entry with `status="uploaded"`.^[10]

Even if parsing fails later, you already have the first pipeline stage complete.

2. Implement parser service

Use a simple parser first. Your synopsis says the system begins by collecting papers and extracting metadata and textual content from PDFs.^[10]

Your `parser.py` should expose something like:

- `extract_text(pdf_path)`
- `extract_metadata(text or pdf)`
- `split_into_sections(text)`

Keep it basic:

- detect headings with regex or heuristic matching,
- separate known sections like Abstract, Introduction, Methodology, Results, Conclusion,
- fallback to raw page text if section detection is weak.

3. Implement chunker

Your synopsis explicitly says text should be segmented into meaningful chunks and converted into embeddings.^[10]

So your chunker should:

- chunk by section if possible,
- keep overlap,
- attach metadata like page number, paper id, section name.^[10]

Example rule:

- chunk size: 500–800 words or token equivalent,
- overlap: 80–120 words,
- never lose source linkage.

4. Add embeddings and vector DB

This is the point where your paper becomes searchable. Your synopsis says embeddings should be stored in a vector database for efficient semantic retrieval. ^[10]

Minimal implementation:

- choose one embedding model,
- store vectors in Chroma or Qdrant,
- save chunk metadata with each vector. ^[10]

At this stage, one paper is enough. Do not optimize for scale yet.

Retrieval first, not generation first

Your biggest temptation will be to build a chat endpoint quickly. But the correct order is retrieval first, then response generation. The synopsis makes grounding and citation traceability central to the system's purpose. ^[10]

So your search endpoint should first return:

- top matching chunks,
- paper title,
- section,
- page number,
- snippet. ^[10]

Only after that add a summarizer on top of retrieved chunks. This keeps the project aligned with its ethical, evidence-based design. ^[10]

Structured extraction plan

After retrieval works, build one extractor that returns structured JSON. Your synopsis specifically requires extracting methodologies, architectures, datasets, baselines, metrics, limitations, and future directions. ^[10]

Start with this output:

```
{
  "paper_id": "123",
  "methodology": "...",
  "dataset": [...],
  "metrics": [...],
  "baselines": [...],
  "limitations": [...],
  "future_work": [...]}

```

Important rule: each field should also store evidence chunk ids, because your project promises traceable evidence for generated insights. ^[10]

Comparison should come after extraction

Do not manually compare raw papers first. Once extraction works for multiple papers, comparison becomes much easier.

Build:

POST /compare

Input:

- list of paper ids

Output:

- methodology comparison
- dataset comparison
- metric comparison
- limitations comparison ^[10]

This directly fulfills your stated objective of enabling structured cross-paper comparison. ^[10]

Suggested coding sequence for the next 2 days

Today

- Set up DB models.
- Set up upload route.
- Save PDFs locally.
- Create status pipeline: uploaded -> parsed -> indexed -> extracted. ^[10]

Tomorrow morning

- Build parser service.
- Store extracted text and sections.
- Test on 1-2 papers. ^[10]

Tomorrow evening

- Build chunking + embeddings.
- Push vectors to DB.
- Create search route. ^[10]

Next day

- Build extraction route.
- Store extracted fields.
- Add basic comparison route.^[10]

That gives you a real backend that supports the first half of the project.

Keep the MVP strict

For now, your backend should answer only these backend questions well:

- Can I upload a paper?
- Can I parse and store it?
- Can I search semantically through it?
- Can I extract structured research facts?
- Can I compare 2 or more papers?^[10]

If yes, you have already implemented the most foundational parts of ResearchOS as defined in your synopsis.^[10]

Engineering approach

Use a pipeline status model so each paper moves through stages cleanly:

- uploaded
- parsed
- chunked
- indexed
- extracted
- failed^[10]

This helps both backend debugging and presentation, because you can visibly show each phase from your methodology being executed.^[10]

Also keep each service isolated:

- parser should not know about API,
- embedder should not know about frontend,
- retriever should read only indexed chunks,
- extractor should use retrieved or section text.^[10]

That modularity will help a lot during debugging.

Best practical advice

The most useful next move is to make **one complete happy path** work before adding more features. Your synopsis is strong because it describes a pipeline from unstructured papers to structured, explainable knowledge, so proving one full pass through that pipeline is more valuable than half-building many advanced modules. ^[10]

So after folder structure, your next exact order should be:

1. DB models.
2. Upload endpoint.
3. Parser service.
4. Chunker.
5. Embeddings + vector DB.
6. Search endpoint.
7. Extraction endpoint.
8. Comparison endpoint. ^[10]

Would you like me to give you a **concrete FastAPI backend checklist with sample file contents for each module** next?

✱

give me the procedure and code for this much

Today

Set up DB models.

Set up upload route.

Save PDFs locally.

Create status pipeline: uploaded -> parsed -> indexed -> extracted. [Synopsis-content.pdf](#)

Yes — for today's scope, you should implement a clean backend slice that covers: database models, upload API, local PDF storage, and a paper lifecycle status pipeline. This directly matches your methodology's first foundation step, where papers are collected from uploaded PDFs, metadata is stored into a structured corpus, and the system prepares them for later parsing, indexing, and extraction. ^[11]

Procedure

Build this in 6 steps so you finish a working backend slice by tonight:

1. Set up FastAPI + SQLAlchemy + SQLite/Postgres connection.
2. Create a Paper model with file metadata and processing status.
3. Create enums for lifecycle stages: uploaded, parsed, indexed, extracted. ^[11]
4. Build an upload endpoint that saves PDFs locally and inserts a DB row.

5. Build simple status update endpoints or service methods.

6. Add a list/get endpoint so you can verify uploads from Postman or Swagger.

For the first phase, that is enough because your synopsis says paper collection and data acquisition create the foundation for all later analysis modules.^[11]

Suggested structure

Use this backend layout:

```
backend/
├── app/
│   ├── api/
│   │   └── routes/
│   │       └── papers.py
│   ├── core/
│   │   └── config.py
│   ├── db/
│   │   └── session.py
│   ├── models/
│   │   └── paper.py
│   ├── schemas/
│   │   └── paper.py
│   ├── services/
│   │   └── paper_service.py
│   ├── utils/
│   │   └── file_storage.py
│   └── main.py
├── storage/
│   └── papers/
├── requirements.txt
└── .env
```

This structure keeps collection, storage, and lifecycle handling modular, which fits the phased design of ResearchOS.^[11]

Install dependencies

Create `requirements.txt`:

```
fastapi
uvicorn
sqlalchemy
pydantic
pydantic-settings
python-multipart
```

Install:

```
pip install -r requirements.txt
```

Config

app/core/config.py

```
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    APP_NAME: str = "ResearchOS Backend"
    DATABASE_URL: str = "sqlite:///./researchos.db"
    STORAGE_DIR: str = "storage/papers"

    class Config:
        env_file = ".env"

settings = Settings()
```

.env

```
DATABASE_URL=sqlite:///./researchos.db
STORAGE_DIR=storage/papers
```

Database session

app/db/session.py

```
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, declarative_base
from app.core.config import settings

connect_args = {"check_same_thread": False} if settings.DATABASE_URL.startswith("sql:

engine = create_engine(settings.DATABASE_URL, connect_args=connect_args)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

Paper model

Your synopsis says metadata such as title, authors, publication year, abstract, keywords, and references should be extracted and stored to build a structured corpus. For **today**, start smaller: filename, stored path, upload metadata, and processing status. You can add abstract/authors later when parsing begins. ^[11]

app/models/paper.py

```
import enum
from sqlalchemy import Column, Integer, String, DateTime, Enum, Text
from sqlalchemy.sql import func
from app.db.session import Base

class PaperStatus(str, enum.Enum):
    uploaded = "uploaded"
    parsed = "parsed"
    indexed = "indexed"
    extracted = "extracted"

class Paper(Base):
    __tablename__ = "papers"

    id = Column(Integer, primary_key=True, index=True)
    original_filename = Column(String(255), nullable=False)
    stored_filename = Column(String(255), nullable=False, unique=True)
    file_path = Column(String(500), nullable=False)
    content_type = Column(String(100), nullable=True)
    file_size = Column(Integer, nullable=True)

    title = Column(String(500), nullable=True)
    authors = Column(Text, nullable=True)
    publication_year = Column(Integer, nullable=True)
    abstract = Column(Text, nullable=True)

    status = Column(Enum(PaperStatus), default=PaperStatus.uploaded, nullable=False)

    created_at = Column(DateTime(timezone=True), server_default=func.now())
    updated_at = Column(DateTime(timezone=True), server_default=func.now(), onupdate=func.now())
```

Schemas

app/schemas/paper.py

```
from datetime import datetime
from typing import Optional
from pydantic import BaseModel
from app.models.paper import PaperStatus
```

```

class PaperResponse(BaseModel):
    id: int
    original_filename: str
    stored_filename: str
    file_path: str
    content_type: Optional[str] = None
    file_size: Optional[int] = None
    title: Optional[str] = None
    authors: Optional[str] = None
    publication_year: Optional[int] = None
    abstract: Optional[str] = None
    status: PaperStatus
    created_at: datetime
    updated_at: Optional[datetime] = None

    class Config:
        from_attributes = True

class PaperStatusUpdate(BaseModel):
    status: PaperStatus

```

Local file storage helper

app/utils/file_storage.py

```

import os
import uuid
from fastapi import UploadFile
from app.core.config import settings

def ensure_storage_dir():
    os.makedirs(settings.STORAGE_DIR, exist_ok=True)

def save_pdf_locally(file: UploadFile) -> tuple[str, str, int]:
    ensure_storage_dir()

    ext = os.path.splitext(file.filename)[1] or ".pdf"
    stored_filename = f"{uuid.uuid4()}{ext}"
    file_path = os.path.join(settings.STORAGE_DIR, stored_filename)

    contents = file.file.read()
    with open(file_path, "wb") as f:
        f.write(contents)

    return stored_filename, file_path, len(contents)

```

Service layer

app/services/paper_service.py

```
from fastapi import HTTPException, UploadFile
from sqlalchemy.orm import Session
from app.models.paper import Paper, PaperStatus
from app.utils.file_storage import save_pdf_locally

def create_paper_record(db: Session, file: UploadFile) -> Paper:
    if file.content_type not in ["application/pdf", "application/octet-stream"]:
        raise HTTPException(status_code=400, detail="Only PDF files are allowed")

    stored_filename, file_path, file_size = save_pdf_locally(file)

    paper = Paper(
        original_filename=file.filename,
        stored_filename=stored_filename,
        file_path=file_path,
        content_type=file.content_type,
        file_size=file_size,
        status=PaperStatus.uploaded,
    )
    db.add(paper)
    db.commit()
    db.refresh(paper)
    return paper

def get_all_papers(db: Session):
    return db.query(Paper).order_by(Paper.created_at.desc()).all()

def get_paper_by_id(db: Session, paper_id: int):
    paper = db.query(Paper).filter(Paper.id == paper_id).first()
    if not paper:
        raise HTTPException(status_code=404, detail="Paper not found")
    return paper

def update_paper_status(db: Session, paper_id: int, new_status: PaperStatus):
    paper = get_paper_by_id(db, paper_id)
    paper.status = new_status
    db.commit()
    db.refresh(paper)
    return paper
```

API routes

app/api/routes/papers.py

```
from fastapi import APIRouter, Depends, File, UploadFile
from sqlalchemy.orm import Session

from app.db.session import get_db
from app.schemas.paper import PaperResponse, PaperStatusUpdate
from app.services.paper_service import (
    create_paper_record,
    get_all_papers,
    get_paper_by_id,
    update_paper_status,
)

router = APIRouter(prefix="/papers", tags=["papers"])

@router.post("/upload", response_model=PaperResponse)
def upload_paper(file: UploadFile = File(...), db: Session = Depends(get_db)):
    return create_paper_record(db, file)

@router.get("/", response_model=list[PaperResponse])
def list_papers(db: Session = Depends(get_db)):
    return get_all_papers(db)

@router.get("/{paper_id}", response_model=PaperResponse)
def get_paper(paper_id: int, db: Session = Depends(get_db)):
    return get_paper_by_id(db, paper_id)

@router.patch("/{paper_id}/status", response_model=PaperResponse)
def patch_paper_status(paper_id: int, payload: PaperStatusUpdate, db: Session = Depends(get_db)):
    return update_paper_status(db, paper_id, payload.status)
```

Main app

app/main.py

```
from fastapi import FastAPI
from app.db.session import Base, engine
from app.api.routes.papers import router as papers_router

Base.metadata.create_all(bind=engine)

app = FastAPI(title="ResearchOS Backend MVP")

app.include_router(papers_router)
```

```
@app.get("/")
def root():
    return {
        "message": "ResearchOS backend running",
        "stages": ["uploaded", "parsed", "indexed", "extracted"]
    }
```

Run the server

From backend/:

```
uvicorn app.main:app --reload
```

Open:

- Swagger UI: <http://127.0.0.1:8000/docs>

How to test today's scope

1. Upload a PDF

Use `/papers/upload` in Swagger. This implements the paper collection step from your methodology using user-uploaded PDF documents.^[11]

Expected response:

```
{
  "id": 1,
  "original_filename": "paper1.pdf",
  "stored_filename": "c2c7....pdf",
  "file_path": "storage/papers/c2c7....pdf",
  "content_type": "application/pdf",
  "file_size": 245123,
  "title": null,
  "authors": null,
  "publication_year": null,
  "abstract": null,
  "status": "uploaded",
  "created_at": "...",
  "updated_at": "..."
}
```

2. Verify file saved locally

Check `storage/papers/`. This satisfies the “save PDFs locally” requirement and gives you a persistent corpus foundation.^[11]

3. List all papers

Call GET `/papers/`.

4. Update status manually

Call PATCH `/papers/{paper_id}/status` with:

```
{
  "status": "parsed"
}
```

Then test:

- `indexed`
- `extracted`

This lets you demonstrate the lifecycle pipeline even before parser and embedding services are fully built. That is useful because your synopsis clearly defines later phases built on top of this pipeline. ^[11]

Why this is enough for today

This backend slice already satisfies the first operational part of your methodology:

- user-uploaded PDFs are accepted,
- metadata placeholders exist in a structured corpus,
- the system has a persistent record of each paper,
- lifecycle stages are ready for later parsing, indexing, and extraction services. ^[11]

That is a correct implementation of the foundation layer your synopsis describes before moving into document parsing, chunking, embeddings, retrieval, and knowledge extraction. ^[11]

What to do tomorrow

After this, your next step is:

1. Add `parse_paper(paper_id)` service.
2. Extract text from PDF.
3. Fill title, abstract, maybe authors.
4. Move status from uploaded to parsed. ^[11]

Then after that:

- chunk text,
- create embeddings,
- store vector references,

- move status to indexed.^[11]

One improvement I recommend

Add one more field now so your demo looks better:

In Paper model add:

```
error_message = Column(Text, nullable=True)
```

This will help if parsing/indexing fails, and it makes your status pipeline more realistic for presentation.

Quick presentation line

You can describe today's progress as:

“We have implemented the research paper acquisition layer, persistent paper storage, and a processing-state pipeline that prepares uploaded PDFs for later parsing, indexing, and structured extraction.”^[11]

If you want, next I can give you **Day 2 code** for:

- PDF text extraction,
- metadata extraction,
- auto-update status to parsed,
- and section-wise parsing.

✱

we're doing mongodb not sqlalchemy and postgres

Perfect — MongoDB is completely fine for this stage. Your synopsis only requires that uploaded PDFs and extracted metadata be stored in a **structured research corpus** that becomes the foundation for later parsing, retrieval, and analysis; it does not require a relational database specifically. So for today, you can implement the same pipeline using **FastAPI + MongoDB**, with paper documents carrying lifecycle status values like uploaded, parsed, indexed, and extracted.^[12]

What to build today

For your MongoDB-based MVP, implement exactly these things:

1. MongoDB connection.
2. papers collection.
3. Upload API.
4. Local PDF storage.

5. Status pipeline fields inside each paper document.

6. List/get/update-status APIs.

That already satisfies the “paper collection and data acquisition” layer from your methodology, where uploaded research papers and their metadata are stored in a structured corpus for later processing.^[12]

Folder structure

Use this:

```
backend/  
├── app/  
│   ├── api/  
│   │   └── routes/  
│   │       └── papers.py  
│   ├── core/  
│   │   └── config.py  
│   ├── db/  
│   │   └── mongodb.py  
│   ├── models/  
│   │   └── paper.py  
│   ├── schemas/  
│   │   └── paper.py  
│   ├── services/  
│   │   └── paper_service.py  
│   ├── utils/  
│   │   └── file_storage.py  
│   └── main.py  
├── storage/  
│   └── papers/  
├── requirements.txt  
└── .env
```

This is enough for a clean phase-1 backend slice.^[12]

Install dependencies

requirements.txt

```
fastapi  
uvicorn  
pymongo  
pydantic  
pydantic-settings  
python-multipart
```

Install:

```
pip install -r requirements.txt
```

Environment config

`.env`

```
MONGODB_URL=mongodb://localhost:27017
DATABASE_NAME=researchos_db
STORAGE_DIR=storage/papers
```

`app/core/config.py`

```
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    MONGODB_URL: str = "mongodb://localhost:27017"
    DATABASE_NAME: str = "researchos_db"
    STORAGE_DIR: str = "storage/papers"

    class Config:
        env_file = ".env"

settings = Settings()
```

MongoDB connection

`app/db/mongodb.py`

```
from pymongo import MongoClient
from app.core.config import settings

client = MongoClient(settings.MONGODB_URL)
db = client[settings.DATABASE_NAME]

papers_collection = db["papers"]
```

Paper schema design

Your synopsis says Phase 1 should store metadata such as title, authors, publication year, abstract, keywords, and references to form a structured corpus. For today, store the core file data plus placeholders for metadata you will populate after parsing. ^[12]

app/models/paper.py

```
from enum import Enum

class PaperStatus(str, Enum):
    uploaded = "uploaded"
    parsed = "parsed"
    indexed = "indexed"
    extracted = "extracted"
```

Pydantic schemas

app/schemas/paper.py

```
from typing import Optional, List
from pydantic import BaseModel, Field

class PaperStatusUpdate(BaseModel):
    status: str

class PaperResponse(BaseModel):
    id: str = Field(alias="_id")
    original_filename: str
    stored_filename: str
    file_path: str
    content_type: Optional[str] = None
    file_size: Optional[int] = None

    title: Optional[str] = None
    authors: Optional[List[str]] = None
    publication_year: Optional[int] = None
    abstract: Optional[str] = None
    keywords: Optional[List[str]] = None
    references: Optional[List[str]] = None

    status: str
    created_at: str
    updated_at: str

    class Config:
        populate_by_name = True

class PaperCreateResponse(BaseModel):
    message: str
    paper_id: str
    status: str
    filename: str
```

File storage utility

app/utils/file_storage.py

```
import os
import uuid
from fastapi import UploadFile
from app.core.config import settings

def ensure_storage_dir():
    os.makedirs(settings.STORAGE_DIR, exist_ok=True)

def save_pdf_locally(file: UploadFile):
    ensure_storage_dir()

    ext = os.path.splitext(file.filename)[1] or ".pdf"
    stored_filename = f"{uuid.uuid4()}{ext}"
    file_path = os.path.join(settings.STORAGE_DIR, stored_filename)

    content = file.file.read()
    with open(file_path, "wb") as f:
        f.write(content)

    return stored_filename, file_path, len(content)
```

Service layer

app/services/paper_service.py

```
from datetime import datetime
from fastapi import HTTPException, UploadFile
from bson import ObjectId

from app.db.mongodb import papers_collection
from app.utils.file_storage import save_pdf_locally
from app.models.paper import PaperStatus

def serialize_paper(doc):
    doc["_id"] = str(doc["_id"])
    return doc

def create_paper_record(file: UploadFile):
    if not file.filename.lower().endswith(".pdf"):
        raise HTTPException(status_code=400, detail="Only PDF files are allowed")

    stored_filename, file_path, file_size = save_pdf_locally(file)
    now = datetime.utcnow().isoformat()
```

```

paper_doc = {
    "original_filename": file.filename,
    "stored_filename": stored_filename,
    "file_path": file_path,
    "content_type": file.content_type,
    "file_size": file_size,

    "title": None,
    "authors": [],
    "publication_year": None,
    "abstract": None,
    "keywords": [],
    "references": [],

    "status": PaperStatus.uploaded.value,
    "created_at": now,
    "updated_at": now,
    "error_message": None
}

result = papers_collection.insert_one(paper_doc)

return {
    "message": "Paper uploaded successfully",
    "paper_id": str(result.inserted_id),
    "status": PaperStatus.uploaded.value,
    "filename": file.filename
}

def list_papers():
    papers = papers_collection.find().sort("created_at", -1)
    return [serialize_paper(p) for p in papers]

def get_paper_by_id(paper_id: str):
    if not ObjectId.is_valid(paper_id):
        raise HTTPException(status_code=400, detail="Invalid paper ID")

    paper = papers_collection.find_one({"_id": ObjectId(paper_id)})
    if not paper:
        raise HTTPException(status_code=404, detail="Paper not found")

    return serialize_paper(paper)

def update_paper_status(paper_id: str, status: str):
    if status not in [s.value for s in PaperStatus]:
        raise HTTPException(status_code=400, detail="Invalid status value")

    if not ObjectId.is_valid(paper_id):
        raise HTTPException(status_code=400, detail="Invalid paper ID")

    existing = papers_collection.find_one({"_id": ObjectId(paper_id)})
    if not existing:
        raise HTTPException(status_code=404, detail="Paper not found")

```

```
papers_collection.update_one(
    {"_id": ObjectId(paper_id)},
    {
        "$set": {
            "status": status,
            "updated_at": datetime.utcnow().isoformat()
        }
    }
)

updated = papers_collection.find_one({"_id": ObjectId(paper_id)})
return serialize_paper(updated)
```

API routes

app/api/routes/papers.py

```
from fastapi import APIRouter, File, UploadFile
from app.schemas.paper import PaperStatusUpdate
from app.services.paper_service import (
    create_paper_record,
    list_papers,
    get_paper_by_id,
    update_paper_status
)

router = APIRouter(prefix="/papers", tags=["papers"])

@router.post("/upload")
def upload_paper(file: UploadFile = File(...)):
    return create_paper_record(file)

@router.get("/")
def get_all_papers():
    return list_papers()

@router.get("/{paper_id}")
def get_paper(paper_id: str):
    return get_paper_by_id(paper_id)

@router.patch("/{paper_id}/status")
def patch_paper_status(paper_id: str, payload: PaperStatusUpdate):
    return update_paper_status(paper_id, payload.status)
```

Main app

app/main.py

```
from fastapi import FastAPI
from app.api.routes.papers import router as papers_router

app = FastAPI(title="ResearchOS MongoDB Backend")

app.include_router(papers_router)

@app.get("/")
def root():
    return {
        "message": "ResearchOS backend running",
        "pipeline": ["uploaded", "parsed", "indexed", "extracted"]
    }
```

Run it

```
uvicorn app.main:app --reload
```

Open Swagger:

<http://127.0.0.1:8000/docs>

How to test

Upload a paper

Use POST `/papers/upload`. This implements the user-uploaded PDF entry point described in Phase 1 of your methodology. ^[12]

Expected response:

```
{
  "message": "Paper uploaded successfully",
  "paper_id": "688b7a2d9e4d8b2adf9d1234",
  "status": "uploaded",
  "filename": "sample.pdf"
}
```

List papers

Use GET `/papers/`.

Example document:

```
{
  "_id": "688b7a2d9e4d8b2adf9d1234",
  "original_filename": "sample.pdf",
  "stored_filename": "0b17e7f8-d43c-4f4f-9a5d-123456789abc.pdf",
  "file_path": "storage/papers/0b17e7f8-d43c-4f4f-9a5d-123456789abc.pdf",
  "content_type": "application/pdf",
  "file_size": 145672,
  "title": null,
  "authors": [],
  "publication_year": null,
  "abstract": null,
  "keywords": [],
  "references": [],
  "status": "uploaded",
  "created_at": "2026-07-31T23:11:00.000000",
  "updated_at": "2026-07-31T23:11:00.000000",
  "error_message": null
}
```

Update status manually

Use PATCH `/papers/{paper_id}/status`

```
{
  "status": "parsed"
}
```

Then test:

- `indexed`
- `extracted`

This gives you a visible backend lifecycle even before the parser and vector DB are implemented, which is useful because later phases in your synopsis depend on this progression.^[12]

Recommended MongoDB document shape

This is the paper document shape I recommend for the whole project:

```
{
  "_id": "ObjectId",
  "original_filename": "paper.pdf",
  "stored_filename": "uuid.pdf",
  "file_path": "storage/papers/uuid.pdf",
  "content_type": "application/pdf",
  "file_size": 123456,

  "title": null,
  "authors": [],
  "publication_year": null,
}
```

```

"abstract": null,
"keywords": [],
"references": [],

"sections": [],
"chunks": [],
"vector_ids": [],

"status": "uploaded",
"error_message": null,
"created_at": "ISO timestamp",
"updated_at": "ISO timestamp"
}

```

This matches your synopsis well because the project needs to move from raw uploaded PDFs to parsed text, chunked machine-readable content, and then searchable knowledge. ^[12]

One small improvement

Add lifecycle validation so the status does not jump incorrectly.

For example:

- uploaded -> parsed
- parsed -> indexed
- indexed -> extracted

That mirrors your phased methodology more honestly than allowing any random transition. ^[12]

Here is the improved function:

replace update_paper_status

```

from app.models.paper import PaperStatus

VALID_TRANSITIONS = {
    PaperStatus.uploaded.value: [PaperStatus.parsed.value],
    PaperStatus.parsed.value: [PaperStatus.indexed.value],
    PaperStatus.indexed.value: [PaperStatus.extracted.value],
    PaperStatus.extracted.value: []
}

def update_paper_status(paper_id: str, status: str):
    if status not in [s.value for s in PaperStatus]:
        raise HTTPException(status_code=400, detail="Invalid status value")

    if not ObjectId.is_valid(paper_id):
        raise HTTPException(status_code=400, detail="Invalid paper ID")

    existing = papers_collection.find_one({"_id": ObjectId(paper_id)})
    if not existing:

```

```

        raise HTTPException(status_code=404, detail="Paper not found")

    current_status = existing["status"]
    allowed = VALID_TRANSITIONS.get(current_status, [])

    if status not in allowed:
        raise HTTPException(
            status_code=400,
            detail=f"Invalid status transition from '{current_status}' to '{status}'"
        )

    papers_collection.update_one(
        {"_id": ObjectId(paper_id)},
        {
            "$set": {
                "status": status,
                "updated_at": datetime.utcnow().isoformat()
            }
        }
    )

    updated = papers_collection.find_one({"_id": ObjectId(paper_id)})
    return serialize_paper(updated)

```

What this gives you by tonight

By the end of this step, you will have:

- a working FastAPI backend,
- a MongoDB papers collection,
- local PDF persistence,
- paper records stored as a structured corpus,
- a lifecycle pipeline ready for parsing and indexing. ^[12]

That is a valid implementation of the first phase of ResearchOS, because your methodology begins with uploaded paper acquisition and structured storage before moving into parsing, chunking, embeddings, and retrieval. ^[12]

Next step after this

Tomorrow, build:

- PDF text extraction,
- section detection,
- metadata filling,
- automatic move from uploaded to parsed. ^[12]

Then after that:

- chunks,

- embeddings,
- vector DB,
- parsed -> indexed.^[12]

I can now give you the MongoDB version of Day 2 code for PDF parsing and auto-status update.

✱

i have created this backend folder structure

```
backend/
├── app/
│   ├── api/
│   │   ├── routes/
│   │   └── papers.py
│   ├── core/
│   │   └── config.py
│   ├── db/
│   │   └── session.py
│   ├── models/
│   │   └── paper.py
│   ├── schemas/
│   │   └── paper.py
│   ├── services/
│   │   └── paper_service.py
│   ├── utils/
│   │   └── file_storage.py
│   └── main.py
├── storage/
│   └── papers/
├── requirements.txt
└── .env
```

give me code and procedure for this much part

Today

Set up DB models.

Set up upload route.

Save PDFs locally.

Create status pipeline: uploaded -> parsed -> indexed -> extracted.

Great — with that folder structure and MongoDB, you can finish **today's whole backend slice** cleanly. This slice covers the paper acquisition foundation your synopsis describes: collect uploaded PDFs, store them in a structured corpus, and track their lifecycle so later parsing, indexing, and extraction can build on top of it.^[13]

What you are implementing today

Your target for today should be exactly this:

- connect FastAPI to MongoDB,
- define the papers document structure,
- upload a PDF,
- save it locally in `storage/papers/`,
- create a DB record,
- maintain lifecycle status: `uploaded -> parsed -> indexed -> extracted`.^[13]

This is enough to demonstrate the first operational phase of ResearchOS, because the methodology begins with paper collection and structured storage before moving into document parsing, embeddings, retrieval, and knowledge extraction.^[13]

Procedure

Follow this order:

1. Add dependencies.
2. Set environment variables.
3. Create MongoDB session/connection.
4. Define status enum and response schemas.
5. Implement local file saving utility.
6. Implement `paper_service.py`.
7. Implement `papers.py` routes.
8. Start FastAPI and test through Swagger/Postman.

Dependencies

`requirements.txt`

```
fastapi
uvicorn
pymongo
pydantic
pydantic-settings
python-multipart
dnspython
```

Install:

```
pip install -r requirements.txt
```

dnspython helps if you later use MongoDB Atlas.

Environment

`.env`

```
MONGODB_URL=mongodb://localhost:27017
DATABASE_NAME=researchos_db
STORAGE_DIR=storage/papers
```

Config

`app/core/config.py`

```
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    MONGODB_URL: str = "mongodb://localhost:27017"
    DATABASE_NAME: str = "researchos_db"
    STORAGE_DIR: str = "storage/papers"

    class Config:
        env_file = ".env"

settings = Settings()
```

MongoDB connection

Since your folder has `app/db/session.py`, use it for MongoDB session setup.

`app/db/session.py`

```
from pymongo import MongoClient
from app.core.config import settings

client = MongoClient(settings.MONGODB_URL)
database = client[settings.DATABASE_NAME]

papers_collection = database["papers"]
```

This gives you a single papers collection, which is enough for the paper acquisition phase described in the synopsis. ^[13]

Model enum

Since MongoDB is schema-flexible, your `models/paper.py` can hold lifecycle enums and transition rules rather than ORM models.

`app/models/paper.py`

```
from enum import Enum

class PaperStatus(str, Enum):
    uploaded = "uploaded"
    parsed = "parsed"
    indexed = "indexed"
    extracted = "extracted"

VALID_STATUS_TRANSITIONS = {
    PaperStatus.uploaded.value: [PaperStatus.parsed.value],
    PaperStatus.parsed.value: [PaperStatus.indexed.value],
    PaperStatus.indexed.value: [PaperStatus.extracted.value],
    PaperStatus.extracted.value: []
}
```

This lifecycle mirrors your phased workflow from collection to parsing to machine-readable indexing to structured extraction.^[13]

Schemas

`app/schemas/paper.py`

```
from typing import Optional, List
from pydantic import BaseModel

class PaperStatusUpdate(BaseModel):
    status: str

class PaperCreateResponse(BaseModel):
    message: str
    paper_id: str
    status: str
    filename: str

class PaperResponse(BaseModel):
    _id: str
    original_filename: str
    stored_filename: str
    file_path: str
    content_type: Optional[str] = None
```

```

file_size: Optional[int] = None

title: Optional[str] = None
authors: Optional[List[str]] = []
publication_year: Optional[int] = None
abstract: Optional[str] = None
keywords: Optional[List[str]] = []
references: Optional[List[str]] = []

status: str
error_message: Optional[str] = None
created_at: str
updated_at: str

```

These fields align with the synopsis requirement that metadata like title, authors, publication year, abstract, keywords, and references become part of a structured corpus. ^[13]

Local file saving utility

app/utils/file_storage.py

```

import os
import uuid
from fastapi import UploadFile
from app.core.config import settings

def ensure_storage_dir():
    os.makedirs(settings.STORAGE_DIR, exist_ok=True)

def save_pdf_locally(file: UploadFile):
    ensure_storage_dir()

    ext = os.path.splitext(file.filename)[1].lower()
    if ext != ".pdf":
        ext = ".pdf"

    unique_name = f"{uuid.uuid4()}{ext}"
    file_path = os.path.join(settings.STORAGE_DIR, unique_name)

    content = file.file.read()
    with open(file_path, "wb") as f:
        f.write(content)

    return unique_name, file_path, len(content)

```

This completes the “save PDFs locally” part of your target for today. ^[13]

Service layer

app/services/paper_service.py

```
from datetime import datetime
from bson import ObjectId
from fastapi import HTTPException, UploadFile

from app.db.session import papers_collection
from app.models.paper import PaperStatus, VALID_STATUS_TRANSITIONS
from app.utils.file_storage import save_pdf_locally


def serialize_paper(doc):
    doc["_id"] = str(doc["_id"])
    return doc


def create_paper_record(file: UploadFile):
    if not file.filename.lower().endswith(".pdf"):
        raise HTTPException(status_code=400, detail="Only PDF files are allowed")

    stored_filename, file_path, file_size = save_pdf_locally(file)
    now = datetime.utcnow().isoformat()

    paper_doc = {
        "original_filename": file.filename,
        "stored_filename": stored_filename,
        "file_path": file_path,
        "content_type": file.content_type,
        "file_size": file_size,

        "title": None,
        "authors": [],
        "publication_year": None,
        "abstract": None,
        "keywords": [],
        "references": [],

        "status": PaperStatus.uploaded.value,
        "error_message": None,
        "created_at": now,
        "updated_at": now
    }

    result = papers_collection.insert_one(paper_doc)

    return {
        "message": "Paper uploaded successfully",
        "paper_id": str(result.inserted_id),
        "status": PaperStatus.uploaded.value,
        "filename": file.filename
    }
```

```

def get_all_papers():
    papers = papers_collection.find().sort("created_at", -1)
    return [serialize_paper(p) for p in papers]

def get_paper_by_id(paper_id: str):
    if not ObjectId.is_valid(paper_id):
        raise HTTPException(status_code=400, detail="Invalid paper ID")

    paper = papers_collection.find_one({"_id": ObjectId(paper_id)})
    if not paper:
        raise HTTPException(status_code=404, detail="Paper not found")

    return serialize_paper(paper)

def update_paper_status(paper_id: str, new_status: str):
    if not ObjectId.is_valid(paper_id):
        raise HTTPException(status_code=400, detail="Invalid paper ID")

    paper = papers_collection.find_one({"_id": ObjectId(paper_id)})
    if not paper:
        raise HTTPException(status_code=404, detail="Paper not found")

    current_status = paper["status"]

    if new_status not in [status.value for status in PaperStatus]:
        raise HTTPException(status_code=400, detail="Invalid status")

    allowed_next = VALID_STATUS_TRANSITIONS.get(current_status, [])
    if new_status not in allowed_next:
        raise HTTPException(
            status_code=400,
            detail=f"Invalid transition from '{current_status}' to '{new_status}'"
        )

    papers_collection.update_one(
        {"_id": ObjectId(paper_id)},
        {
            "$set": {
                "status": new_status,
                "updated_at": datetime.utcnow().isoformat()
            }
        }
    )

    updated_paper = papers_collection.find_one({"_id": ObjectId(paper_id)})
    return serialize_paper(updated_paper)

```

This service gives you:

- upload record creation,
- paper listing,
- paper detail fetch,

- strict status pipeline enforcement.

That is enough to establish a valid paper-processing backend foundation. ^[13]

API routes

app/api/routes/papers.py

```
from fastapi import APIRouter, File, UploadFile
from app.schemas.paper import PaperStatusUpdate
from app.services.paper_service import (
    create_paper_record,
    get_all_papers,
    get_paper_by_id,
    update_paper_status,
)

router = APIRouter(prefix="/papers", tags=["papers"])

@router.post("/upload")
def upload_paper(file: UploadFile = File(...)):
    return create_paper_record(file)

@router.get("/")
def list_papers():
    return get_all_papers()

@router.get("/{paper_id}")
def get_paper(paper_id: str):
    return get_paper_by_id(paper_id)

@router.patch("/{paper_id}/status")
def change_paper_status(paper_id: str, payload: PaperStatusUpdate):
    return update_paper_status(paper_id, payload.status)
```

Main endpoint

app/main.py

```
from fastapi import FastAPI
from app.api.routes.papers import router as papers_router

app = FastAPI(title="ResearchOS Backend")

app.include_router(papers_router)

@app.get("/")
```

```
def root():
    return {
        "message": "ResearchOS backend is running",
        "status_pipeline": ["uploaded", "parsed", "indexed", "extracted"]
    }
```

How to run

From inside backend/:

```
uvicorn app.main:app --reload
```

Open Swagger:

<http://127.0.0.1:8000/docs>

How to test

Step 1: Upload PDF

Use POST `/papers/upload` and upload any PDF. This matches the methodology step where the system begins from user-uploaded research papers. ^[13]

Expected response:

```
{
  "message": "Paper uploaded successfully",
  "paper_id": "688b80e9277c41e28c5b1011",
  "status": "uploaded",
  "filename": "sample.pdf"
}
```

Step 2: Verify local save

Check `storage/papers/`. The file should be there with a UUID filename. This gives you persistent local storage for the research corpus foundation. ^[13]

Step 3: List papers

Call GET `/papers/`.

Step 4: Fetch one paper

Call GET `/papers/{paper_id}`.

Step 5: Move status

Call PATCH `/papers/{paper_id}/status` with:

```
{
  "status": "parsed"
}
```

Then:

```
{
  "status": "indexed"
}
```

Then:

```
{
  "status": "extracted"
}
```

Because transitions are validated, you cannot jump directly from uploaded to indexed, which makes your pipeline closer to the phased methodology in the synopsis. [\[13\]](#)

Example document in MongoDB

Your papers collection document will look like this:

```
{
  "_id": "688b80e9277c41e28c5b1011",
  "original_filename": "sample.pdf",
  "stored_filename": "7fc4c8fe-7f16-49e0-82ec-11d4e4f3543d.pdf",
  "file_path": "storage/papers/7fc4c8fe-7f16-49e0-82ec-11d4e4f3543d.pdf",
  "content_type": "application/pdf",
  "file_size": 228493,
  "title": null,
  "authors": [],
  "publication_year": null,
  "abstract": null,
  "keywords": [],
  "references": [],
  "status": "uploaded",
  "error_message": null,
  "created_at": "2026-07-31T23:16:00.000000",
  "updated_at": "2026-07-31T23:16:00.000000"
}
```

This is already a structured starting corpus, which is exactly what Phase 1 in your synopsis aims to create before later parsing and semantic processing begin. [\[13\]](#)

What to say in your presentation

You can present this as:

“We have implemented the paper ingestion layer of ResearchOS, including PDF upload, persistent file storage, a structured MongoDB corpus, and a controlled lifecycle pipeline that prepares papers for parsing, indexing, and knowledge extraction.” ^[13]

What to do next

After this works, tomorrow’s backend step should be:

- extract text from PDF,
- fill title/abstract/authors,
- update uploaded -> parsed automatically,
- then move on to chunking and embeddings. ^[13]

The best next improvement is to add a `POST /papers/{paper_id}/parse` endpoint rather than manually patching status, because that will make the pipeline feel real instead of mocked. ^[13]

Would you like me to give you the **next exact code for `parse_paper()` with PyMuPDF + MongoDB**, fitting into this same folder structure?

✱

i got this output

```
{
  "message": "ResearchOS backend is running",
  "status_pipeline": [
    "uploaded",
    "parsed",
    "indexed",
    "extracted"
  ]
}
```

is this enough or is there more ?

That output only proves your FastAPI app is **running**; it is not enough by itself to claim the paper-ingestion phase is implemented. For your project, Phase 1 requires collecting uploaded PDFs and storing them as a structured research corpus, and Phase 2 then expects parsed text to be prepared for later semantic retrieval. So you still need to verify that the upload route works, files are saved locally, MongoDB records are created, and the status pipeline can actually be updated per paper. ^[14]

What “enough” means

For today’s milestone, “enough” means all four of these are working:

- POST /papers/upload accepts a PDF.
- The PDF gets stored in storage/papers/.
- A paper document gets inserted into MongoDB.
- The paper status can move through uploaded -> parsed -> indexed -> extracted.^[14]

If only / works, then your server is up, but the actual paper collection layer described in your methodology is not yet demonstrated.^[14]

What to test now

Go to Swagger at <http://127.0.0.1:8000/docs> and test these endpoints:

Endpoint	What should happen
POST /papers/upload	Upload PDF and create paper record
GET /papers/	Show all uploaded papers
GET /papers/{paper_id}	Show one paper document
PATCH /papers/{paper_id}/status	Change lifecycle stage

If these all work, then you can say you have implemented the first backend slice for paper acquisition and processing-state management.^[14]

Minimum success checklist

You are done for today only if this checklist passes:

- App starts successfully.
- MongoDB connects successfully.
- Uploading a PDF returns a paper_id.
- File appears inside storage/papers/.
- GET /papers/ returns the uploaded paper.
- GET /papers/{paper_id} returns the correct document.
- PATCH /papers/{paper_id}/status updates status correctly.
- Invalid status transitions are blocked, if you added validation.^[14]

That matches the structured corpus foundation your synopsis expects before later parsing, chunking, embeddings, and retrieval are added.^[14]

Expected next output

After upload, you should see something like:

```
{
  "message": "Paper uploaded successfully",
  "paper_id": "688b80e9277c41e28c5b1011",
  "status": "uploaded",
  "filename": "sample.pdf"
}
```

Then GET `/papers/{paper_id}` should show the saved metadata fields and current status. That is much more important than the root route response, because it proves your system is actually collecting research papers into a structured corpus. ^[14]

Best next step

So the answer is: **not yet enough**. Your root endpoint is correct, but now you must test the actual paper workflow endpoints because your methodology depends on uploaded PDFs being stored and tracked before any parsing or retrieval can happen. ^[14]

Do this next, in order:

1. Test POST `/papers/upload`.
2. Check local file save.
3. Check MongoDB document.
4. Test GET `/papers/`.
5. Test PATCH `/papers/{paper_id}/status`. ^[14]

Paste me the response from POST `/papers/upload`, and I'll tell you immediately whether your backend slice is fully correct or what to fix.

✱✱

i dont have storage/papers

should i create it?

Yes — either **create it manually** or let your code create it automatically before saving files. Since Phase 1 of your project begins with collecting user-uploaded PDF documents into a structured research corpus, you do need a real storage location for those PDFs. ^[15]

Best option

The better approach is: **don't rely on manual creation every time**. Your backend should auto-create `storage/papers/` when the first upload happens, because uploaded PDFs are the foundation for all later parsing and retrieval stages in your workflow. ^[15]

If you already added this in `file_storage.py`:

```
os.makedirs(settings.STORAGE_DIR, exist_ok=True)
```

then the folder will be created automatically on upload. If not, add it.

What to do now

From your backend root, you can manually create it once:

```
mkdir -p storage/papers
```

So your structure becomes:

```
backend/  
├─ app/  
├─ storage/  
│   └─ papers/  
├─ requirements.txt  
└─ .env
```

Make sure your code handles it

Your `app/utils/file_storage.py` should contain this:

```
import os  
import uuid  
from fastapi import UploadFile  
from app.core.config import settings  
  
def ensure_storage_dir():  
    os.makedirs(settings.STORAGE_DIR, exist_ok=True)  
  
def save_pdf_locally(file: UploadFile):  
    ensure_storage_dir()  
  
    ext = os.path.splitext(file.filename)[1].lower()  
    if ext != ".pdf":  
        ext = ".pdf"  
  
    unique_name = f"{uuid.uuid4()}{ext}"  
    file_path = os.path.join(settings.STORAGE_DIR, unique_name)  
  
    content = file.file.read()  
    with open(file_path, "wb") as f:  
        f.write(content)  
  
    return unique_name, file_path, len(content)
```

This ensures the folder exists before any write happens.

Check your `.env`

Make sure:

```
STORAGE_DIR=storage/papers
```

If that is correct, the files will save under `backend/storage/papers/`.

What this means for your progress

Once that folder exists and upload works, you can legitimately say the backend supports **paper ingestion and local PDF persistence**, which is part of the first methodology phase before parsing and embeddings are added. ^[15]

Next step

So yes:

- create `storage/papers/` now,
- keep `os.makedirs(..., exist_ok=True)` in code,
- then test `POST /papers/upload`. ^[15]

After you upload one PDF successfully, paste the response here.

✱✱

it did not create that storage/papers folder

and these problems are shown

The main issue is **your Python environment is not set up correctly**, not the folder logic. The errors `Import "fastapi" could not be resolved`, `pydantic could not be resolved`, and `bson could not be resolved` mean VS Code/Pylance cannot find the installed packages, so your app is likely not running the upload code at all yet. Because of that, `storage/papers` was never created. ^[16]

Why the folder was not created

`storage/papers/` is created only when your `save_pdf_locally()` code actually runs, usually after you hit the upload endpoint. If FastAPI and related packages are not installed in the selected interpreter, the backend code will not execute properly, so the folder creation line never gets triggered. ^[16]

So first fix:

- Python environment

- package installation
- selected VS Code interpreter

Only after that test upload again.

Fix environment

Do this from your backend/ folder.

1. Create virtual environment

```
python -m venv venv
```

If python does not work:

```
python3 -m venv venv
```

2. Activate it

Windows PowerShell

```
venv\Scripts\activate
```

Windows CMD

```
venv\Scripts\activate.bat
```

Mac/Linux

```
source venv/bin/activate
```

3. Install packages

```
pip install -r requirements.txt
```

Or directly:

```
pip install fastapi uvicorn pymongo pydantic pydantic-settings python-multipart dnspython
```

These imports are required for the backend slice you are implementing around uploaded PDF collection and structured storage. ^[16]

Fix VS Code interpreter

After installing packages:

1. Press Ctrl+Shift+P
2. Search: Python: Select Interpreter
3. Select the interpreter from your project venv

It will look something like:

- `./backend/venv/Scripts/python.exe` on Windows
- `./backend/venv/bin/python` on Linux/Mac

Once VS Code points to the correct interpreter, the `could not be resolved import` warnings should disappear.

Then manually create folder once

Even though code should auto-create it, for now just create it once manually so you remove one variable while debugging.

From `backend/`:

```
mkdir storage
mkdir storage\papers
```

If PowerShell/Linux:

```
mkdir -p storage/papers
```

Then verify your helper code

Your `app/utils/file_storage.py` should contain:

```
import os
import uuid
from fastapi import UploadFile
from app.core.config import settings

def ensure_storage_dir():
    os.makedirs(settings.STORAGE_DIR, exist_ok=True)

def save_pdf_locally(file: UploadFile):
    ensure_storage_dir()

    ext = os.path.splitext(file.filename)[1].lower()
    if ext != ".pdf":
        ext = ".pdf"
```

```
unique_name = f"{uuid.uuid4()}{ext}"
file_path = os.path.join(settings.STORAGE_DIR, unique_name)

content = file.file.read()
with open(file_path, "wb") as f:
    f.write(content)

return unique_name, file_path, len(content)
```

This is what supports user-uploaded PDF persistence, which your first project phase depends on. ^[16]

Run the backend

After interpreter + install are fixed:

```
uvicorn app.main:app --reload
```

If that works, open:

<http://127.0.0.1:8000/docs>

What to test next

Test in this order:

1. GET / — confirms app is running.
2. POST /papers/upload — upload one PDF.
3. Check storage/papers/ for saved file.
4. Check MongoDB for inserted paper document.
5. GET /papers/ — confirms paper corpus listing.

That is the real proof that your Phase 1 paper acquisition layer is working, not just the root endpoint. ^[16]

Likely cause summary

From your screenshot, the most likely root causes are:

- virtual environment not created,
- packages not installed,
- wrong Python interpreter selected in VS Code.

The folder problem is secondary. The package resolution errors must be fixed first.

Quick checklist

Do these now:

- Create venv
- Activate venv
- `pip install -r requirements.txt`
- Select interpreter in VS Code
- Create storage/papers
- Run `uvicorn app.main:app --reload`
- Test upload endpoint

One extra note

If bson is unresolved, do **not** install a separate bson package first. pymongo already provides bson, so installing pymongo properly usually solves that import issue.

Paste your:

1. requirements.txt
2. app/utils/file_storage.py
3. terminal error from running `uvicorn app.main:app --reload`

and I'll pinpoint the exact fix.

✱✱

image.jpg

This means your FastAPI app is **starting**, but it is crashing when FastAPI tries to generate the OpenAPI schema for Swagger. The /docs page loads, but /openapi.json is throwing an internal server error, which usually happens because one of your route definitions, request models, or response models is invalid. ^[18]

Most likely cause

From what you shared earlier, the most likely problem is your PaperResponse schema. In Pydantic/FastAPI, fields starting with `_id` often cause OpenAPI/schema issues unless handled carefully, and Swagger generation can fail because of that. This is a backend schema issue, not a storage-folder issue. ^[18]

Fix this first

Change your schema so it uses `id` instead of `_id` in the API response. Keep MongoDB `_id` internally, but convert it before returning.

Replace `app/schemas/paper.py`

```
from typing import Optional, List
from pydantic import BaseModel

class PaperStatusUpdate(BaseModel):
    status: str

class PaperCreateResponse(BaseModel):
    message: str
    paper_id: str
    status: str
    filename: str

class PaperResponse(BaseModel):
    id: str
    original_filename: str
    stored_filename: str
    file_path: str
    content_type: Optional[str] = None
    file_size: Optional[int] = None

    title: Optional[str] = None
    authors: Optional[List[str]] = None
    publication_year: Optional[int] = None
    abstract: Optional[str] = None
    keywords: Optional[List[str]] = None
    references: Optional[List[str]] = None

    status: str
    error_message: Optional[str] = None
    created_at: str
    updated_at: str
```

Fix serializer

Update `app/services/paper_service.py`

Replace:

```
def serialize_paper(doc):
    doc["_id"] = str(doc["_id"])
    return doc
```

with:

```
def serialize_paper(doc):
    return {
        "id": str(doc["_id"]),
        "original_filename": doc.get("original_filename"),
        "stored_filename": doc.get("stored_filename"),
        "file_path": doc.get("file_path"),
        "content_type": doc.get("content_type"),
        "file_size": doc.get("file_size"),
        "title": doc.get("title"),
        "authors": doc.get("authors", []),
        "publication_year": doc.get("publication_year"),
        "abstract": doc.get("abstract"),
        "keywords": doc.get("keywords", []),
        "references": doc.get("references", []),
        "status": doc.get("status"),
        "error_message": doc.get("error_message"),
        "created_at": doc.get("created_at"),
        "updated_at": doc.get("updated_at"),
    }
```

Add response models explicitly

In `app/api/routes/papers.py`, make the route contracts explicit.

Replace with this

```
from typing import List
from fastapi import APIRouter, File, UploadFile
from app.schemas.paper import PaperStatusUpdate, PaperCreateResponse, PaperResponse
from app.services.paper_service import (
    create_paper_record,
    get_all_papers,
    get_paper_by_id,
    update_paper_status,
)

router = APIRouter(prefix="/papers", tags=["papers"])

@router.post("/upload", response_model=PaperCreateResponse)
def upload_paper(file: UploadFile = File(...)):
    return create_paper_record(file)

@router.get("/", response_model=List[PaperResponse])
def list_papers():
    return get_all_papers()

@router.get("/{paper_id}", response_model=PaperResponse)
def get_paper(paper_id: str):
```

```
        return get_paper_by_id(paper_id)

    @router.patch("/{paper_id}/status", response_model=PaperResponse)
    def change_paper_status(paper_id: str, payload: PaperStatusUpdate):
        return update_paper_status(paper_id, payload.status)
```

Also check for this common issue

If your `main.py` includes only the root route and router include, it should be fine. Keep it simple:

`app/main.py`

```
from fastapi import FastAPI
from app.api.routes.papers import router as papers_router

app = FastAPI(title="ResearchOS Backend")

app.include_router(papers_router)

@app.get("/")
def root():
    return {
        "message": "ResearchOS backend is running",
        "status_pipeline": ["uploaded", "parsed", "indexed", "extracted"]
    }
```

Then restart server

Stop server and run again:

```
uvicorn app.main:app --reload
```

Then open:

- <http://127.0.0.1:8000/openapi.json>
- <http://127.0.0.1:8000/docs>

If `/openapi.json` opens, Swagger will work.

If it still fails

Then the next thing to inspect is the **terminal traceback**. The exact FastAPI error will be in the terminal where `uvicorn` is running. That traceback matters more than the browser screenshot.

Most likely categories are:

- invalid Pydantic model field,
- bad type annotation,

- missing import,
- route response not serializable. ^[18]

About the storage folder

The storage/papers issue is still secondary. First fix `/openapi.json`. Once Swagger works and you can hit `POST /papers/upload`, the folder can be auto-created by your file-saving utility as part of the uploaded-paper collection flow. ^[18]

Do this now

1. Replace `PaperResponse` with `id`, not `_id`.
2. Update `serialize_paper()`.
3. Add `response_model=` in routes.
4. Restart `uvicorn`.
5. Open `/openapi.json`.

Paste the **terminal traceback** if it still errors, because that will identify the exact line breaking Swagger.

**

Traceback (most recent call last):

```
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-packages\uvicorn\protocols\http\h11_impl.py", line 409, in run_asgi
result = await app( # type: ignore[func-returns-value]
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-packages\uvicorn\middleware\proxy_headers.py", line 60, in call
return await self.app(scope, receive, send)
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-packages\fastapi\applications.py", line 1138, in call
await super().call(scope, receive, send)
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-packages\starlette\applications.py", line 113, in call
await self.middleware_stack(scope, receive, send)
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-packages\starlette\middleware\errors.py", line 186, in call
raise exc
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-packages\starlette\middleware\errors.py", line 164, in call
await self.app(scope, receive, _send)
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-packages\starlette\middleware\exceptions.py", line 63, in call
await wrap_app_handling_exceptions(self.app, conn)(scope, receive, send)
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-
```

packages\starlette_exception_handler.py", line 53, in wrapped_app
raise exc
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-packages\starlette_exception_handler.py", line 42, in wrapped_app
await app(scope, receive, sender)
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-packages\fastapi\middleware\asyncexitstack.py", line 18, in call
await self.app(scope, receive, send)
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-packages\starlette\routing.py", line 716, in call
await self.middleware_stack(scope, receive, send)
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-packages\starlette\routing.py", line 736, in app
await route.handle(scope, receive, send)
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-packages\starlette\routing.py", line 290, in handle
await self.app(scope, receive, send)
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-packages\starlette\routing.py", line 78, in app
await wrap_app_handling_exceptions(app, request)(scope, receive, send)
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-packages\starlette_exception_handler.py", line 53, in wrapped_app
raise exc
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-packages\starlette_exception_handler.py", line 42, in wrapped_app
await app(scope, receive, sender)
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-packages\starlette\routing.py", line 75, in app
response = await f(request)
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-packages\fastapi\applications.py", line 1093, in openapi
return JSONResponse(self.openapi())
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-packages\fastapi\applications.py", line 1064, in openapi
self.openapi_schema = get_openapi(
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-packages\fastapi\openapi\utils.py", line 518, in get_openapi
field_mapping, definitions = get_definitions(
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-packages\fastapi_compat\v2.py", line 236, in get_definitions
schema_generator = GenerateJsonSchema(ref_template=REF_TEMPLATE)
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-packages\pydantic\json_schema.py", line 301, in init
self.schema_type_to_method = self.build_schema_type_to_method()
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-packages\pydantic\json_schema.py", line 340, in build_schema_type_to_method

```
method_name = f'{key.replace("-", "")}_schema'
```

```
File "C:\Users\Prem\AppData\Local\Programs\Python\Python39\lib\typing.py", line 647, in getattr  
return getattr(self.origin, attr)
```

```
AttributeError: '_SpecialForm' object has no attribute 'replace'
```

```
INFO: 127.0.0.1:51351 - "GET /favicon.ico HTTP/1.1" 404 Not Found
```

```
INFO: 127.0.0.1:52009 - "GET /docs HTTP/1.1" 200 OK
```

```
INFO: 127.0.0.1:52009 - "GET /openapi.json HTTP/1.1" 500 Internal Server Error
```

```
ERROR: Exception in ASGI application
```

```
Traceback (most recent call last):
```

```
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-  
packages\uvicorn\protocols\http\h11_impl.py", line 409, in run_asgi
```

```
result = await app( # type: ignore[func-returns-value]
```

```
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-  
packages\uvicorn\middleware\proxy_headers.py", line 60, in call
```

```
return await self.app(scope, receive, send)
```

```
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-packages\fastapi\  
applications.py", line 1138, in call
```

```
await super().call(scope, receive, send)
```

```
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-packages\starlette\  
applications.py", line 113, in call
```

```
await self.middleware_stack(scope, receive, send)
```

```
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-  
packages\starlette\middleware\errors.py", line 186, in call
```

```
raise exc
```

```
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-  
packages\starlette\middleware\errors.py", line 164, in call
```

```
await self.app(scope, receive, _send)
```

```
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-  
packages\starlette\middleware\exceptions.py", line 63, in call
```

```
await wrap_app_handling_exceptions(self.app, conn)(scope, receive, send)
```

```
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-  
packages\starlette_exception_handler.py", line 53, in wrapped_app
```

```
raise exc
```

```
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-  
packages\starlette_exception_handler.py", line 42, in wrapped_app
```

```
await app(scope, receive, sender)
```

```
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-  
packages\fastapi\middleware\asyncexitstack.py", line 18, in call
```

```
await self.app(scope, receive, send)
```

```
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-packages\starlette\  
routing.py", line 716, in call
```

```
await self.middleware_stack(scope, receive, send)
```

```
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-packages\starlette\  
routing.py", line 736, in app
```

```
await route.handle(scope, receive, send)
```

```
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-packages\starlette\
```

```

routing.py", line 290, in handle
await self.app(scope, receive, send)
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-packages\starlette\routing.py", line 78, in app
await wrap_app_handling_exceptions(app, request)(scope, receive, send)
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-packages\starlette_exception_handler.py", line 53, in wrapped_app
raise exc
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-packages\starlette_exception_handler.py", line 42, in wrapped_app
await app(scope, receive, sender)
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-packages\starlette\routing.py", line 75, in app
response = await f(request)
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-packages\fastapi\applications.py", line 1093, in openapi
return JSONResponse(self.openapi())
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-packages\fastapi\applications.py", line 1064, in openapi
self.openapi_schema = get_openapi(
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-packages\fastapi\openapi\utils.py", line 518, in get_openapi
field_mapping, definitions = get_definitions(
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-packages\fastapi_compat\v2.py", line 236, in get_definitions
schema_generator = GenerateJsonSchema(ref_template=REF_TEMPLATE)
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-packages\pydantic\json_schema.py", line 301, in init
self.schema_type_to_method = self.build_schema_type_to_method()
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-packages\pydantic\json_schema.py", line 340, in build_schema_type_to_method
method_name = f'{key.replace("-", "")}_schema'
File "C:\Users\Prem\AppData\Local\Programs\Python\Python39\lib\typing.py", line 647, in getattr
return getattr(self.origin, attr)
AttributeError: '_SpecialForm' object has no attribute 'replace'

```

This traceback strongly suggests a **version/type-hint compatibility issue**, not your business logic. FastAPI's OpenAPI generation is failing inside Pydantic JSON schema creation, and one common cause is an incompatible FastAPI/Pydantic/Python combination or a problematic type annotation in a response model. Current FastAPI guidance emphasizes that type hints directly drive OpenAPI generation, so a bad or incompatible annotation can break `/openapi.json`.^{[20] [21]}

Most likely fix

You are on Python 3.9, and newer FastAPI guidance notes that recent FastAPI versions have moved toward Python 3.10+ as the standard baseline, with 3.9 support dropped in newer releases. So the safest path is either: ^[22]

- use a **compatible pinned stack** on Python 3.9, or
- move to **Python 3.10+** and reinstall dependencies. ^[22]

For your current setup, do this first:

```
pip uninstall fastapi pydantic starlette -y
pip install "fastapi==0.115.0" "pydantic==2.8.2" "starlette==0.38.6"
```

Then restart:

```
uvicorn app.main:app --reload
```

Also fix route annotations

Your earlier route examples used things like `list[PaperResponse]`, which can be sensitive depending on Python/version/tooling combinations. FastAPI docs support using `response_model=` explicitly, and that is the cleanest way to avoid schema confusion. ^[20]

So in `papers.py`, make sure you use:

```
from typing import List
from fastapi import APIRouter, File, UploadFile
from app.schemas.paper import PaperStatusUpdate, PaperCreateResponse, PaperResponse
from app.services.paper_service import (
    create_paper_record,
    get_all_papers,
    get_paper_by_id,
    update_paper_status,
)

router = APIRouter(prefix="/papers", tags=["papers"])

@router.post("/upload", response_model=PaperCreateResponse)
def upload_paper(file: UploadFile = File(...)):
    return create_paper_record(file)

@router.get("/", response_model=List[PaperResponse])
def list_papers():
    return get_all_papers()

@router.get("/{paper_id}", response_model=PaperResponse)
def get_paper(paper_id: str):
```

```

        return get_paper_by_id(paper_id)

@router.patch("/{paper_id}/status", response_model=PaperResponse)
def change_paper_status(paper_id: str, payload: PaperStatusUpdate):
    return update_paper_status(paper_id, payload.status)

```

That keeps OpenAPI generation predictable. ^[20]

Clean schema version

Also keep `PaperResponse` simple. Since FastAPI and Pydantic build OpenAPI from your model definitions, avoiding tricky aliases and underscored field names reduces schema-generation risk. ^[23] ^[20]

Use this:

`app/schemas/paper.py`

```

from typing import Optional, List
from pydantic import BaseModel

class PaperStatusUpdate(BaseModel):
    status: str

class PaperCreateResponse(BaseModel):
    message: str
    paper_id: str
    status: str
    filename: str

class PaperResponse(BaseModel):
    id: str
    original_filename: str
    stored_filename: str
    file_path: str
    content_type: Optional[str] = None
    file_size: Optional[int] = None
    title: Optional[str] = None
    authors: Optional[List[str]] = None
    publication_year: Optional[int] = None
    abstract: Optional[str] = None
    keywords: Optional[List[str]] = None
    references: Optional[List[str]] = None
    status: str
    error_message: Optional[str] = None
    created_at: str
    updated_at: str

```

Serializer must match schema

app/services/paper_service.py

```
def serialize_paper(doc):
    return {
        "id": str(doc["_id"]),
        "original_filename": doc.get("original_filename"),
        "stored_filename": doc.get("stored_filename"),
        "file_path": doc.get("file_path"),
        "content_type": doc.get("content_type"),
        "file_size": doc.get("file_size"),
        "title": doc.get("title"),
        "authors": doc.get("authors", []),
        "publication_year": doc.get("publication_year"),
        "abstract": doc.get("abstract"),
        "keywords": doc.get("keywords", []),
        "references": doc.get("references", []),
        "status": doc.get("status"),
        "error_message": doc.get("error_message"),
        "created_at": doc.get("created_at"),
        "updated_at": doc.get("updated_at"),
    }
```

Best debugging shortcut

If it still fails after pinning versions, temporarily **remove all response models** from routes and test `/docs` again. FastAPI allows response-model control through decorators, so removing them helps isolate whether the crash is schema-related. [\[20\]](#)

Example temporary test:

```
@router.get("/")
def list_papers():
    return get_all_papers()
```

If `/docs` starts working, the issue is definitely in one of your Pydantic schemas or type hints.

Recommended order now

1. Pin FastAPI/Pydantic/Starlette versions. [\[22\]](#)
2. Use `typing.List`, not fancy/ambiguous annotations, in response models. [\[20\]](#)
3. Keep `PaperResponse` plain with `id`, not `_id`. [\[20\]](#)
4. Restart server.
5. Test `/openapi.json` first.
6. Then test `/papers/upload`.

About your project progress

Once Swagger works and upload starts saving PDF records, you'll actually have the **paper collection foundation** required by Phase 1 of your synopsis: uploaded PDFs stored as a structured corpus that later phases can parse, index, and analyze. ^[24]

Paste these 3 things next if it still breaks:

- `pip freeze`
- `app/schemas/paper.py`
- `app/api/routes/papers.py`

✱

its working now with the correct version install

what does this prove?

Yes — this proves your **FastAPI backend is now correctly configured and generating OpenAPI documentation**. The fact that `/openapi.json` loads and Swagger shows your papers routes means the import/version problem is fixed and your route + schema definitions are valid enough for FastAPI to build the API contract. ^[36]

What this proves

It proves these backend basics are working:

- FastAPI app starts successfully.
- Pydantic schemas are compatible with your installed version.
- Your papers routes are registered correctly.
- Swagger/OpenAPI can introspect your endpoints.
- The upload route exists with the expected request body and response model. ^[36]

That is a meaningful milestone because earlier the app could not even expose API docs due to schema-generation failure. Now the backend is in a usable state for real testing. ^[36]

What it does not prove yet

It does not yet prove that the research pipeline is complete. You still need to verify the actual runtime behavior of the core MVP flow:

- uploading a PDF,
- saving it locally,
- inserting a MongoDB record,
- listing the uploaded papers,
- updating lifecycle status. ^[36]

In your synopsis, Phase 1 is about collecting user-uploaded PDFs and storing metadata in a structured corpus, which is the foundation for later parsing, embeddings, retrieval, and extraction. Swagger working only means the interface exists; the data flow still needs to be tested end-to-end. ^[36]

What you should do next

Run these tests in order:

1. Open POST `/papers/upload`.
2. Upload one PDF.
3. Confirm the file appears in `storage/papers/`.
4. Confirm the document appears in MongoDB.
5. Call GET `/papers/`.
6. Call PATCH `/papers/{paper_id}/status` to move it to `parsed`. ^[36]

If all of those work, then you can confidently say you have completed the **paper ingestion and lifecycle foundation** of ResearchOS, which is exactly the first stage of the project methodology. ^[36]

Good presentation line

You can say:

“The backend API contract is now successfully generated and documented through Swagger, which confirms that the core FastAPI routes and schemas are correctly configured. The next step is validating the paper ingestion pipeline end-to-end through upload, storage, and MongoDB persistence.” ^[36]

Bottom line

So yes, this is a strong sign, but it is still only the **API layer** being ready. The real proof comes when a PDF upload successfully creates a stored file and a MongoDB document with uploaded status. ^[36]

Do you want the exact **upload test procedure** in Swagger next, with what response you should expect at each step?



it got uploaded to storage\papers

but it has this error

Traceback (most recent call last):

```
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-packages\uvicorn\protocols\http\h11_impl.py", line 409, in run_asgi
result = await app( # type: ignore[func-returns-value]
```

File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-packages\uvicorn\middleware\proxy_headers.py", line 60, in call
return await self.app(scope, receive, send)
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-packages\fastapi\applications.py", line 1054, in call
await super().call(scope, receive, send)
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-packages\starlette\applications.py", line 113, in call
await self.middleware_stack(scope, receive, send)
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-packages\starlette\middleware\errors.py", line 187, in call
raise exc
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-packages\starlette\middleware\errors.py", line 165, in call
await self.app(scope, receive, _send)
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-packages\starlette\middleware\exceptions.py", line 62, in call
await wrap_app_handling_exceptions(self.app, conn)(scope, receive, send)
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-packages\starlette_exception_handler.py", line 62, in wrapped_app
raise exc
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-packages\starlette_exception_handler.py", line 51, in wrapped_app
await app(scope, receive, sender)
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-packages\starlette\routing.py", line 715, in call
await self.middleware_stack(scope, receive, send)
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-packages\starlette\routing.py", line 735, in app
await route.handle(scope, receive, send)
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-packages\starlette\routing.py", line 288, in handle
await self.app(scope, receive, send)
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-packages\starlette\routing.py", line 76, in app
await wrap_app_handling_exceptions(app, request)(scope, receive, send)
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-packages\starlette_exception_handler.py", line 62, in wrapped_app
raise exc
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-packages\starlette_exception_handler.py", line 51, in wrapped_app
await app(scope, receive, sender)
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-packages\starlette\routing.py", line 73, in app
response = await f(request)
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-packages\fastapi\

routing.py", line 301, in app
raw_response = await run_endpoint_function(
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-packages\fastapi\
routing.py", line 214, in run_endpoint_function
return await run_in_threadpool(dependant.call, **values)
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-packages\starlette\
concurrency.py", line 39, in run_in_threadpool
return await anyio.to_thread.run_sync(func, *args)
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-
packages\anyio\to_thread.py", line 63, in run_sync
return await get_async_backend().run_sync_in_worker_thread(
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-
packages\anyio_backends_asyncio.py", line 2502, in run_sync_in_worker_thread
return await future
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-
packages\anyio_backends_asyncio.py", line 986, in run
result = context.run(func, *args)
File "C:\Users\Prem\Desktop\projects\researchOS\backend\app\api\routes\papers.py", line 16, in
upload_paper
return create_paper_record(file)
File "C:\Users\Prem\Desktop\projects\researchOS\backend\app\services\paper_service.py", line
56, in create_paper_record
result = papers_collection.insert_one(paper_doc)
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-
packages\pymongo\synchronous\collection.py", line 890, in insert_one
self._insert_one(
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-
packages\pymongo\synchronous\collection.py", line 830, in _insert_one
self._database.client._retryable_write(
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-
packages\pymongo\synchronous\mongo_client.py", line 2113, in _retryable_write
return self._retry_with_session(retryable, func, s, bulk, operation, operation_id)
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-
packages\pymongo\synchronous\mongo_client.py", line 1986, in _retry_with_session
return self._retry_internal(
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-
packages\pymongo_csot.py", line 125, in csot_wrapper
return func(self, *args, **kwargs)
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-
packages\pymongo\synchronous\mongo_client.py", line 2025, in _retry_internal
return _ClientConnectionRetryable(
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-
packages\pymongo\synchronous\mongo_client.py", line 2811, in run
res = self._read() if self._is_read else self._write()
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-
packages\pymongo\synchronous\mongo_client.py", line 2992, in _write

```

self._server = self._get_server()
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-
packages\pymongo\synchronous\mongo_client.py", line 2975, in _get_server
return self._client._select_server(
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-
packages\pymongo\synchronous\mongo_client.py", line 1851, in _select_server
server = topology.select_server(
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-
packages\pymongo\synchronous\topology.py", line 428, in select_server
server = self._select_server(
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-
packages\pymongo\synchronous\topology.py", line 402, in _select_server
servers = self.select_servers(
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-
packages\pymongo\synchronous\topology.py", line 298, in select_servers
server_descriptions = self._select_servers_loop(
File "c:\users\prem\desktop\projects\researchos\backend\venv\lib\site-
packages\pymongo\synchronous\topology.py", line 359, in _select_servers_loop
raise ServerSelectionTimeoutError(
pymongo.errors.ServerSelectionTimeoutError: localhost:27017: [WinError 10061] No
connection could be made because the target machine actively refused it (configured timeouts:
socketTimeoutMS: 20000.0ms, connectTimeoutMS: 20000.0ms), Timeout: 30s, Topology
Description: <TopologyDescription id: 6a6cf089f6496e30775568a9, topology_type: Unknown,
servers: [<ServerDescription ('localhost', 27017) server_type: Unknown, rtt: None,
error=AutoReconnect('localhost:27017: [WinError 10061] No connection could be made because
the target machine actively refused it (configured timeouts: socketTimeoutMS: 20000.0ms,
connectTimeoutMS: 20000.0ms)')>]>

```

The upload itself is working; the failure happens **after saving the file**, when your code tries to insert the paper record into MongoDB and cannot connect to localhost:27017. The traceback shows `papers_collection.insert_one(paper_doc)` failing with `pymongo.errors.ServerSelectionTimeoutError` and `WinError 10061`, which means no MongoDB server is accepting connections on that host/port. ^[39]

Root cause

Your flow is currently:

1. file uploaded to storage/papers
2. `create_paper_record(file)` runs
3. `papers_collection.insert_one(...)` is called
4. Mongo connection fails ^[39]

So the bug is **not** in FastAPI upload handling and not in file saving. It is a database connectivity issue: either MongoDB is not running, it is not running on port 27017, or your app is pointing to the wrong URI. ^[39]

What to check first

Run these checks on your machine:

- Check whether MongoDB is running.
- Check whether your app config uses `mongodb://localhost:27017/...`
- Check whether you are using MongoDB Compass/Atlas instead of a local Mongo service.
- Check whether the database service name exists in Windows Services. ^[39]

On Windows, the most common reason for `WinError 10061` is simply that the MongoDB service is stopped, so the connection is refused. ^[39]

Fast fix path

Try these in order:

1. Start MongoDB locally.
 - If installed as a service: start the MongoDB service from Services or terminal.
 - If not a service: run `mongod --dbpath <your-db-folder>`.
2. Verify your connection string.
 - In `.env`, it should be something like:
`MONGODB_URI=mongodb://localhost:27017/researchos`
3. Verify your DB bootstrap code.
 - Make sure you are doing:

```
from pymongo import MongoClient
client = MongoClient(MONGODB_URI)
db = client["researchos"]
papers_collection = db["papers"]
```
4. Add a startup ping.
 - This catches the DB issue before the first upload request fails. ^[39]

Example:

```
from pymongo import MongoClient
from pymongo.errors import PyMongoError

client = MongoClient(MONGODB_URI, serverSelectionTimeoutMS=5000)

try:
    client.admin.command("ping")
    print("MongoDB connected")
except PyMongoError as e:
    print("MongoDB connection failed:", e)
```

If MongoDB is not installed/running

For your 3–4 day presentation timeline, the quickest options are:

- **Option A:** Use local MongoDB if already installed, just start the service.
- **Option B:** Use MongoDB Atlas and replace `localhost:27017` with the Atlas URI.
- **Option C:** Temporary fallback to JSON/SQLite for demo purposes if DB setup is blocking progress.^[39]

For a first-phase demo, persistence matters less than proving ingestion and retrieval pipeline progress, so even a fallback storage layer is acceptable if clearly explained as a temporary implementation.^[39]

Better error handling

Right now your API crashes with a full stack trace. For presentation, you should return a clean API error instead.

Example:

```
from fastapi import HTTPException
from pymongo.errors import PyMongoError

def create_paper_record(file):
    try:
        paper_doc = {
            "filename": file.filename,
            "status": "uploaded"
        }
        result = papers_collection.insert_one(paper_doc)
        return {"id": str(result.inserted_id), "message": "Paper uploaded"}
    except PyMongoError as e:
        raise HTTPException(status_code=500, detail=f"Database connection failed: {s
```

That way the frontend can show:

- “File uploaded, but metadata could not be saved because database is unavailable.”^[39]

Best architecture fix

Since ResearchOS is supposed to build a structured corpus from uploaded papers, database reliability is part of the Phase 1 foundation in your methodology. So the right short-term structure is:^[39]

- save file to storage,
- save metadata to DB,
- mark status as uploaded,
- later run parsing/indexing asynchronously.^[39]

Also add statuses like:

- `uploaded`
- `db_failed`
- `parsed`
- `indexed`^[39]

That makes the system more demo-friendly and easier to debug.

Most likely resolution

Based on this traceback, the **most likely** fix is:

- start MongoDB service,
- confirm port 27017 is active,
- restart FastAPI,
- try upload again.^[39]

If you want, paste these files and I'll pinpoint the exact fix in your code:

- `app/services/paper_service.py`
- DB connection file
- `.env` Mongo URI
- `app/api/routes/papers.py`

✱✱

lets complete the remaining part for phase 1 (40-50%)

Yes — now the right move is to freeze phase 1 scope and finish a clean 40–50% build, not keep expanding features. For ResearchOS, phase 1 should cover the foundation your synopsis depends on: paper collection, parsing/preprocessing, semantic retrieval, structured extraction, and a basic comparison workflow.^[40] ^[41]

Scope to finish

Your synopsis already defines the early pipeline clearly: collect papers and metadata, parse PDFs into chunks, store machine-readable representations for semantic retrieval, then extract structured information for cross-paper comparison. So the remaining phase-1 target should be this exact bundle: ^[41]

Module	Status target
PDF upload + storage	Finish fully
Paper metadata record in DB	Finish fully
PDF text parsing	Finish fully
Chunking + chunk storage	Finish fully

Module	Status target
Embedding + vector indexing	Finish basic working version
Query/search over papers	Finish fully
Structured extraction	Finish partial but demoable
Cross-paper comparison table	Finish basic version
Gap detection / drafting / reviewer simulation	Defer to phase 2 ^[41]

That is enough to honestly present that the “research intelligence core” is implemented, while advanced generation and validation modules are upcoming. ^[41]

What to build now

The fastest way to close phase 1 is to organize work into one linear backend pipeline plus two frontend views. Your platform is meant to transform papers into a structured, searchable knowledge base, so the backend should move a paper through statuses like uploaded, parsed, indexed, extracted. ^[41]

Backend pipeline

Build this exact request flow:

1. Upload PDF.
2. Save file in storage/papers.
3. Save metadata record in MongoDB.
4. Parse PDF text.
5. Split into chunks.
6. Generate embeddings.
7. Store chunks and vectors.
8. Run extraction on core fields.
9. Return paper status + extracted summary. ^[41]

If Mongo is slowing you down, do not over-engineer. Make the first goal: one uploaded paper successfully becomes searchable and explorable. ^[40]

Frontend views

For phase 1, you only need:

- **Paper Library page:** uploaded papers, status, metadata, “parse/index” button.
- **Paper Analysis page:** extracted fields, retrieved chunks, compare selected papers. ^[41]

That is enough for a strong live demo.

Best team split

Since you likely have 3 people, divide like this:

Person	Responsibility
Person 1	Upload API, DB record, parser, storage pipeline
Person 2	Chunking, embeddings, vector search, retrieval API
Person 3	Frontend pages, extracted info cards, comparison UI

Then one person can also help wire extraction prompts and clean response formatting once the basics run. This suits your preference for modular FastAPI router-based systems and keeps everyone working in parallel. [\[42\]](#) [\[43\]](#)

Exact deliverables

By the end of phase 1, aim to show these **visible outcomes**:

1. Paper upload works

A paper appears in the library with:

- filename
- upload date
- status
- optional title/authors/year if extracted [\[41\]](#)

2. Parsed text exists

Each paper can show:

- abstract
- introduction
- methodology or section snippets
- parsed page/chunk count [\[41\]](#)

3. Search works

User types:

- “What dataset is used?”
- “What baselines are compared?”
- “What limitations are mentioned?”

System returns:

- top chunks

- source paper
- page or section reference ^[41]

4. Structured extraction works

For each paper, show cards like:

- Objective
- Methodology
- Dataset
- Metrics
- Baselines
- Limitations
- Future work ^[41]

5. Comparison works

Pick 2–3 papers and show a table:

Field	Paper 1	Paper 2	Paper 3
Method	...	...	...
Dataset	...	...	...
Metric	...	...	...
Baseline	...	...	...
Limitation	...	...	...

This directly reflects your project objective of structured cross-paper comparison. ^[41]

What not to do now

Do not spend phase-1 time on:

- full manuscript generation,
- LaTeX export,
- novelty scoring,
- conference/journal recommendation,
- originality checker,
- peer-review simulation. ^[41]

These are valuable, but your methodology places them after retrieval and structured analysis, so cutting them for now is completely defensible. ^[41]

Minimum backend structure

Keep the backend simple and modular. Suggested FastAPI modules:

- `routes/papers.py` — upload, list, get paper
- `routes/analysis.py` — parse, extract, compare
- `routes/search.py` — semantic query/search
- `services/paper_service.py` — storage + DB record
- `services/parser_service.py` — PDF text extraction
- `services/chunk_service.py` — split into chunks
- `services/embed_service.py` — embeddings + vector DB
- `services/extract_service.py` — structured field extraction ^[43] ^[42]

Suggested Mongo collections:

- `papers`
- `paper_chunks`
- `paper_extractions`
- optionally `search_logs`

This is enough for phase 1.

Execution order

Do the remaining work in this order:

Step 1: Stabilize upload + DB

First make sure upload does not fail after saving the PDF. Since you already hit a MongoDB connection issue, get this fully stable before anything else. A paper record must be created successfully after upload. ^[40]

Step 2: Add parser

Once DB works:

- read PDF text,
- store raw extracted text or page blocks,
- update status to parsed. ^[41]

Step 3: Add chunking

Create section-aware chunks with metadata:

- `paper_id`
- `chunk_id`

- `section`
- `page`
- `text` ^[41]

Step 4: Add embeddings

Use one simple embedding model and store vectors in Chroma/FAISS/Qdrant. The synopsis requires embeddings and semantic retrieval, but for phase 1 only a basic working version is enough. ^[41]

Step 5: Add search endpoint

Implement:

- `POST /search/query`
- input: question + optional selected paper ids
- output: top-k chunks + simple answer + citations/section references ^[41]

Step 6: Add extraction

Start with only 5 fields:

- `methodology`
- `dataset`
- `evaluation metric`
- `limitations`
- `future work` ^[41]

This is enough to look substantial.

Step 7: Add comparison endpoint

Implement:

- `POST /analysis/compare`
- input: array of paper ids
- output: merged structured comparison object ^[41]

Step 8: Polish UI

Show:

- paper cards,
- extraction tabs,
- compare table,

- search results panel.

That is your demo-ready phase 1.

2-day sprint plan

If you are very short on time, compress it like this:

Day 1

- Fix Mongo connectivity and paper record insertion.
- Complete upload flow.
- Add parser and chunking.
- Save parsed text/chunks in DB. ^[41]

Day 2

- Add embeddings and semantic search.
- Add basic field extraction.
- Add one comparison table UI.
- Prepare demo script/screenshots. ^[40] ^[41]

If you get a third day:

- add loading states,
- better error handling,
- citation/source panel,
- one “coming in phase 2” screen.

Demo script

For your presentation, show this exact flow:

1. Open Paper Library.
2. Upload 2–3 PDFs.
3. Show status changing from uploaded -> parsed -> indexed -> extracted.
4. Open one paper and show extracted methodology/dataset/metric fields.
5. Ask a semantic question and show top evidence chunks.
6. Compare 2–3 papers in a table.
7. End with “phase 2 will add gap detection, grounded drafting, and reviewer simulation.” ^[41]

That demo is tightly aligned with the objectives in your synopsis and makes the project feel real rather than conceptual. ^[41]

Success criteria

You can call phase 1 complete when all of these are true:

- A paper upload creates both a saved file and DB record.
- At least one paper is parsed into chunks.
- A search query returns relevant chunks.
- At least 5 structured fields are extracted per paper.
- Two papers can be compared in one table.
- The UI clearly shows source-backed outputs. ^[41]

That is a credible 40–50% implementation because it covers the core research corpus, retrieval, knowledge extraction, and comparison foundation the rest of ResearchOS depends on. ^[40] ^[41]

Recommendation

Your next move should be to treat phase 1 as “**ingest -> parse -> retrieve -> extract -> compare**” and lock everything else. That matches both your synopsis and your preference for building systems with real workflow value instead of shallow demos. ^[44] ^[41]

Would you like me to turn this into a **very concrete task checklist with API endpoints, Mongo schemas, and frontend components** so your team can start coding immediately?

✱

okay give me the steps

Yes — here are the **exact steps** to finish phase 1 at a strong 40–50% level. These steps follow your synopsis flow: collect papers, parse them, store searchable representations, extract structured knowledge, and support comparison. ^[45]

Build order

Do the work in this sequence so nothing blocks later modules:

1. Fix upload + DB save.
2. Parse PDF text.
3. Create chunks.
4. Generate embeddings and index them.
5. Add semantic search.
6. Add structured extraction.
7. Add comparison view.
8. Polish demo flow. ^[45]

Backend steps

Step 1: Make upload stable

Your first goal is: when a PDF is uploaded, both the file and the paper record must be saved successfully. Phase 1 in your synopsis begins with collecting uploaded papers and storing metadata to create a structured corpus. ^[45]

Do this:

- Save file in `storage/papers/`.
- Insert a Mongo document in `papers`.
- Store:
 - `filename`
 - `filepath`
 - `uploaded_at`
 - `status = "uploaded"`
 - optional `title`, `authors`, `year`, `abstract`
- Return `paper_id`. ^[45]

Step 2: Add paper schema

Create a clean `papers` collection schema so every later step can update the same record. The synopsis explicitly treats metadata as the foundation for later analysis. ^[45]

Suggested fields:

```
{
  "_id": "...",
  "filename": "paper.pdf",
  "filepath": "storage/papers/paper.pdf",
  "title": "",
  "authors": [],
  "year": null,
  "abstract": "",
  "keywords": [],
  "references": [],
  "status": "uploaded",
  "uploaded_at": "...",
  "parsed_at": null,
  "indexed_at": null,
  "extracted_at": null
}
```

Step 3: Build parser service

Phase 2 in the synopsis says papers are parsed to extract textual content from PDFs. So next, create `parser_service.py`.^[45]

Do this:

- Read PDF with PyMuPDF or pdfplumber.
- Extract page-wise text.
- Join pages into full text.
- Try to detect section headings like Abstract, Introduction, Methodology, Conclusion.
- Update paper status to parsed.^[45]

Store parsed output in either:

- `papers.raw_text`, or
- a separate `paper_pages` collection.

Step 4: Chunk the text

The synopsis says extracted text is cleaned, segmented into meaningful chunks, and turned into machine-readable representations. So after parsing, split into chunks.^[45]

Do this:

- chunk by section if possible,
- else chunk by paragraphs or token window,
- keep overlap of about 100–150 words,
- attach metadata:
 - `paper_id`
 - `chunk_index`
 - `section_title`
 - `page_number`
 - `text`^[45]

Create collection:

```
paper_chunks
{
  "_id": "...",
  "paper_id": "...",
  "chunk_index": 0,
  "section_title": "Introduction",
  "page_number": 2,
  "text": "...."
}
```

Step 5: Add embeddings

Your synopsis explicitly includes embeddings stored in a vector database for semantic retrieval. Use one simple embedding model for now. ^[45]

Do this:

- Generate embedding for each chunk.
- Store in Chroma, FAISS, or Qdrant.
- Save metadata with vector:
 - `paper_id`
 - `chunk_id`
 - `section_title`
 - `page_number` ^[45]

Minimal approach:

- embed after parsing,
- mark paper as indexed.

Step 6: Create search endpoint

Phase 3 in the synopsis uses RAG to retrieve relevant sections based on user queries. Build one endpoint first. ^[45]

Example:

POST `/search/query`

Input:

```
{
  "query": "What dataset is used?",
  "paper_ids": ["id1", "id2"]
}
```

Output:

```
{
  "answer": "...",
  "results": [
    {
      "paper_id": "...",
      "section_title": "Experiments",
      "page_number": 5,
      "text": "..."
    }
  ]
}
```

For phase 1, even if you skip final LLM answer generation, returning top chunks is enough to prove semantic retrieval. ^[45]

Step 7: Add extraction service

Your synopsis says the system should extract methodologies, datasets, architectures, baselines, evaluation metrics, limitations, and future directions. For phase 1, do only a subset. ^[45]

Start with:

- objective
- methodology
- dataset
- evaluation metric
- limitations
- future work ^[45]

Create `extract_service.py`:

- feed parsed text or top sections to LLM,
- ask for strict JSON output,
- store result in `paper_extractions`.

Example schema:

```
{
  "_id": "...",
  "paper_id": "...",
  "objective": "...",
  "methodology": "...",
  "dataset": "...",
  "evaluation_metric": "...",
  "limitations": "...",
  "future_work": "...",
  "created_at": "..."
}
```

Step 8: Add compare endpoint

Phase 4 in the synopsis is research analysis and comparison across studies. So build one endpoint that merges extracted fields. ^[45]

Example:

POST `/analysis/compare`

Input:

```
{
  "paper_ids": ["id1", "id2", "id3"]
}
```

Output:

```
{
  "rows": [
    {"field": "Methodology", "values": {...}},
    {"field": "Dataset", "values": {...}},
    {"field": "Metric", "values": {...}},
    {"field": "Limitations", "values": {...}}
  ]
}
```

That is enough for the first demo.

API steps

Keep your FastAPI routes simple:

`papers.py`

- POST `/papers/upload`
- GET `/papers`
- GET `/papers/{paper_id}`

`analysis.py`

- POST `/analysis/parse/{paper_id}`
- POST `/analysis/extract/{paper_id}`
- POST `/analysis/compare`

`search.py`

- POST `/search/query`

This is enough for phase 1 and matches your modular backend preference. [\[46\]](#) [\[47\]](#)

Frontend steps

Step 9: Build Paper Library page

This should show uploaded papers because the synopsis starts with creating a structured research corpus. [\[45\]](#)

Show:

- file name
- title
- upload date
- status
- actions:
 - Parse
 - Extract
 - View
 - Compare

Step 10: Build Paper Detail page

This should show the parsed and extracted knowledge from one paper. The synopsis emphasizes transforming unstructured papers into structured, explainable knowledge. ^[45]

Sections to show:

- metadata
- parsed preview
- extracted fields
- chunk list
- “Ask about this paper” search box

Step 11: Build Search panel

This is your semantic retrieval demo. The synopsis uses semantic retrieval as the bridge between documents and grounded insights. ^[45]

Show:

- input query box
- top matching chunks
- paper title
- section title
- page number
- optional short answer

Step 12: Build Comparison page

This is one of your strongest visible features because your objectives explicitly mention structured cross-paper comparison of methodologies, datasets, and performance metrics. ^[45]

UI flow:

- multi-select papers
- click Compare
- show table:
 - Method
 - Dataset
 - Metric
 - Limitation
 - Future work

Team assignment

Person 1

- Upload route
- Mongo record creation
- Parser service
- paper status updates

Person 2

- Chunking
- Embeddings
- Vector DB
- Search API

Person 3

- Extraction service
- Compare endpoint
- Frontend pages

That division keeps everyone parallel and avoids overlap.

2-day implementation checklist

Day 1

- Fix Mongo connection and insert flow.
- Finish `/papers/upload`.
- Add `GET /papers`.
- Build parser service.
- Save parsed text/chunks.
- Show library page with statuses. ^[45]

Day 2

- Add embeddings and indexing.
- Add `/search/query`.
- Add extraction service.
- Add `/analysis/compare`.
- Build paper detail and compare UI.
- Prepare demo script. ^[45]

What to say each step is achieving

Use these lines in your presentation:

- Upload module creates the structured corpus foundation. ^[45]
- Parsing and chunking convert PDFs into searchable units. ^[45]
- Embeddings and vector indexing enable semantic retrieval. ^[45]
- Structured extraction turns raw paper text into comparable research attributes. ^[45]
- Comparison view supports state-of-the-art analysis across multiple papers. ^[45]

Done criteria

Phase 1 is done when:

- One PDF uploads successfully.
- Mongo stores the paper record.
- Parsed text is generated.
- Chunks are created and stored.
- Search returns relevant chunks.
- Extraction shows 5–6 fields.
- Two papers can be compared in one table. ^[45]

Best immediate next action

Start with this order tonight:

1. Fix Mongo and upload save.
2. Make parser run on one paper.
3. Save chunks.
4. Add one search endpoint.
5. Add one extraction JSON output.
6. Build one compare table. ^[45]

That will give you a very believable and demoable 40–50% phase 1.

Would you like me to give you the exact Mongo schemas + FastAPI endpoint contracts + folder structure next?

✱✱

give me all the relevant code and steps

Yes — below is a complete phase-1 implementation plan with code skeletons for a modular FastAPI backend that matches your preferred structure, plus the exact steps to wire upload, parse, chunk, search, extract, and compare. This scope is aligned with your synopsis phases for paper collection, parsing/preprocessing, retrieval, knowledge extraction, and comparison. ^[48]
^[49]

Scope

For phase 1, build only the core research pipeline: uploaded papers become a structured and searchable corpus, key fields are extracted, and multiple papers can be compared side by side. That directly matches the synopsis objectives around retrieval, structured knowledge extraction, and cross-paper comparison. ^[48]

Use this feature cut:

- PDF upload + paper record
- PDF parsing
- Chunking
- Embeddings + vector index
- Semantic query
- Structured extraction
- Compare 2–3 papers ^[48]

Folder structure

Use this backend layout since you prefer separate routers, services, and models.^[49]

```
backend/
  app/
    main.py
    config.py
    db.py
    models/
      schemas.py
    routers/
      papers.py
      analysis.py
      search.py
    services/
      paper_service.py
      parser_service.py
      chunk_service.py
      embedding_service.py
      vector_service.py
      extraction_service.py
      compare_service.py
  storage/
    papers/
  requirements.txt
  .env
```

Config and DB

app/config.py

```
from pydantic_settings import BaseSettings
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent.parent
STORAGE_DIR = BASE_DIR / "storage" / "papers"
VECTOR_DIR = BASE_DIR / "storage" / "vectors"

class Settings(BaseSettings):
    APP_NAME: str = "ResearchOS Backend"
    APP_ENV: str = "dev"
    FRONTEND_URL: str = "http://localhost:3000"

    MONGODB_URI: str = "mongodb://localhost:27017"
    MONGODB_DB: str = "researchos"

    EMBEDDING_MODEL: str = "all-MiniLM-L6-v2"
    TOP_K: int = 5

    class Config:
        env_file = ".env"
```

```
settings = Settings()
STORAGE_DIR.mkdir(parents=True, exist_ok=True)
VECTOR_DIR.mkdir(parents=True, exist_ok=True)
```

app/db.py

```
from pymongo import MongoClient
from app.config import settings

client = MongoClient(settings.MONGODB_URI, serverSelectionTimeoutMS=5000)
db = client[settings.MONGODB_DB]

papers_collection = db["papers"]
chunks_collection = db["paper_chunks"]
extractions_collection = db["paper_extractions"]
```

Pydantic schemas

app/models/schemas.py

```
from pydantic import BaseModel, Field
from typing import List, Optional, Dict, Any

class PaperResponse(BaseModel):
    id: str
    filename: str
    title: Optional[str] = ""
    status: str

class SearchRequest(BaseModel):
    query: str
    paper_ids: Optional[List[str]] = None
    top_k: Optional[int] = 5

class SearchResultItem(BaseModel):
    paper_id: str
    chunk_id: str
    section_title: Optional[str] = ""
    page_number: Optional[int] = None
    text: str
    score: float

class SearchResponse(BaseModel):
    answer: str
    results: List[SearchResultItem]

class ExtractionResponse(BaseModel):
    paper_id: str
    objective: Optional[str] = ""
    methodology: Optional[str] = ""
    dataset: Optional[str] = ""
    evaluation_metric: Optional[str] = ""
```

```

        limitations: Optional[str] = ""
        future_work: Optional[str] = ""

class CompareRequest(BaseModel):
    paper_ids: List[str]

class CompareResponse(BaseModel):
    rows: List[Dict[str, Any]]

```

Main app

app/main.py

```

from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

from app.config import settings
from app.routers import papers, analysis, search

app = FastAPI(title=settings.APP_NAME)

app.add_middleware(
    CORSMiddleware,
    allow_origins=[settings.FRONTEND_URL],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

app.include_router(papers.router)
app.include_router(analysis.router)
app.include_router(search.router)

@app.get("/")
def root():
    return {"app_name": settings.APP_NAME, "environment": settings.APP_ENV}

@app.get("/health")
def health():
    return {"ok": True}

```

Step 1: Upload papers

The first implemented module should create the structured corpus foundation by storing uploaded papers and metadata. That is the first methodology phase in your synopsis. ^[48]

app/services/paper_service.py

```
from fastapi import UploadFile, HTTPException
from bson import ObjectId
from datetime import datetime
from pathlib import Path
import shutil

from app.config import STORAGE_DIR
from app.db import papers_collection

def create_paper_record(file: UploadFile):
    filename = file.filename
    if not filename.lower().endswith(".pdf"):
        raise HTTPException(status_code=400, detail="Only PDF files are allowed")

    save_path = STORAGE_DIR / filename
    with open(save_path, "wb") as buffer:
        shutil.copyfileobj(file.file, buffer)

    paper_doc = {
        "filename": filename,
        "filepath": str(save_path),
        "title": "",
        "authors": [],
        "year": None,
        "abstract": "",
        "keywords": [],
        "references": [],
        "status": "uploaded",
        "uploaded_at": datetime.utcnow(),
        "parsed_at": None,
        "indexed_at": None,
        "extracted_at": None,
    }

    result = papers_collection.insert_one(paper_doc)
    return {
        "id": str(result.inserted_id),
        "filename": filename,
        "title": "",
        "status": "uploaded",
    }

def list_papers():
    papers = []
    for p in papers_collection.find().sort("uploaded_at", -1):
        papers.append({
            "id": str(p["_id"]),
            "filename": p.get("filename", ""),
            "title": p.get("title", ""),
            "status": p.get("status", "uploaded"),
        })
    return papers

def get_paper_by_id(paper_id: str):
```

```

paper = papers_collection.find_one({"_id": ObjectId(paper_id)})
if not paper:
    raise HTTPException(status_code=404, detail="Paper not found")
paper["id"] = str(paper["_id"])
del paper["_id"]
return paper

```

app/routers/papers.py

```

from fastapi import APIRouter, UploadFile, File
from app.services.paper_service import create_paper_record, list_papers, get_paper_by_id

router = APIRouter(prefix="/papers", tags=["papers"])

@router.post("/upload")
def upload_paper(file: UploadFile = File(...)):
    return create_paper_record(file)

@router.get("/")
def get_all_papers():
    return list_papers()

@router.get("/{paper_id}")
def get_paper(paper_id: str):
    return get_paper_by_id(paper_id)

```

Step 2: Parse PDFs

Your synopsis phase 2 requires parsing PDFs into usable textual content. ^[48]

app/services/parser_service.py

```

import fitz
from datetime import datetime
from bson import ObjectId
from app.db import papers_collection

def parse_pdf_text(paper_id: str):
    paper = papers_collection.find_one({"_id": ObjectId(paper_id)})
    if not paper:
        raise ValueError("Paper not found")

    doc = fitz.open(paper["filepath"])
    pages = []
    full_text = []

    for i, page in enumerate(doc):
        text = page.get_text("text")
        pages.append({
            "page_number": i + 1,
            "text": text
        })
    full_text.append("".join(page["text"] for page in pages))

```

```

        full_text.append(text)

    combined_text = "\n".join(full_text)

    papers_collection.update_one(
        {"_id": ObjectId(paper_id)},
        {
            "$set": {
                "raw_text": combined_text,
                "pages": pages,
                "status": "parsed",
                "parsed_at": datetime.utcnow(),
            }
        }
    )

    return {
        "paper_id": paper_id,
        "pages_count": len(pages),
        "status": "parsed"
    }

```

Step 3: Chunk text

The synopsis requires cleaned, meaningful chunks tied to source context for semantic retrieval. ^[48]

app/services/chunk_service.py

```

from bson import ObjectId
from app.db import papers_collection, chunks_collection

def split_text(text: str, chunk_size: int = 1200, overlap: int = 200):
    chunks = []
    start = 0
    while start < len(text):
        end = start + chunk_size
        chunks.append(text[start:end])
        start += chunk_size - overlap
    return chunks

def chunk_paper(paper_id: str):
    paper = papers_collection.find_one({"_id": ObjectId(paper_id)})
    if not paper or "raw_text" not in paper:
        raise ValueError("Paper not parsed")

    chunks_collection.delete_many({"paper_id": paper_id})
    chunks = split_text(paper["raw_text"])

    docs = []
    for i, chunk in enumerate(chunks):
        docs.append({
            "paper_id": paper_id,
            "chunk_index": i,

```

```

        "section_title": "Unknown",
        "page_number": None,
        "text": chunk,
    })

    if docs:
        chunks_collection.insert_many(docs)

    return {"paper_id": paper_id, "chunks_created": len(docs)}

```

Step 4: Embeddings and vector store

Your methodology explicitly calls for embeddings in a vector database for semantic retrieval.

[48]

app/services/embedding_service.py

```

from sentence_transformers import SentenceTransformer
from app.config import settings

_model = None

def get_embedding_model():
    global _model
    if _model is None:
        _model = SentenceTransformer(settings.EMBEDDING_MODEL)
    return _model

def embed_texts(texts):
    model = get_embedding_model()
    return model.encode(texts).tolist()

```

app/services/vector_service.py

```

import chromadb
from app.config import VECTOR_DIR
from app.db import chunks_collection
from app.services.embedding_service import embed_texts

client = chromadb.PersistentClient(path=str(VECTOR_DIR))
collection = client.get_or_create_collection("research_papers")

def index_paper_chunks(paper_id: str):
    chunks = list(chunks_collection.find({"paper_id": paper_id}))
    if not chunks:
        raise ValueError("No chunks found for paper")

    ids = [str(c["_id"]) for c in chunks]
    texts = [c["text"] for c in chunks]
    embeddings = embed_texts(texts)
    metadatas = [{
        "paper_id": c["paper_id"],

```

```

        "chunk_index": c["chunk_index"],
        "section_title": c.get("section_title", "Unknown"),
    } for c in chunks]

collection.upsert(
    ids=ids,
    documents=texts,
    embeddings=embeddings,
    metadatas=metadatas
)

return {"paper_id": paper_id, "indexed_count": len(ids)}

```

Step 5: Search / semantic retrieval

This is the phase-3 bridge between corpus and grounded answers. ^[48]

app/services/compare_service.py

```

from bson import ObjectId
from app.db import extractions_collection, papers_collection

def compare_papers(paper_ids):
    rows = []
    extracted_docs = list(extractions_collection.find({"paper_id": {"$in": paper_ids}}))
    paper_map = {}
    for p in papers_collection.find({"_id": {"$in": [ObjectId(pid) for pid in paper_ids]}}):
        paper_map[str(p["_id"])] = p.get("title") or p.get("filename")

    fields = ["methodology", "dataset", "evaluation_metric", "limitations", "future_work"]

    for field in fields:
        row = {"field": field, "values": {}}
        for doc in extracted_docs:
            row["values"][paper_map.get(doc["paper_id"], doc["paper_id"])] = doc.get(field)
        rows.append(row)

    return {"rows": rows}

```

app/routers/analysis.py

```

from fastapi import APIRouter, HTTPException
from datetime import datetime
from bson import ObjectId

from app.models.schemas import CompareRequest
from app.services.parser_service import parse_pdf_text
from app.services.chunk_service import chunk_paper
from app.services.vector_service import index_paper_chunks
from app.services.extraction_service import extract_fields_for_paper
from app.services.compare_service import compare_papers
from app.db import papers_collection

```

```

router = APIRouter(prefix="/analysis", tags=["analysis"])

@router.post("/parse/{paper_id}")
def parse_paper(paper_id: str):
    try:
        result = parse_pdf_text(paper_id)
        chunk_paper(paper_id)
        index_paper_chunks(paper_id)
        papers_collection.update_one(
            {"_id": ObjectId(paper_id)},
            {"$set": {"status": "indexed", "indexed_at": datetime.utcnow()}}
        )
        return {"ok": True, **result, "status": "indexed"}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.post("/extract/{paper_id}")
def extract_paper(paper_id: str):
    try:
        return extract_fields_for_paper(paper_id)
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.post("/compare")
def compare_route(payload: CompareRequest):
    try:
        return compare_papers(payload.paper_ids)
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

```

app/routers/search.py

```

from fastapi import APIRouter, HTTPException
from app.models.schemas import SearchRequest
from app.services.search_service import semantic_search

router = APIRouter(prefix="/search", tags=["search"])

@router.post("/query")
def query_search(payload: SearchRequest):
    try:
        return semantic_search(payload.query, payload.paper_ids, payload.top_k or 5)
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

```

Step 6: Structured extraction

Your synopsis says the system should transform papers into structured, explainable knowledge like methodologies, datasets, baselines, metrics, limitations, and future work. For phase 1, extract a smaller subset. ^[48]

app/services/extraction_service.py

```
from bson import ObjectId
from datetime import datetime
from app.db import papers_collection, extractions_collection

def naive_extract_section(text: str, keyword: str, window: int = 600):
    idx = text.lower().find(keyword.lower())
    if idx == -1:
        return ""
    return text[idx: idx + window]

def extract_fields_for_paper(paper_id: str):
    paper = papers_collection.find_one({"_id": ObjectId(paper_id)})
    if not paper or "raw_text" not in paper:
        raise ValueError("Paper not parsed")

    text = paper["raw_text"]

    extraction = {
        "paper_id": paper_id,
        "objective": naive_extract_section(text, "objective"),
        "methodology": naive_extract_section(text, "method"),
        "dataset": naive_extract_section(text, "dataset"),
        "evaluation_metric": naive_extract_section(text, "accuracy"),
        "limitations": naive_extract_section(text, "limitation"),
        "future_work": naive_extract_section(text, "future work"),
        "created_at": datetime.utcnow(),
    }

    extractions_collection.update_one(
        {"paper_id": paper_id},
        {"$set": extraction},
        upsert=True
    )

    papers_collection.update_one(
        {"_id": ObjectId(paper_id)},
        {"$set": {"status": "extracted", "extracted_at": datetime.utcnow()}}
    )

    return extraction
```

For a better demo, replace `naive_extract_section` later with an LLM JSON extractor, but this version is enough to get phase 1 working end to end.

Requirements

requirements.txt

```
fastapi
uvicorn
python-multipart
pymongo
pydantic
pydantic-settings
pymupdf
sentence-transformers
chromadb
```

Frontend steps

Since you standardize on React/Next.js with FastAPI, build only two pages for phase 1. ^[50]

1. Paper Library page

Show:

- papers list,
- upload form,
- status badges,
- buttons: Parse, Extract, View, Compare.

2. Paper Detail / Compare page

Show:

- extracted fields,
- query box,
- top chunks,
- compare table.

Minimal API helper

```
import axios from "axios";

export const api = axios.create({
  baseURL: "http://localhost:8000",
});

export const uploadPaper = async (file: File) => {
  const formData = new FormData();
  formData.append("file", file);
  const res = await api.post("/papers/upload", formData);
  return res.data;
};
```

```

export const listPapers = async () => {
  const res = await api.get("/papers");
  return res.data;
};

export const parsePaper = async (paperId: string) => {
  const res = await api.post(`/analysis/parse/${paperId}`);
  return res.data;
};

export const extractPaper = async (paperId: string) => {
  const res = await api.post(`/analysis/extract/${paperId}`);
  return res.data;
};

export const searchPapers = async (query: string, paperIds: string[]) => {
  const res = await api.post("/search/query", { query, paper_ids: paperIds, top_k: 5 });
  return res.data;
};

export const comparePapers = async (paperIds: string[]) => {
  const res = await api.post("/analysis/compare", { paper_ids: paperIds });
  return res.data;
};

```

Exact implementation steps

1. Install MongoDB and ensure localhost:27017 is running.
2. Create the folder structure above.
3. Add config.py, db.py, schemas.py, main.py.
4. Implement /papers/upload, /papers, /papers/{id}.
5. Test file upload and DB insertion.
6. Implement parser service and /analysis/parse/{paper_id}.
7. Confirm raw_text and chunks are stored.
8. Add embeddings + Chroma indexing.
9. Implement /search/query.
10. Implement extraction service and /analysis/extract/{paper_id}.
11. Implement compare endpoint.
12. Build two frontend pages and wire buttons. ^[49] ^[48]

Demo flow

A strong phase-1 demo is:

- Upload 2 papers.
- Show they are stored with uploaded status.
- Parse one paper and show status becomes indexed.

- Open the paper and show extracted fields.
- Ask a question and show relevant chunks.
- Compare 2 papers in a table. ^[48]

That demonstrates the exact foundation your synopsis describes: structured corpus creation, parsing, retrieval, knowledge extraction, and comparison. ^[48]

What to defer

Do not add these yet:

- manuscript drafting,
- LaTeX export,
- reviewer simulation,
- novelty score,
- originality checker. ^[48]

Those are later-phase modules in the synopsis and will only slow you down right now. ^[48]

Paste your current backend folder tree or your existing `main.py`, and I'll merge this into your exact project structure instead of leaving it as a generic scaffold.

✱✱

give me detailed frontend steps

Yes — for phase 1, the frontend should be a **clean workflow dashboard**, not a big multi-page product. Since ResearchOS is supposed to help users collect papers, inspect extracted knowledge, run retrieval, and compare papers, your UI should focus on those actions first: paper library, paper detail/analysis, semantic search, and comparison workspace. Also, since you prefer stable dashboard structures and layered development, build the interface as one main dashboard with a few focused panels rather than redesigning everything page by page. ^[51]
^[52]

Frontend structure

For phase 1, build only these pages:

Page	Purpose
/dashboard	Paper library and upload entry point
/papers/[id]	Single-paper analysis view
/compare	Multi-paper comparison workspace

This is enough to support the structured corpus, searchable knowledge base, and cross-paper comparison goals in your synopsis. ^[52]

Recommended Next.js structure:

```
frontend/  
  app/  
    dashboard/  
      page.tsx  
    papers/  
      [id]/  
        page.tsx  
    compare/  
      page.tsx  
    layout.tsx  
    page.tsx  
  components/  
    layout/  
      AppShell.tsx  
      Sidebar.tsx  
      Topbar.tsx  
    papers/  
      UploadCard.tsx  
      PaperTable.tsx  
      PaperStatusBadge.tsx  
      PaperMetaCard.tsx  
      ExtractionCards.tsx  
      ChunkList.tsx  
    search/  
      SearchBox.tsx  
      SearchResults.tsx  
    compare/  
      CompareTable.tsx  
      PaperSelector.tsx  
    ui/  
      LoadingState.tsx  
      EmptyState.tsx  
      ErrorState.tsx  
  lib/  
    api.ts  
    types.ts
```

Page 1: Dashboard

The dashboard should act as the research corpus control center, because the synopsis starts with collecting papers, organizing metadata, and building a searchable base for all later analysis. This page should answer one question instantly: “What papers do I have, and what state are they in?”^[52]

Layout

Use a dashboard shell:

- left sidebar,
- topbar,

- main content area.

Sidebar items

Keep it minimal:

- Dashboard
- Compare
- Search
- Future modules (disabled or “coming soon”)

Since this is phase 1, you do not need Drafting, Reviewer Simulation, or Venue Recommendation in the live navigation yet, because those belong to later phases in the synopsis. ^[52]

Dashboard sections

Build the page in this order:

1. Upload area

Top-left card:

- drag-and-drop or file picker
- accepted: PDF only
- upload button
- success/error toast

This directly supports the “Paper Collection and Data Acquisition” phase. ^[52]

2. Paper stats strip

Show 4 compact summary cards:

- Total papers
- Parsed
- Indexed
- Extracted

This makes the system feel alive and shows progress through the pipeline defined in your methodology. ^[52]

3. Paper table

Main section:

- title / filename
- status
- upload date
- actions

Actions:

- View
- Parse
- Extract
- Select for Compare

The table should be the main element on the page because your project is workflow-first, similar to the way you prefer operator dashboards rather than flashy landing pages. ^[51] ^[53]

Dashboard visual hierarchy

Use this structure:

- top row: upload + stats
- bottom row: full-width paper table

Do not overcrowd the dashboard with charts or fancy cards yet. For phase 1, clarity matters more than polish.

Page 2: Paper detail

This is the most important frontend page in phase 1 because your synopsis emphasizes transforming raw papers into structured, explainable, and verifiable knowledge. The paper detail page should make one uploaded PDF feel “intelligent.” ^[52]

Page layout

Use a 3-column feel, even if implemented responsively:

- left: paper metadata and actions
- center: extracted knowledge and search
- right: retrieved chunks / source evidence

This layout matches the ethical, citation-grounded idea of keeping evidence visible next to generated or extracted content. ^[52]

Left panel: metadata

Show:

- title
- filename
- authors
- year
- abstract preview
- status timeline:
 - uploaded
 - parsed
 - indexed
 - extracted

Buttons:

- Parse paper
- Extract fields
- Re-index
- Add to compare

This supports the structured research corpus objective described in the synopsis.^[52]

Center panel: extracted knowledge

This is where the paper becomes useful.

Use cards for:

- Objective
- Methodology
- Dataset
- Evaluation metric
- Limitations
- Future work

Your synopsis explicitly lists these kinds of extracted research attributes as core outputs of the system.^[52]

UI suggestions:

- 2-column grid of cards
- each card with heading + short extracted text

- optional “show source” button later

For phase 1, keep each card compact and readable.

Center panel: search box

Below extraction cards, add a search area:

- text input: “Ask about this paper”
- submit button
- loading spinner

Example query placeholders:

- What dataset is used?
- What baselines are compared?
- What limitations are mentioned?

This maps directly to your RAG + semantic retrieval workflow. ^[52]

Right panel: evidence/results

Show retrieved chunks:

- section title
- page number
- chunk text
- similarity score optional

This right panel is important because your project vision stresses verifiable, citation-backed outputs instead of unsupported generation. ^[52]

Page 3: Compare

The compare page should exist because one of your strongest stated objectives is cross-paper comparison of methodologies, datasets, results, and limitations. This will likely be one of the most impressive demo screens. ^[52]

Compare page layout

Top:

- multi-select paper picker
- compare button

Body:

- structured comparison table

Optional:

- highlight differences in color
- blank cells if data missing

Compare table rows

At minimum show:

- Methodology
- Dataset
- Evaluation Metric
- Limitations
- Future Work

This directly matches your “Research Analysis and Comparison” phase.^[52]

UI behavior

- user selects 2–3 papers
- clicks Compare
- table loads in main panel
- each column is a paper title
- each row is one extracted attribute

This page should feel like a research workspace, not a generic data table.

Components to build

`AppShell.tsx`

This should wrap all pages so the layout stays stable, which matches your preference to keep the dashboard structure consistent while iterating.

Include:

- sidebar
- topbar
- page container

`Sidebar.tsx`

Items:

- Dashboard
- Compare
- Search

- Drafting (disabled)
- Reviewer Simulation (disabled)

Disabled future items still help communicate roadmap, but label them clearly as upcoming rather than functional.

`UploadCard.tsx`

Features:

- file picker
- upload button
- uploading state
- success/error text

`PaperTable.tsx`

Columns:

- title
- filename
- status
- uploaded_at
- actions

Actions:

- View
- Parse
- Extract
- Compare checkbox

`PaperStatusBadge.tsx`

Badge colors:

- uploaded = gray
- parsed = blue
- indexed = purple
- extracted = green

This visually communicates the modular pipeline from your methodology. ^[52]

`ExtractionCards.tsx`

Accept one extraction object and render cards.

Fields:

- objective
- methodology
- dataset
- evaluation metric
- limitations
- future work

`SearchBox.tsx`

Simple controlled input + button.

`SearchResults.tsx`

List of chunk cards:

- source section
- page number
- snippet

`PaperSelector.tsx`

Used in compare page:

- searchable dropdown or checkbox list
- select max 3 papers

`CompareTable.tsx`

Render comparison rows cleanly.

Use scrollable table on small screens.

State flow

Keep frontend state simple.

Dashboard state

- papers
- `uploading`
- `selectedCompareIds`

Paper detail state

- paper
- extraction
- query
- searchResults
- loadingParse
- loadingExtract
- loadingSearch

Compare state

- selectedPaperIds
- compareRows
- loadingCompare

This keeps the app manageable and demo-ready.

UX steps to implement

Dashboard workflow

1. User uploads PDF.
2. New row appears in table with uploaded.
3. User clicks Parse.
4. Status changes to indexed.
5. User clicks Extract.
6. Status changes to extracted.
7. User clicks View.

This mirrors the phased backend flow in your synopsis. [\[52\]](#)

Paper detail workflow

1. User opens paper page.
2. Metadata loads.
3. Extraction cards appear.
4. User asks a question.
5. Relevant chunks show in right panel.

Compare workflow

1. User selects 2–3 papers from dashboard or compare page.
2. Opens compare.
3. Table renders cross-paper comparison.

Suggested UI order to build

Build the frontend in this sequence:

Step 1

Create the app shell:

- `layout.tsx`
- `AppShell.tsx`
- `Sidebar.tsx`
- `Topbar.tsx`

Step 2

Build dashboard page static layout:

- upload card
- stats cards
- paper table

Step 3

Connect dashboard to backend:

- fetch papers
- upload paper
- parse paper
- extract paper

Step 4

Build paper detail page:

- metadata card
- extraction cards
- search box
- search results panel

Step 5

Connect detail page:

- fetch paper by id
- call search endpoint
- render chunk results

Step 6

Build compare page:

- paper selector
- compare table

Step 7

Connect compare page:

- fetch compare API response
- render rows dynamically

That layered build order matches how you like to work: functional workflow first, refinement later.

Styling guidance

Since this is a workflow dashboard, the frontend should look more like a compact SaaS/admin console than a landing page, which is also how you've preferred POC UIs in earlier projects. ^[53]

^[51]

Use:

- Tailwind
- subtle dark or neutral theme
- clean cards
- sticky sidebar
- compact tables
- clear status badges

Avoid:

- marketing hero sections
- oversized animations
- too many charts
- unrelated widgets

Phase-1 polish

Add only these polish items:

- loading spinners on buttons,
- empty state when no papers uploaded,
- error banner if parse/extract fails,
- badge colors for status,
- toast on upload success.

That is enough to look real without wasting time.

Best demo-ready structure

If you want the cleanest first-phase frontend, make it feel like this:

- **Dashboard:** upload + manage corpus ^[52]
- **Paper page:** inspect one paper as structured knowledge ^[52]
- **Compare page:** compare multiple papers for state-of-the-art analysis ^[52]

That directly reflects your synopsis claim that ResearchOS integrates literature exploration, knowledge extraction, comparative analysis, and research support in one unified workflow. ^[52]

Would you like me to next give you the exact Next.js component code structure for these pages, starting with AppShell, Dashboard, and PaperTable?

✱

give me relevant codes +how to make nextjs project

Use Next.js App Router with TypeScript; create-next-app is the standard way to scaffold it, and you can add shadcn/ui after setup for ready-made components. For your phase-1 ResearchOS frontend, build just three routes—dashboard, paper detail, and compare—and wire them to your backend with simple fetch wrappers. ^[54] ^[55] ^[56] ^[57]

Project setup

Create the app with App Router, TypeScript, ESLint, and alias support, then start the dev server. ^[55] ^[58] ^[54]

```
npx create-next-app@latest researchos-frontend
cd researchos-frontend
npm run dev
```

When prompted, choose:

- TypeScript: Yes
- ESLint: Yes

- Tailwind CSS: Yes
- src/ directory: No
- App Router: Yes
- Import alias: @/* ^[58] ^[54]

If you want shadcn/ui components, initialize it and add the few components you need, such as button, card, input, badge, table, and toast-related utilities. ^[56] ^[59]

```
npx shadcn@latest init
npx shadcn@latest add button card input badge table textarea separator sonner
npm i lucide-react
```

Folder structure

Keep the app small and modular so you can finish fast. The structure below maps directly to upload, parsing, retrieval, extraction, and comparison workflows from your synopsis. ^[57]

```
researchos-frontend/
  app/
    compare/
      page.tsx
    dashboard/
      page.tsx
    papers/
      [id]/
        page.tsx
    globals.css
    layout.tsx
    page.tsx
  components/
    layout/
      app-shell.tsx
      sidebar.tsx
      topbar.tsx
    papers/
      upload-card.tsx
      paper-table.tsx
      paper-status-badge.tsx
      extraction-cards.tsx
      chunk-results.tsx
    compare/
      paper-selector.tsx
      compare-table.tsx
    ui/
      loading-state.tsx
      empty-state.tsx
  lib/
    api.ts
    types.ts
    utils.ts
```

Core files

lib/types.ts

These types reflect the structured paper metadata, extraction, chunk evidence, and comparison data your frontend needs to display. ^[57]

```
export type PaperStatus = "uploaded" | "parsed" | "indexed" | "extracted";

export interface Paper {
  id: string;
  title: string;
  filename: string;
  authors?: string[];
  year?: number;
  abstract?: string;
  uploaded_at?: string;
  status: PaperStatus;
}

export interface Extraction {
  objective?: string;
  methodology?: string;
  dataset?: string;
  evaluation_metric?: string;
  limitations?: string;
  future_work?: string;
}

export interface ChunkResult {
  id: string;
  section_title?: string;
  page?: number;
  text: string;
  score?: number;
}

export interface CompareRow {
  field: string;
  values: Record<string, string>;
}
```

lib/api.ts

Keep API calls centralized so every page stays clean. This supports the paper collection, retrieval, extraction, and comparison actions in your workflow. ^[57]

```
import { CompareRow, Extraction, Paper, ChunkResult } from "../types";

const API_BASE = process.env.NEXT_PUBLIC_API_BASE || "http://localhost:8000";

async function api<T>(path: string, options?: RequestInit): Promise<T> {
```

```

const res = await fetch(`${API_BASE}${path}`, {
  ...options,
  headers: {
    ...(options?.headers || {}),
  },
  cache: "no-store",
});

if (!res.ok) {
  const text = await res.text();
  throw new Error(text || "Request failed");
}

return res.json();
}

export const getPapers = () => api<Paper[]>("/papers");
export const getPaper = (id: string) => api<Paper>(`/papers/${id}`);
export const getExtraction = (id: string) => api<Extraction>(`/papers/${id}/extractions`);
export const searchPaper = (id: string, query: string) =>
  api<ChunkResult[]>(`/papers/${id}/search`, {
    method: "POST",
    body: JSON.stringify({ query }),
    headers: { "Content-Type": "application/json" },
  });

export const comparePapers = (paperIds: string[]) =>
  api<CompareRow[]>("/compare", {
    method: "POST",
    body: JSON.stringify({ paper_ids: paperIds }),
    headers: { "Content-Type": "application/json" },
  });

export async function uploadPaper(file: File) {
  const formData = new FormData();
  formData.append("file", file);

  const res = await fetch(`${API_BASE}/papers/upload`, {
    method: "POST",
    body: formData,
  });

  if (!res.ok) throw new Error("Upload failed");
  return res.json();
}

export const parsePaper = (id: string) =>
  api(`/papers/${id}/parse`, { method: "POST" });

export const extractPaper = (id: string) =>
  api(`/papers/${id}/extract`, { method: "POST" });

```

app/layout.tsx

```
import './globals.css';
import { ReactNode } from 'react';

export const metadata = {
  title: 'ResearchOS',
  description: 'Ethical research workflow assistant',
};

export default function RootLayout({ children }: { children: ReactNode }) {
  return (
    <html lang="en">
      <body className="bg-zinc-950 text-zinc-100">{children}</body>
    </html>
  );
}
```

app/page.tsx

```
import { redirect } from 'next/navigation';

export default function Home() {
  redirect("/dashboard");
}
```

Layout components

components/layout/sidebar.tsx

```
"use client";

import Link from 'next/link';
import { usePathname } from 'next/navigation';
import { Files, GitCompare, Search, FlaskConical } from 'lucide-react';
import clsx from 'clsx';

const items = [
  { href: "/dashboard", label: "Dashboard", icon: Files },
  { href: "/compare", label: "Compare", icon: GitCompare },
  { href: "/search", label: "Search", icon: Search },
];

export function Sidebar() {
  const pathname = usePathname();

  return (
    <aside className="w-64 border-r border-zinc-800 bg-zinc-900/70 p-4">
      <div className="mb-8 flex items-center gap-2 text-xl font-semibold">
        <FlaskConical className="h-5 w-5 text-emerald-400" />
        ResearchOS
      </div>
      <ul>
        {items.map((item) => (
          <li>
            <Link href={item.href} className={clsx(
              "text-zinc-400 hover:text-zinc-100",
              pathname === item.href & "text-zinc-100"
            )}>
              {item.icon} {item.label}
            </Link>
          </li>
        ))}
      </ul>
    </aside>
  );
}
```

```

</div>

<nav className="space-y-2">
  {items.map((item) => {
    const Icon = item.icon;
    const active = pathname.startsWith(item.href);
    return (
      <Link
        key={item.label}
        href={item.href}
        className={clsx(
          "flex items-center gap-3 rounded-lg px-3 py-2 text-sm transition",
          active
            ? "bg-emerald-500/15 text-emerald-300"
            : "text-zinc-300 hover:bg-zinc-800 hover:text-white"
        )}
      >
        <Icon className="h-4 w-4" />
        {item.label}
      </Link>
    );
  })}
</nav>

<div className="mt-8 rounded-lg border border-dashed border-zinc-700 p-3 text-zinc-400">
  Upcoming: Drafting, reviewer simulation, integrity checks.
</div>
</aside>
);
}

```

components/layout/topbar.tsx

```

export function Topbar() {
  return (
    <header className="flex h-16 items-center justify-between border-b border-zinc-800">
      <div>
        <h1 className="text-lg font-semibold">Research Workflow Dashboard</h1>
        <p className="text-sm text-zinc-400">
          Upload, extract, search, and compare papers
        </p>
      </div>
    </header>
  );
}

```

components/layout/app-shell.tsx

```

import { ReactNode } from "react";
import { Sidebar } from "../sidebar";
import { Topbar } from "../topbar";

export function AppShell({ children }: { children: ReactNode }) {

```

```

    return (
      <div className="flex min-h-screen">
        <Sidebar />
        <div className="flex min-w-0 flex-1 flex-col">
          <Topbar />
          <main className="flex-1 p-6">{children}</main>
        </div>
      </div>
    );
  }
}

```

Dashboard

components/papers/paper-status-badge.tsx

```

import { PaperStatus } from "@lib/types";

const styles: Record<PaperStatus, string> = {
  uploaded: "bg-zinc-700/50 text-zinc-200",
  parsed: "bg-blue-500/15 text-blue-300",
  indexed: "bg-violet-500/15 text-violet-300",
  extracted: "bg-emerald-500/15 text-emerald-300",
};

export function PaperStatusBadge({ status }: { status: PaperStatus }) {
  return (
    <span className={`rounded-full px-2.5 py-1 text-xs font-medium ${styles[status]}`}
      {status}
    </span>
  );
}

```

components/papers/upload-card.tsx

```

"use client";

import { useState } from "react";
import { uploadPaper } from "@lib/api";

export function UploadCard({ onUploaded }: { onUploaded: () => void }) {
  const [file, setFile] = useState<File | null>(null);
  const [loading, setLoading] = useState(false);

  const handleUpload = async () => {
    if (!file) return;
    try {
      setLoading(true);
      await uploadPaper(file);
      setFile(null);
      onUploaded();
    } catch (e) {
      alert("Upload failed");
    }
  };
}

```

```

    } finally {
      setLoading(false);
    }
  };

  return (
    <div className="rounded-2xl border border-zinc-800 bg-zinc-900 p-5">
      <h2 className="mb-2 text-lg font-semibold">Upload paper</h2>
      <p className="mb-4 text-sm text-zinc-400">Add a research PDF to the corpus.</p>

      <input
        type="file"
        accept="application/pdf"
        onChange={(e) => setFile(e.target.files?.[0] || null)}
        className="mb-4 block w-full text-sm text-zinc-300"
      />

      <button
        onClick={handleUpload}
        disabled={!file || loading}
        className="rounded-lg bg-emerald-500 px-4 py-2 text-sm font-medium text-black"
      >
        {loading ? "Uploading..." : "Upload PDF"}
      </button>
    </div>
  );
}

```

components/papers/paper-table.tsx

```

"use client";

import Link from "next/link";
import { extractPaper, parsePaper } from "@lib/api";
import { Paper } from "@lib/types";
import { PaperStatusBadge } from "../paper-status-badge";
import { useState } from "react";

export function PaperTable({
  papers,
  onRefresh,
}: {
  papers: Paper[];
  onRefresh: () => void;
}) {
  const [loadingId, setLoadingId] = useState<string | null>(null);

  const runAction = async (id: string, action: "parse" | "extract") => {
    try {
      setLoadingId(`${id}-${action}`);
      if (action === "parse") await parsePaper(id);
      else await extractPaper(id);
      onRefresh();
    } catch {
      alert(`${action} failed`);
    }
  };
}

```

```

    } finally {
      setLoadingId(null);
    }
  };

return (
  <div className="overflow-hidden rounded-2xl border border-zinc-800 bg-zinc-900">
    <table className="w-full text-left text-sm">
      <thead className="border-b border-zinc-800 bg-zinc-950/50 text-zinc-400">
        <tr>
          <th className="px-4 py-3">Title</th>
          <th className="px-4 py-3">Filename</th>
          <th className="px-4 py-3">Status</th>
          <th className="px-4 py-3">Year</th>
          <th className="px-4 py-3">Actions</th>
        </tr>
      </thead>
      <tbody>
        {papers.map((paper) => (
          <tr key={paper.id} className="border-b border-zinc-800/70">
            <td className="px-4 py-3">{paper.title || "Untitled paper"}</td>
            <td className="px-4 py-3 text-zinc-400">{paper.filename}</td>
            <td className="px-4 py-3">
              <PaperStatusBadge status={paper.status} />
            </td>
            <td className="px-4 py-3">{paper.year || "-"}</td>
            <td className="px-4 py-3">
              <div className="flex flex-wrap gap-2">
                <Link
                  href={`/${papers}/${paper.id}`}
                  className="rounded-lg border border-zinc-700 px-3 py-1.5"
                >
                  View
                </Link>
                <button
                  onClick={() => runAction(paper.id, "parse")}
                  className="rounded-lg border border-zinc-700 px-3 py-1.5"
                >
                  {loadingId === `${paper.id}-parse` ? "Parsing..." : "Parse"}
                </button>
                <button
                  onClick={() => runAction(paper.id, "extract")}
                  className="rounded-lg border border-zinc-700 px-3 py-1.5"
                >
                  {loadingId === `${paper.id}-extract` ? "Extracting..." : "Extract"}
                </button>
              </div>
            </td>
          </tr>
        )))
      </tbody>
    </table>
  </div>
);
}

```

app/dashboard/page.tsx

This dashboard is the corpus entry point for upload, parsing, extraction, and tracking paper status. ^[57]

```
"use client";

import { useEffect, useState } from "react";
import { AppShell } from "@/components/layout/app-shell";
import { UploadCard } from "@/components/papers/upload-card";
import { PaperTable } from "@/components/papers/paper-table";
import { getPapers } from "@/lib/api";
import { Paper } from "@/lib/types";

export default function DashboardPage() {
  const [papers, setPapers] = useState<Paper[]>([]);
  const [loading, setLoading] = useState(true);

  const loadPapers = async () => {
    try {
      setLoading(true);
      const data = await getPapers();
      setPapers(data);
    } catch {
      alert("Failed to fetch papers");
    } finally {
      setLoading(false);
    }
  };

  useEffect(() => {
    loadPapers();
  }, []);

  const stats = {
    total: papers.length,
    parsed: papers.filter((p) => ["parsed", "indexed", "extracted"].includes(p.status)).length,
    indexed: papers.filter((p) => ["indexed", "extracted"].includes(p.status)).length,
    extracted: papers.filter((p) => p.status === "extracted").length,
  };

  return (
    <AppShell>
      <div className="grid gap-6 lg:grid-cols-[360px_1fr]">
        <UploadCard onUploaded={loadPapers} />

        <div className="grid gap-4 sm:grid-cols-2 xl:grid-cols-4">
          {Object.entries(stats).map(([label, value]) => (
            <div key={label} className="rounded-2xl border border-zinc-800 bg-zinc-900 p-4">
              <p className="text-sm capitalize text-zinc-400">{label}</p>
              <p className="mt-2 text-3xl font-semibold">{value}</p>
            </div>
          ))}
        </div>
      </div>
    </AppShell>
  );
}
```

```

    <div className="mt-6">
      {loading ? (
        <div className="rounded-2xl border border-zinc-800 bg-zinc-900 p-6">
          Loading papers...
        </div>
      ) : (
        <PaperTable papers={papers} onRefresh={loadPapers} />
      )}
    </div>
  </AppShell>
);
}

```

Paper detail page

components/papers/extraction-cards.tsx

These cards expose the structured fields your synopsis prioritizes, such as methodology, datasets, metrics, limitations, and future directions. ^[57]

```

import { Extraction } from "@lib/types";

const fields: { key: keyof Extraction; label: string }[] = [
  { key: "objective", label: "Objective" },
  { key: "methodology", label: "Methodology" },
  { key: "dataset", label: "Dataset" },
  { key: "evaluation_metric", label: "Evaluation Metric" },
  { key: "limitations", label: "Limitations" },
  { key: "future_work", label: "Future Work" },
];

export function ExtractionCards({ extraction }: { extraction: Extraction }) {
  return (
    <div className="grid gap-4 md:grid-cols-2">
      {fields.map((field) => (
        <div key={field.key} className="rounded-2xl border border-zinc-800 bg-zinc-900 p-6">
          <h3 className="mb-2 text-sm font-semibold text-emerald-300">{field.label}</h3>
          <p className="text-sm leading-6 text-zinc-300">
            {extraction[field.key] || "Not extracted yet."}
          </p>
        </div>
      ))}
    </div>
  );
}

```

components/papers/chunk-results.tsx

Keeping evidence visible helps reflect the citation-grounded, verifiable approach described in your project concept. ^[57]

```
import { ChunkResult } from "@lib/types";

export function ChunkResults({ results }: { results: ChunkResult[] }) {
  return (
    <div className="space-y-3">
      {results.map((chunk) => (
        <div key={chunk.id} className="rounded-xl border border-zinc-800 bg-zinc-900">
          <div className="mb-2 flex items-center justify-between text-xs text-zinc-400">
            <span>{chunk.section_title || "Section"}</span>
            <span>Page {chunk.page || "-"}</span>
          </div>
          <p className="text-sm leading-6 text-zinc-300">{chunk.text}</p>
        </div>
      ))}
    </div>
  );
}
```

app/papers/[id]/page.tsx

```
"use client";

import { useEffect, useState } from "react";
import { AppShell } from "@components/layout/app-shell";
import { getExtraction, getPaper, searchPaper } from "@lib/api";
import { ChunkResult, Extraction, Paper } from "@lib/types";
import { ExtractionCards } from "@components/papers/extraction-cards";
import { ChunkResults } from "@components/papers/chunk-results";

export default function PaperDetailPage({
  params,
}: {
  params: { id: string };
}) {
  const [paper, setPaper] = useState<Paper | null>(null);
  const [extraction, setExtraction] = useState<Extraction>({});
  const [query, setQuery] = useState("");
  const [results, setResults] = useState<ChunkResult[]>([]);
  const [searching, setSearching] = useState(false);

  useEffect(() => {
    async function load() {
      const [paperData, extractionData] = await Promise.all([
        getPaper(params.id),
        getExtraction(params.id).catch(() => ({})),
      ]);
      setPaper(paperData);
      setExtraction(extractionData);
    }
  });
}
```

```

    }
    load();
  }, [params.id]);

const handleSearch = async () => {
  if (!query.trim()) return;
  try {
    setSearching(true);
    const data = await searchPaper(params.id, query);
    setResults(data);
  } catch {
    alert("Search failed");
  } finally {
    setSearching(false);
  }
};

if (!paper) {
  return (
    <AppShell>
      <div>Loading paper...</div>
    </AppShell>
  );
}

return (
  <AppShell>
    <div className="grid gap-6 xl:grid-cols-[300px_1fr_360px]">
      <div className="rounded-2xl border border-zinc-800 bg-zinc-900 p-5">
        <h2 className="text-lg font-semibold">{paper.title}</h2>
        <p className="mt-2 text-sm text-zinc-400">{paper.filename}</p>
        <div className="mt-4 space-y-2 text-sm text-zinc-300">
          <p><span className="text-zinc-500">Year:</span> {paper.year || "-"}</p>
          <p><span className="text-zinc-500">Authors:</span> {paper.authors?.join('')}</p>
          <p><span className="text-zinc-500">Status:</span> {paper.status}</p>
        </div>
        <div className="mt-5">
          <h3 className="mb-2 text-sm font-semibold text-zinc-200">Abstract</h3>
          <p className="text-sm leading-6 text-zinc-400">
            {paper.abstract || "No abstract available."}
          </p>
        </div>
      </div>

      <div className="space-y-6">
        <ExtractionCards extraction={extraction} />
        <div className="rounded-2xl border border-zinc-800 bg-zinc-900 p-5">
          <h3 className="mb-3 text-lg font-semibold">Ask this paper</h3>
          <div className="flex gap-3">
            <input
              value={query}
              onChange={(e) => setQuery(e.target.value)}
              placeholder="What dataset is used?"
              className="flex-1 rounded-lg border border-zinc-700 bg-zinc-950 px-4"
            />
            <button

```

```

        onClick={handleSearch}
        className="rounded-lg bg-emerald-500 px-4 py-2 text-sm font-medium text-white"
      >
        {searching ? "Searching..." : "Search"}
      </button>
    </div>
  </div>
</div>

<div>
  <h3 className="mb-3 text-lg font-semibold">Evidence</h3>
  <ChunkResults results={results} />
</div>
</div>
</AppShell>
);
}

```

Compare page

components/compare/paper-selector.tsx

```

"use client";

import { Paper } from "@lib/types";

export function PaperSelector({
  papers,
  selected,
  onChange,
}: {
  papers: Paper[];
  selected: string[];
  onChange: (ids: string[]) => void;
}) {
  const toggle = (id: string) => {
    if (selected.includes(id)) onChange(selected.filter((x) => x !== id));
    else if (selected.length < 3) onChange([...selected, id]);
  };

  return (
    <div className="grid gap-3 md:grid-cols-2 xl:grid-cols-3">
      {papers.map((paper) => (
        <label
          key={paper.id}
          className="flex cursor-pointer items-start gap-3 rounded-xl border border-gray-200 p-3"
        >
          <input
            type="checkbox"
            checked={selected.includes(paper.id)}
            onChange={() => toggle(paper.id)}
            className="mt-1"
          />
        </div>
      ))}
    </div>
  );
}

```

```

        <p className="font-medium">{paper.title}</p>
        <p className="text-sm text-zinc-400">{paper.filename}</p>
      </div>
    </label>
  )}
</div>
);
}

```

components/compare/compare-table.tsx

Comparison tables are a direct frontend expression of your cross-paper analysis objective. ^[57]

```

import { CompareRow } from "@lib/types";

export function CompareTable({
  rows,
  paperTitles,
}: {
  rows: CompareRow[];
  paperTitles: Record<string, string>;
}) {
  const paperIds = Object.keys(paperTitles);

  return (
    <div className="overflow-x-auto rounded-2xl border border-zinc-800 bg-zinc-900">
      <table className="w-full min-w-[800px] text-left text-sm">
        <thead className="border-b border-zinc-800 bg-zinc-950/50">
          <tr>
            <th className="px-4 py-3">Field</th>
            {paperIds.map((id) => (
              <th key={id} className="px-4 py-3">{paperTitles[id]}</th>
            ))}
          </tr>
        </thead>
        <tbody>
          {rows.map((row) => (
            <tr key={row.field} className="border-b border-zinc-800/70">
              <td className="px-4 py-3 font-medium text-emerald-300">{row.field}</td>
              {paperIds.map((id) => (
                <td key={id} className="px-4 py-3 text-zinc-300">
                  {row.values[id] || "-"}
                </td>
              ))}
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  );
}

```

```

"use client";

import { useEffect, useState } from "react";
import { AppShell } from "@components/layout/app-shell";
import { comparePapers, getPapers } from "@lib/api";
import { CompareRow, Paper } from "@lib/types";
import { PaperSelector } from "@components/compare/paper-selector";
import { CompareTable } from "@components/compare/compare-table";

export default function ComparePage() {
  const [papers, setPapers] = useState<Paper[]>([]);
  const [selected, setSelected] = useState<string[]>([]);
  const [rows, setRows] = useState<CompareRow[]>([]);
  const [loading, setLoading] = useState(false);

  useEffect(() => {
    getPapers().then(setPapers).catch(() => alert("Failed to load papers"));
  }, []);

  const handleCompare = async () => {
    try {
      setLoading(true);
      const data = await comparePapers(selected);
      setRows(data);
    } catch {
      alert("Compare failed");
    } finally {
      setLoading(false);
    }
  };

  const paperTitles = Object.fromEntries(
    papers.filter((p) => selected.includes(p.id)).map((p) => [p.id, p.title])
  );

  return (
    <AppShell>
      <div className="space-y-6">
        <div className="rounded-2xl border border-zinc-800 bg-zinc-900 p-5">
          <h2 className="mb-2 text-lg font-semibold">Select papers to compare</h2>
          <p className="mb-4 text-sm text-zinc-400">Choose up to 3 extracted papers.</p>
          <PaperSelector papers={papers} selected={selected} onChange={setSelected} />
          <button
            onClick={handleCompare}
            disabled={selected.length < 2 || loading}
            className="mt-4 rounded-lg bg-emerald-500 px-4 py-2 text-sm font-medium"
          >
            {loading ? "Comparing..." : "Compare Papers"}
          </button>
        </div>

        {rows.length > 0 && <CompareTable rows={rows} paperTitles={paperTitles} />}
      </div>
    </AppShell>
  );
}

```

```
);  
}
```

Global styling

app/globals.css

```
@tailwind base;  
@tailwind components;  
@tailwind utilities;  
  
html, body {  
  margin: 0;  
  padding: 0;  
}  
  
* {  
  box-sizing: border-box;  
}
```

Backend endpoints expected

Your frontend above assumes these endpoints exist, which line up with upload, parsing, extraction, retrieval, and comparison phases in the synopsis. ^[57]

```
GET    /papers  
POST   /papers/upload  
GET    /papers/:id  
POST   /papers/:id/parse  
POST   /papers/:id/extract  
GET    /papers/:id/extraction  
POST   /papers/:id/search  
POST   /compare
```

How to run

Create an environment file:

.env.local

```
NEXT_PUBLIC_API_BASE=http://localhost:8000
```

Then run:

```
npm run dev
```

Open <http://localhost:3000/dashboard>. ^[58]

What to build first

To hit your deadline, implement in this order:

1. Scaffold Next.js app. [\[54\]](#) [\[55\]](#)
2. Create AppShell, Sidebar, and Topbar.
3. Build dashboard UI.
4. Connect upload and fetch papers.
5. Add parse and extract actions.
6. Build paper detail page.
7. Add search results panel.
8. Build compare page.

This gives you a usable phase-1 frontend for the structured corpus, semantic retrieval, extraction display, and cross-paper comparison workflow described in your synopsis. [\[57\]](#)

Would you like me to give you the **matching FastAPI backend starter code** for these exact endpoints next?



theres no css styling, pls fix this

Your screenshot shows the app is rendering as plain HTML, which usually means Tailwind is **not being loaded at all** or `globals.css` is missing the Tailwind directives. In Next.js App Router, global styles must be imported in `app/layout.tsx`, and `app/globals.css` must include Tailwind's stylesheet directives for the utility classes to apply. [\[70\]](#) [\[71\]](#) [\[72\]](#) [\[73\]](#)

Most likely fix

If you created the project with Tailwind support, the fastest fix is to make sure these files look like this.

`app/globals.css`

For the common Tailwind setup, your global stylesheet needs the Tailwind directives; without them, classes like `bg-zinc-950`, `flex`, and `rounded-2xl` won't generate any CSS. [\[71\]](#) [\[74\]](#) [\[75\]](#) [\[70\]](#)

```
@tailwind base;
@tailwind components;
@tailwind utilities;

:root {
  --background: #09090b;
  --foreground: #fafafa;
}

html,
```

```
body {
  margin: 0;
  padding: 0;
  background: var(--background);
  color: var(--foreground);
  font-family: Arial, Helvetica, sans-serif;
}

* {
  box-sizing: border-box;
}
```

app/layout.tsx

In App Router, `globals.css` must be imported in the root layout so styles apply across all routes. [\[74\]](#) [\[72\]](#) [\[73\]](#) [\[71\]](#)

```
import "../globals.css";
import type { ReactNode } from "react";

export const metadata = {
  title: "ResearchOS",
  description: "Ethical research workflow assistant",
};

export default function RootLayout({ children }: { children: ReactNode }) {
  return (
    <html lang="en">
      <body>{children}</body>
    </html>
  );
}
```

If you are on newer Tailwind

If your project uses the newer Tailwind CSS flow from recent create-next-app templates, the generated `globals.css` may use a single import instead of the three `@tailwind` directives. Tailwind v4-style setups can use a direct Tailwind import and theme customization in CSS. [\[76\]](#)

Try this version if the first one throws errors:

app/globals.css **for newer setups**

```
@import "tailwindcss";

:root {
  --background: #09090b;
  --foreground: #fafafa;
}

html,
body {
```

```

margin: 0;
padding: 0;
background: var(--background);
color: var(--foreground);
font-family: Arial, Helvetica, sans-serif;
}

* {
  box-sizing: border-box;
}

```

What to check next

These are the usual reasons Tailwind appears “missing” in Next.js:

- `globals.css` not imported in `app/layout.tsx`. [\[72\]](#) [\[73\]](#)
- Tailwind directives missing from `app/globals.css`. [\[75\]](#) [\[70\]](#) [\[74\]](#)
- Wrong file name, such as importing `global.css` instead of `globals.css`. [\[77\]](#)
- Dev server not restarted after config changes. [\[73\]](#)

Minimal working test

Add this temporarily to `app/dashboard/page.tsx` to verify styling is active:

```

export default function DashboardPage() {
  return (
    <div className="min-h-screen bg-zinc-950 p-10 text-white">
      <div className="rounded-2xl border border-zinc-800 bg-zinc-900 p-6 shadow-2xl">
        <h1 className="text-3xl font-bold text-emerald-400">Tailwind is working</h1>
        <p className="mt-2 text-zinc-300">If this looks dark and styled, your CSS is
      </div>
    </div>
  );
}

```

Better styling baseline

Once Tailwind starts working, use this improved `globals.css` so the app looks cleaner by default.

```

@tailwind base;
@tailwind components;
@tailwind utilities;

:root {
  --background: #09090b;
  --foreground: #fafafa;
}

html {

```

```

    color-scheme: dark;
  }

  html,
  body {
    margin: 0;
    padding: 0;
    background: var(--background);
    color: var(--foreground);
    font-family: Inter, Arial, Helvetica, sans-serif;
  }

  body {
    min-height: 100vh;
  }

  * {
    box-sizing: border-box;
  }

  a {
    color: inherit;
    text-decoration: none;
  }

  button,
  input,
  textarea,
  select {
    font: inherit;
  }

```

If you want the full fixed UI

After CSS starts loading, your earlier components should render with dark cards, spacing, rounded corners, and sidebar styling because they already use Tailwind utility classes. The project structure and UI components I suggested are aligned with your phase-1 ResearchOS workflow of upload, parsing, retrieval, extraction, and comparison. ^[78]

Paste these two files first, restart with:

```
npm run dev
```

Then check the dashboard again. Would you like me to give you a **full working** `globals.css` + `dashboard/page.tsx` + `paper table` **version** you can paste directly?

✱

i want it to look user friendly with good styling and use light colors

Yes — switch it to a light academic dashboard with soft backgrounds, white cards, subtle borders, blue accents, and better spacing. In Next.js App Router, global styling should live in `app/globals.css`, and fonts can be applied globally from `next/font/google` in `app/layout.tsx` for a cleaner, more polished UI. [\[87\]](#) [\[88\]](#) [\[89\]](#)

Design direction

For ResearchOS, a light interface fits well because the product is about reading, comparing, and organizing research papers rather than showing a dark developer console. Your synopsis emphasizes structured knowledge extraction, comparison, and evidence-backed workflows, so the UI should feel calm, readable, and trustworthy. [\[90\]](#)

Use this visual style:

- background: very light slate
- cards: white
- primary accent: blue
- secondary accent: teal
- text: dark slate
- borders: soft gray
- radius: medium-large
- spacing: generous [\[90\]](#)

Replace `app/layout.tsx`

Use a clean font setup with Inter, which Next.js supports via `next/font/google`. This gives you a more polished base and avoids the default serif look. [\[88\]](#) [\[89\]](#)

```
import './globals.css';
import { Inter } from 'next/font/google';
import type { ReactNode } from 'react';

const inter = Inter({
  subsets: ['latin'],
  variable: '--font-sans',
});

export const metadata = {
  title: 'ResearchOS',
  description: 'Ethical research workflow assistant',
};

export default function RootLayout({ children }: { children: ReactNode }) {
  return (
```

```

    <html lang="en">
      <body className={` ${inter.variable} font-sans`} >{children}</body>
    </html>
  );
}

```

Replace `app/globals.css`

This gives you a soft light theme, cleaner typography, better spacing, nicer scrollbars, and reusable utility classes. Tailwind utilities only work if your global stylesheet includes the Tailwind import or directives, depending on your setup. [\[89\]](#) [\[91\]](#) [\[92\]](#)

If your project is using the newer Tailwind style, use this:

```

@import "tailwindcss";

:root {
  --background: #f8fbff;
  --foreground: #0f172a;
  --card: #ffffff;
  --card-2: #fdfefe;
  --border: #dbe4ee;
  --muted: #64748b;
  --primary: #2563eb;
  --primary-soft: #dbeafe;
  --success: #0f766e;
  --success-soft: #ccfbf1;
  --warning: #b45309;
  --warning-soft: #fef3c7;
  --sidebar: #eef4ff;
}

html {
  scroll-behavior: smooth;
}

body {
  margin: 0;
  min-height: 100vh;
  background:
    radial-gradient(circle at top left, #eef6ff 0%, transparent 35%),
    radial-gradient(circle at bottom right, #f0fdf4 0%, transparent 30%),
    var(--background);
  color: var(--foreground);
  font-family: var(--font-sans), Arial, Helvetica, sans-serif;
}

* {
  box-sizing: border-box;
}

a {
  color: inherit;
  text-decoration: none;
}

```

```

}

button,
input,
textarea,
select {
  font: inherit;
}

table {
  border-collapse: collapse;
}

@layer utilities {
  .bg-app {
    background-color: var(--background);
  }

  .bg-card {
    background-color: var(--card);
  }

  .bg-card-soft {
    background-color: var(--card-2);
  }

  .bg-sidebar {
    background-color: var(--sidebar);
  }

  .text-main {
    color: var(--foreground);
  }

  .text-muted {
    color: var(--muted);
  }

  .border-soft {
    border-color: var(--border);
  }

  .shadow-soft {
    box-shadow:
      0 8px 24px rgba(15, 23, 42, 0.06),
      0 2px 8px rgba(15, 23, 42, 0.04);
  }

  .ring-soft {
    box-shadow: 0 0 0 4px rgba(37, 99, 235, 0.08);
  }
}

```

If your project expects the older directives, use this instead:

```

@tailwind base;
@tailwind components;
@tailwind utilities;

:root {
  --background: #f8fbff;
  --foreground: #0f172a;
  --card: #ffffff;
  --card-2: #fdfefe;
  --border: #dbe4ee;
  --muted: #64748b;
  --primary: #2563eb;
  --primary-soft: #dbeafe;
  --success: #0f766e;
  --success-soft: #ccfbf1;
  --warning: #b45309;
  --warning-soft: #fef3c7;
  --sidebar: #eef4ff;
}

html {
  scroll-behavior: smooth;
}

body {
  margin: 0;
  min-height: 100vh;
  background:
    radial-gradient(circle at top left, #eef6ff 0%, transparent 35%),
    radial-gradient(circle at bottom right, #f0fdf4 0%, transparent 30%),
    var(--background);
  color: var(--foreground);
  font-family: var(--font-sans), Arial, Helvetica, sans-serif;
}

* {
  box-sizing: border-box;
}

a {
  color: inherit;
  text-decoration: none;
}

button,
input,
textarea,
select {
  font: inherit;
}

table {
  border-collapse: collapse;
}

```

Better sidebar

Your current sidebar looks too plain and cramped. This version makes it softer, more app-like, and easier to scan.

components/layout/sidebar.tsx

```
"use client";

import Link from "next/link";
import { usePathname } from "next/navigation";
import {
  LayoutDashboard,
  GitCompareArrows,
  Search,
  FlaskConical,
  FileText,
} from "lucide-react";
import clsx from "clsx";

const items = [
  { href: "/dashboard", label: "Dashboard", icon: LayoutDashboard },
  { href: "/compare", label: "Compare Papers", icon: GitCompareArrows },
  { href: "/search", label: "Search", icon: Search },
];

export function Sidebar() {
  const pathname = usePathname();

  return (
    <aside className="hidden w-72 flex-col border-r border-slate-200 bg-sidebar px-5" >
      <div className="mb-8 flex items-center gap-3">
        <div className="flex h-11 w-11 items-center justify-center rounded-2xl bg-blue-100">
          <FlaskConical className="h-5 w-5" />
        </div>
        <div>
          <h1 className="text-lg font-semibold text-slate-900">ResearchOS</h1>
          <p className="text-xs text-slate-500">Research workflow copilot</p>
        </div>
      </div>

      <nav className="space-y-2">
        {items.map((item) => {
          const Icon = item.icon;
          const active = pathname.startsWith(item.href);

          return (
            <Link
              key={item.href}
              href={item.href}
              className={clsx(
                "flex items-center gap-3 rounded-2xl px-4 py-3 text-sm font-medium transition-colors",
                active
                  ? "bg-white text-blue-700 shadow-sm ring-1 ring-blue-100"
                  : "text-slate-600 hover:bg-white hover:text-slate-900 hover:shadow-sm"
              )}>
              <Icon className="h-5 w-5" />
              {item.label}
            </Link>
          );
        })}
      </nav>
    </aside>
  );
}
```

```

    })
    >
    <Icon className="h-4 w-4" />
    {item.label}
  </Link>
);
}}
</nav>

<div className="mt-8 rounded-2xl border border-blue-100 bg-white p-4 shadow-sm">
  <div className="mb-2 flex items-center gap-2">
    <FileText className="h-4 w-4 text-blue-600" />
    <p className="text-sm font-semibold text-slate-900">Phase 1 progress</p>
  </div>
  <p className="text-sm leading-6 text-slate-600">
    Paper upload, parsing, search, extraction, and comparison can be shown in
    review.
  </p>
</div>
</aside>
);
}

```

Better topbar

components/layout/topbar.tsx

```

export function Topbar() {
  return (
    <header className="border-b border-slate-200 bg-white/80 backdrop-blur">
      <div className="flex min-h-20 items-center justify-between px-6 py-4">
        <div>
          <h1 className="text-3xl font-semibold tracking-tight text-slate-900">
            Research Workflow Dashboard
          </h1>
          <p className="mt-1 text-sm text-slate-500">
            Upload, extract, search, and compare research papers
          </p>
        </div>

        <div className="hidden rounded-2xl border border-slate-200 bg-slate-50 px-4 py-2">
          Ethical • Citation-grounded • Structured
        </div>
      </div>
    </header>
  );
}

```

Better layout shell

components/layout/app-shell.tsx

```
import { ReactNode } from "react";
import { Sidebar } from "../sidebar";
import { Topbar } from "../topbar";

export function AppShell({ children }: { children: ReactNode }) {
  return (
    <div className="min-h-screen bg-app text-main">
      <div className="flex min-h-screen">
        <Sidebar />
        <div className="flex min-w-0 flex-1 flex-col">
          <Topbar />
          <main className="flex-1 px-4 py-6 md:px-8">{children}</main>
        </div>
      </div>
    </div>
  );
}
```

Better dashboard page

This dashboard feels much more user-friendly: soft stats cards, a clean upload panel, and a better table container. That fits your goal of showing uploaded papers, processing status, and a searchable corpus for the first project phase. ^[90]

app/dashboard/page.tsx

```
"use client";

import { useEffect, useState } from "react";
import { AppShell } from "@components/layout/app-shell";
import { UploadCard } from "@components/papers/upload-card";
import { PaperStatusBadge } from "@components/papers/paper-status-badge";
import { getPapers } from "@lib/api";
import { Paper } from "@lib/types";
import Link from "next/link";
import { ArrowRight, FileText, Database, Sparkles, SearchCheck } from "lucide-react"

export default function DashboardPage() {
  const [papers, setPapers] = useState<Paper[]>([]);
  const [loading, setLoading] = useState(true);

  async function loadPapers() {
    try {
      setLoading(true);
      const data = await getPapers();
      setPapers(data);
    } catch (error) {
      console.error(error);
    }
  }
}
```

```

    } finally {
      setLoading(false);
    }
  }
}

useEffect(() => {
  loadPapers();
}, []);

const total = papers.length;
const parsed = papers.filter((p) => p.status === "parsed" || p.status === "indexed");
const indexed = papers.filter((p) => p.status === "indexed" || p.status === "extracted");
const extracted = papers.filter((p) => p.status === "extracted").length;

const stats = [
  { label: "Total papers", value: total, icon: FileText, color: "bg-blue-50 text-blue-700"},
  { label: "Parsed", value: parsed, icon: Database, color: "bg-cyan-50 text-cyan-700"},
  { label: "Indexed", value: indexed, icon: SearchCheck, color: "bg-emerald-50 text-emerald-700"},
  { label: "Extracted", value: extracted, icon: Sparkles, color: "bg-violet-50 text-violet-700"}
];

return (
  <AppShell>
    <div className="space-y-8">
      <section className="grid gap-6 lg:grid-cols-[1.2fr_0.8fr]">
        <div className="rounded-3xl border border-slate-200 bg-white p-8 shadow-slate-100">
          <div className="max-w-2xl">
            <div className="mb-4 inline-flex rounded-full bg-blue-50 px-3 py-1 text-blue-700">
              Research pipeline
            </div>
            <h2 className="text-4xl font-semibold tracking-tight text-slate-900">
              Organize papers into structured, searchable research knowledge
            </h2>
            <p className="mt-4 text-base leading-7 text-slate-600">
              Upload papers, parse sections, retrieve relevant evidence, and compare
              datasets, metrics, and limitations in one workspace.
            </p>
          </div>
        </div>
        <UploadCard onUploaded={loadPapers} />
      </section>

      <section className="grid gap-4 sm:grid-cols-2 xl:grid-cols-4">
        {stats.map((stat) => {
          const Icon = stat.icon;
          return (
            <div
              key={stat.label}
              className="rounded-2xl border border-slate-200 bg-white p-5 shadow-slate-100"
            >
              <div className="flex items-start justify-between">
                <div>
                  <p className="text-sm text-slate-500">{stat.label}</p>
                  <p className="mt-3 text-3xl font-semibold text-slate-900">{stat.value}</p>
                </div>

```

```

        <div className={`rounded-2xl p-3 ${stat.color}`}>
          <Icon className="h-5 w-5" />
        </div>
      </div>
    </div>
  );
}
</section>

<section className="rounded-3xl border border-slate-200 bg-white shadow-soft">
  <div className="flex items-center justify-between border-b border-slate-200">
    <div>
      <h3 className="text-lg font-semibold text-slate-900">Paper library</h3>
      <p className="text-sm text-slate-500">
        Uploaded papers and processing status
      </p>
    </div>
  </div>

  <div className="overflow-x-auto">
    <table className="min-w-full">
      <thead className="bg-slate-50">
        <tr className="text-left text-sm text-slate-500">
          <th className="px-6 py-4 font-medium">Title</th>
          <th className="px-6 py-4 font-medium">Filename</th>
          <th className="px-6 py-4 font-medium">Status</th>
          <th className="px-6 py-4 font-medium">Year</th>
          <th className="px-6 py-4 font-medium">Actions</th>
        </tr>
      </thead>
      <tbody>
        {loading ? (
          <tr>
            <td colspan={5} className="px-6 py-10 text-center text-slate-500">
              Loading papers...
            </td>
          </tr>
        ) : papers.length === 0 ? (
          <tr>
            <td colspan={5} className="px-6 py-10 text-center text-slate-500">
              No papers uploaded yet.
            </td>
          </tr>
        ) : (
          papers.map((paper) => (
            <tr key={paper.id} className="border-t border-slate-100">
              <td className="px-6 py-4 font-medium text-slate-900">
                {paper.title || "Untitled paper"}
              </td>
              <td className="px-6 py-4 text-slate-600">{paper.filename}</td>
              <td className="px-6 py-4">
                <PaperStatusBadge status={paper.status} />
              </td>
              <td className="px-6 py-4 text-slate-600">{paper.year || "-"}</td>
              <td className="px-6 py-4">
                <Link

```

```

                href={`\papers/${paper.id}`}
                className="inline-flex items-center gap-2 rounded-xl bg-slate-100 p-2"
            >
                Open
                <ArrowRight className="h-4 w-4" />
            </Link>
        </td>
    </tr>
</tbody>
</table>
</div>
</section>
</div>
</AppShell>
);
}

```

Better upload card

components/papers/upload-card.tsx

```

"use client";

import { useState } from "react";
import { uploadPaper } from "@lib/api";
import { UploadCloud } from "lucide-react";

export function UploadCard({ onUploaded }: { onUploaded: () => void }) {
    const [file, setFile] = useState<File | null>(null);
    const [loading, setLoading] = useState(false);

    const handleUpload = async () => {
        if (!file) return;

        try {
            setLoading(true);
            await uploadPaper(file);
            setFile(null);
            onUploaded();
        } catch (e) {
            console.error(e);
            alert("Upload failed");
        } finally {
            setLoading(false);
        }
    };

    return (
        <div className="rounded-3xl border border-slate-200 bg-white p-6 shadow-soft">
            <div className="mb-4 flex h-12 w-12 items-center justify-center rounded-2xl bg-slate-100">
                <UploadCloud className="h-5 w-5" />
            </div>

```

```

    <h3 className="text-xl font-semibold text-slate-900">Upload paper</h3>
    <p className="mt-2 text-sm leading-6 text-slate-500">
      Add a research PDF to build your structured corpus.
    </p>

    <label className="mt-5 block rounded-2xl border border-dashed border-slate-300"
      <span className="mb-2 block font-medium text-slate-700">Choose PDF file</span>
      <input
        type="file"
        accept="application/pdf"
        onChange={(e) => setFile(e.target.files?.[0] || null)}
        className="block w-full text-sm"
      />
    </label>

    {file && (
      <p className="mt-3 text-sm text-slate-600">
        Selected: <span className="font-medium text-slate-900">{file.name}</span>
      </p>
    )}

    <button
      onClick={handleUpload}
      disabled={!file || loading}
      className="mt-5 inline-flex w-full items-center justify-center rounded-2xl bg-slate-500 text-white"
    >
      {loading ? "Uploading..." : "Upload PDF"}
    </button>
  </div>
);
}

```

Better status badge

components/papers/paper-status-badge.tsx

```

import { PaperStatus } from "@lib/types";

const styles: Record<PaperStatus, string> = {
  uploaded: "bg-slate-100 text-slate-700",
  parsed: "bg-blue-50 text-blue-700",
  indexed: "bg-amber-50 text-amber-700",
  extracted: "bg-emerald-50 text-emerald-700",
};

export function PaperStatusBadge({ status }: { status: PaperStatus }) {
  return (
    <span className={`rounded-full px-3 py-1 text-xs font-medium capitalize ${styles[status]}`}>
      {status}
    </span>
  );
}

```

Why this will look better

This version improves usability by using card grouping, hierarchy, whitespace, and lighter colors instead of a heavy dark panel layout. That makes sense for a research assistant whose main tasks are paper exploration, structured comparison, and evidence-backed drafting support. ^[90]

Quick checklist

Make sure:

- `globals.css` is correctly imported in `app/layout.tsx`. ^[93] ^[94]
- Tailwind is actually active through either `@import "tailwindcss";` or the `@tailwind` directives. ^[91] ^[92]
- `lucide-react` is installed if you use the icons:

```
npm i lucide-react
```

Small extras you can add

To make it even more user-friendly, add:

- a mobile hamburger sidebar,
- a search bar in the topbar,
- a recent activity card,
- empty-state illustrations,
- toast notifications after upload. ^[90]

If you want, I can give you the **full light-theme code** for `compare/page.tsx` and `papers/[id]/page.tsx` in the same design so the whole app looks consistent.

✱✱

yes give

Yes — below is a **consistent light-theme UI** for both `compare/page.tsx` and `papers/[id]/page.tsx`, matching the cleaner dashboard style. In Next.js App Router, dynamic paper pages are created with a folder like `app/papers/[id]/page.tsx`, and the route parameter is passed through `params`. ^[108] ^[109] ^[110]

Compare page

This page lets users choose papers and view a structured comparison, which directly supports your synopsis goal of comparing methodologies, datasets, and evaluation metrics across papers. ^[111]

```

"use client";

import { useEffect, useState } from "react";
import { AppShell } from "@/components/layout/app-shell";
import { getPapers, comparePapers } from "@/lib/api";
import { CompareRow, Paper } from "@/lib/types";
import { GitCompareArrows, CheckCircle2 } from "lucide-react";

export default function ComparePage() {
  const [papers, setPapers] = useState<Paper[]>([]);
  const [selected, setSelected] = useState<string[]>([]);
  const [rows, setRows] = useState<CompareRow[]>([]);
  const [loadingPapers, setLoadingPapers] = useState(true);
  const [loadingCompare, setLoadingCompare] = useState(false);

  useEffect(() => {
    async function load() {
      try {
        setLoadingPapers(true);
        const data = await getPapers();
        setPapers(data);
      } catch (err) {
        console.error(err);
      } finally {
        setLoadingPapers(false);
      }
    }
    load();
  }, []);

  function togglePaper(id: string) {
    setSelected((prev) =>
      prev.includes(id) ? prev.filter((p) => p !== id) : [...prev, id].slice(0, 4)
    );
  }

  async function handleCompare() {
    if (selected.length < 2) {
      alert("Select at least 2 papers to compare");
      return;
    }

    try {
      setLoadingCompare(true);
      const data = await comparePapers(selected);
      setRows(data);
    } catch (err) {
      console.error(err);
      alert("Comparison failed");
    } finally {
      setLoadingCompare(false);
    }
  }
}

```

```

const selectedPapers = papers.filter((p) => selected.includes(p.id));

return (
  <AppShell>
    <div className="space-y-8">
      <section className="rounded-3xl border border-slate-200 bg-white p-8 shadow-s
        <div className="mb-6 flex items-start justify-between gap-4">
          <div>
            <div className="mb-3 inline-flex rounded-full bg-blue-50 px-3 py-1 tex
              Cross-paper analysis
            </div>
            <h2 className="text-3xl font-semibold tracking-tight text-slate-900">
              Compare research papers side by side
            </h2>
            <p className="mt-2 max-w-2xl text-sm leading-6 text-slate-600">
              Select papers to compare methods, datasets, metrics, baselines, and
              in one structured view.
            </p>
          </div>

          <div className="hidden rounded-2xl bg-slate-50 p-3 md:block">
            <GitCompareArrows className="h-8 w-8 text-blue-600" />
          </div>
        </div>

        <div className="grid gap-4 md:grid-cols-2 xl:grid-cols-3">
          {loadingPapers ? (
            <div className="text-sm text-slate-500">Loading papers...</div>
          ) : papers.length === 0 ? (
            <div className="text-sm text-slate-500">No papers available for compar
          ) : (
            papers.map((paper) => {
              const active = selected.includes(paper.id);

              return (
                <button
                  key={paper.id}
                  onClick={() => togglePaper(paper.id)}
                  className={`rounded-2xl border p-5 text-left transition ${
                    active
                      ? "border-blue-200 bg-blue-50 ring-soft"
                      : "border-slate-200 bg-white hover:border-blue-100 hover:bg-s
                  }`}
                >
                  <div className="flex items-start justify-between gap-3">
                    <div>
                      <h3 className="line-clamp-2 text-base font-semibold text-sla
                        {paper.title || "Untitled paper"}
                      </h3>
                      <p className="mt-2 text-sm text-slate-500">{paper.filename}<
                      <p className="mt-1 text-xs text-slate-400">
                        {paper.year || "Year unavailable"}
                      </p>
                    </div>
                    {active && <CheckCircle2 className="h-5 w-5 shrink-0 text-blue

```

```

        </div>
      </button>
    );
  })
  })
</div>

<div className="mt-6 flex flex-col gap-4 md:flex-row md:items-center md:justify-between">
  <p className="text-sm text-slate-500">
    Selected: <span className="font-semibold text-slate-900">{selected.length}</span>
  </p>

  <button
    onClick={handleCompare}
    disabled={selected.length < 2 || loadingCompare}
    className="inline-flex items-center justify-center rounded-2xl bg-blue-600 text-white px-4 py-2"
  >
    {loadingCompare ? "Generating comparison..." : "Compare selected papers"}
  </button>
</div>
</section>

<section className="rounded-3xl border border-slate-200 bg-white shadow-soft">
  <div className="border-b border-slate-200 px-6 py-5">
    <h3 className="text-lg font-semibold text-slate-900">Comparison table</h3>
    <p className="text-sm text-slate-500">
      Structured output across selected research papers
    </p>
  </div>

  {rows.length === 0 ? (
    <div className="px-6 py-10 text-center text-slate-500">
      No comparison generated yet.
    </div>
  ) : (
    <div className="overflow-x-auto">
      <table className="min-w-full">
        <thead className="bg-slate-50">
          <tr className="text-left text-sm text-slate-500">
            <th className="px-6 py-4 font-medium">Field</th>
            <th>
              {selectedPapers.map((paper) => (
                <th key={paper.id} className="px-6 py-4 font-medium">
                  <div className="max-w-[220px]">
                    <p className="line-clamp-2 font-semibold text-slate-700">{paper.title}</p>
                    <p className="mt-1 text-xs text-slate-400">{paper.filename}</p>
                  </div>
                </th>
              ))}
            </th>
          </tr>
        </thead>
        <tbody>
          {rows.map((row) => (
            <tr key={row.field} className="border-t border-slate-100 align-top">
              <td className="whitespace-nowrap px-6 py-4 font-medium text-slate-900">
                {row.field}
              </td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  )}
</section>

```

```

        {selectedPapers.map((paper) => (
          <td key={paper.id} className="px-6 py-4 text-sm leading-6 te
            {row.values[paper.id] || "-"}
          </td>
        ))}
      </tr>
    ))}
  </tbody>
</table>
</div>
)}
</section>
</div>
</AppShell>
);
}

```

Paper detail page

This page presents one paper with metadata, extracted fields, and semantic search results, which aligns with your project's paper parsing, structured extraction, and retrieval phases.^[111]

app/papers/[id]/page.tsx

```

"use client";

import { useEffect, useState } from "react";
import { useParams } from "next/navigation";
import { AppShell } from "@/components/layout/app-shell";
import { extractPaper, getExtraction, getPaper, parsePaper, searchPaper } from "@/lib/api";
import { ChunkResult, Extraction, Paper } from "@/lib/types";
import {
  FileText,
  Sparkles,
  Search,
  Database,
  FlaskConical,
  ArrowLeftCircle,
} from "lucide-react";
import Link from "next/link";

export default function PaperDetailPage() {
  const params = useParams<{ id: string }>();
  const id = params.id;

  const [paper, setPaper] = useState<Paper | null>(null);
  const [extraction, setExtraction] = useState<Extraction | null>(null);
  const [results, setResults] = useState<ChunkResult[]>([]);
  const [query, setQuery] = useState("");
  const [loading, setLoading] = useState(true);
  const [searching, setSearching] = useState(false);
  const [parsing, setParsing] = useState(false);
  const [extracting, setExtracting] = useState(false);

```

```

async function loadPaper() {
  try {
    setLoading(true);
    const [paperData, extractionData] = await Promise.all([
      getPaper(id),
      getExtraction(id).catch(() => null),
    ]);

    setPaper(paperData);
    setExtraction(extractionData);
  } catch (err) {
    console.error(err);
  } finally {
    setLoading(false);
  }
}

useEffect(() => {
  if (id) loadPaper();
}, [id]);

async function handleParse() {
  try {
    setParsing(true);
    await parsePaper(id);
    await loadPaper();
  } catch (err) {
    console.error(err);
    alert("Parsing failed");
  } finally {
    setParsing(false);
  }
}

async function handleExtract() {
  try {
    setExtracting(true);
    await extractPaper(id);
    await loadPaper();
  } catch (err) {
    console.error(err);
    alert("Extraction failed");
  } finally {
    setExtracting(false);
  }
}

async function handleSearch() {
  if (!query.trim()) return;
  try {
    setSearching(true);
    const data = await searchPaper(id, query);
    setResults(data);
  } catch (err) {
    console.error(err);
    alert("Search failed");
  }
}

```

```

    } finally {
      setSearching(false);
    }
  }
}

const extractedCards = [
  { label: "Objective", value: extraction?.objective, icon: FlaskConical },
  { label: "Methodology", value: extraction?.methodology, icon: Database },
  { label: "Dataset", value: extraction?.dataset, icon: FileText },
  { label: "Metric", value: extraction?.evaluation_metric, icon: Sparkles },
  { label: "Limitations", value: extraction?.limitations, icon: Search },
  { label: "Future work", value: extraction?.future_work, icon: ArrowLeftCircle },
];

return (
  <AppShell>
    <div className="space-y-8">
      <div className="flex items-center gap-3">
        <Link
          href="/dashboard"
          className="inline-flex items-center gap-2 rounded-xl border border-slate-200 p-2"
        >
          <ArrowLeftCircle className="h-4 w-4" />
          Back to dashboard
        </Link>
      </div>

      {loading ? (
        <div className="rounded-3xl border border-slate-200 bg-white p-8 text-slate-600">
          Loading paper...
        </div>
      ) : !paper ? (
        <div className="rounded-3xl border border-slate-200 bg-white p-8 text-slate-600">
          Paper not found.
        </div>
      ) : (
        <>
          <section className="rounded-3xl border border-slate-200 bg-white p-8 shadow">
            <div className="flex flex-col gap-6 lg:flex-row lg:items-start lg:justify-between">
              <div className="max-w-3xl">
                <div className="mb-3 inline-flex rounded-full bg-blue-50 px-3 py-1">
                  Paper detail
                </div>
                <h1 className="text-3xl font-semibold tracking-tight text-slate-900">
                  {paper.title || "Untitled paper"}
                </h1>
                <p className="mt-3 text-sm leading-6 text-slate-600">{paper.filename}
                </p>
                {paper.abstract && (
                  <div className="mt-6 rounded-2xl bg-slate-50 p-5">
                    <h2 className="text-sm font-semibold text-slate-900">Abstract</h2>
                    <p className="mt-2 text-sm leading-7 text-slate-600">{paper.abstract}
                  </div>
                )}
              </div>
            </div>
          </section>
        </>
      )}
    </div>
  </AppShell>
)

```

```

<div className="grid gap-3 sm:grid-cols-2 lg:w-[320px] lg:grid-cols-1" style={{width: 320px}}>
  <div className="rounded-2xl border border-slate-200 bg-slate-50 p-4">
    <p className="text-xs uppercase tracking-wide text-slate-400">Yes</p>
    <p className="mt-2 text-lg font-semibold text-slate-900">{paper.name}</p>
  </div>
  <div className="rounded-2xl border border-slate-200 bg-slate-50 p-4">
    <p className="text-xs uppercase tracking-wide text-slate-400">Status</p>
    <p className="mt-2 text-lg font-semibold capitalize text-slate-900">
      {paper.status}
    </p>
  </div>
  <button
    onClick={handleParse}
    disabled={parsing}
    className="rounded-2xl bg-blue-600 px-4 py-3 text-sm font-medium"
  >
    {parsing ? "Parsing..." : "Parse paper"}
  </button>
  <button
    onClick={handleExtract}
    disabled={extracting}
    className="rounded-2xl bg-emerald-600 px-4 py-3 text-sm font-medium"
  >
    {extracting ? "Extracting..." : "Run extraction"}
  </button>
</div>
</div>
</section>

<section className="rounded-3xl border border-slate-200 bg-white p-6 shadow-sm">
  <div className="mb-5">
    <h2 className="text-xl font-semibold text-slate-900">Structured extraction</h2>
    <p className="text-sm text-slate-500">
      Key fields extracted from the paper
    </p>
  </div>

  <div className="grid gap-4 md:grid-cols-2 xl:grid-cols-3">
    {extractedCards.map((item) => {
      const Icon = item.icon;
      return (
        <div
          key={item.label}
          className="rounded-2xl border border-slate-200 bg-slate-50 p-5"
        >
          <div className="mb-3 flex items-center gap-3">
            <div className="rounded-xl bg-white p-2 shadow-sm">
              <Icon className="h-4 w-4 text-blue-600" />
            </div>
            <h3 className="font-semibold text-slate-900">{item.label}</h3>
          </div>
          <p className="text-sm leading-6 text-slate-600">
            {item.value || "Not extracted yet."}
          </p>
        </div>
      );
    })}
  </div>

```

```

    }}}
  </div>
</section>

<section className="rounded-3xl border border-slate-200 bg-white p-6 shadow" >
  <div className="mb-5">
    <h2 className="text-xl font-semibold text-slate-900">Semantic search</h2>
    <p className="text-sm text-slate-500">
      Search relevant chunks from this paper
    </p>
  </div>

  <div className="flex flex-col gap-3 md:flex-row">
    <input
      value={query}
      onChange={(e) => setQuery(e.target.value)}
      placeholder="e.g. What dataset is used in this paper?"
      className="w-full rounded-2xl border border-slate-200 bg-slate-50"
    />
    <button
      onClick={handleSearch}
      disabled={searching}
      className="rounded-2xl bg-slate-900 px-5 py-3 text-sm font-medium"
    >
      {searching ? "Searching..." : "Search"}
    </button>
  </div>

  <div className="mt-6 space-y-4">
    {results.length === 0 ? (
      <div className="rounded-2xl border border-dashed border-slate-200 p-5">
        No search results yet.
      </div>
    ) : (
      results.map((chunk) => (
        <div
          key={chunk.id}
          className="rounded-2xl border border-slate-200 bg-slate-50 p-5"
        >
          <div className="mb-2 flex flex-wrap items-center gap-2 text-xs">
            <span className="rounded-full bg-white px-3 py-1">
              {chunk.section_title || "Section"}
            </span>
            <span className="rounded-full bg-white px-3 py-1">
              Page {chunk.page || "-"}
            </span>
            {typeof chunk.score === "number" && (
              <span className="rounded-full bg-blue-50 px-3 py-1 text-blue-800">
                Score {chunk.score.toFixed(2)}
              </span>
            )}
          </div>
          <p className="text-sm leading-7 text-slate-700">{chunk.text}</p>
        </div>
      ))
    )}
  </div>

```

```

        </div>
      </section>
    </>
  )}
</div>
</AppShell>
);
}

```

Add search route too

Your sidebar includes `/search`, so add a simple placeholder page to avoid broken navigation. App Router uses folder-based routing, so create `app/search/page.tsx` for that route. [\[112\]](#) [\[108\]](#)

`app/search/page.tsx`

```

import { AppShell } from "@components/layout/app-shell";
import { Search } from "lucide-react";

export default function SearchPage() {
  return (
    <AppShell>
      <div className="rounded-3xl border border-slate-200 bg-white p-10 shadow-soft">
        <div className="mb-4 flex h-14 w-14 items-center justify-center rounded-2xl">
          <Search className="h-6 w-6" />
        </div>
        <h1 className="text-3xl font-semibold tracking-tight text-slate-900">
          Global semantic search
        </h1>
        <p className="mt-3 max-w-2xl text-sm leading-7 text-slate-600">
          This page can later support cross-paper search across the uploaded corpus,
          retrieve citation-grounded evidence from multiple papers in one place.
        </p>
      </div>
    </AppShell>
  );
}

```

Small API reminder

These pages assume your backend already exposes:

- GET `/papers`
- GET `/papers/:id`
- GET `/papers/:id/extraction`
- POST `/papers/:id/search`
- POST `/papers/:id/parse`
- POST `/papers/:id/extract`
- POST `/compare` [\[111\]](#)

That API split matches the paper collection, preprocessing, retrieval, extraction, and comparison flow from your project methodology. [\[111\]](#)

Final polish tips

To make the full app feel even more user-friendly:

- add loading skeletons,
- show success toasts after upload/parse/extract,
- truncate very long titles with hover tooltips,
- add mobile sidebar support,
- keep all cards using the same `rounded-2x1` and `border-slate-200` pattern for consistency. [\[111\]](#)

Would you like me to give you the **backend FastAPI routes and Pydantic schemas** for these pages next?



i got this error

That error means your frontend is trying to call the backend, but the browser cannot complete the request. `TypeError: Failed to fetch` usually points to one of three things: the FastAPI server is not running, the URL/port is wrong, or the browser is blocking the request because of CORS. [\[124\]](#) [\[125\]](#) [\[126\]](#)

Most likely cause

Your `lib/api.ts` is using:

```
const API_BASE = process.env.NEXT_PUBLIC_API_BASE || "http://localhost:8000";
```

So the frontend is trying to fetch from `http://localhost:8000/...`, and if your FastAPI app is not running there, or if CORS is not enabled for your Next.js origin, the browser throws `Failed to fetch`. On localhost, different ports are different origins, so `localhost:3000` to `localhost:8000` needs explicit CORS permission. [\[125\]](#) [\[127\]](#) [\[126\]](#) [\[124\]](#)

Fix 1: Start the backend

Make sure FastAPI is actually running on port 8000. A browser-level fetch error often happens because nothing is listening on the target port. [\[128\]](#) [\[125\]](#)

Run:

```
uvicorn main:app --reload --port 8000
```

Then test in browser or terminal:

```
curl http://127.0.0.1:8000/papers
```

If that fails, the problem is backend startup or wrong route, not Next.js. Testing the API directly is the first step recommended for separating backend issues from frontend issues. [\[129\]](#) [\[130\]](#) [\[128\]](#)

Fix 2: Use 127.0.0.1 instead of localhost

A common local fix is to use 127.0.0.1 consistently instead of localhost, because some setups behave differently across browser, Node, and backend bindings. [\[131\]](#)

Create .env.local in your Next.js project:

```
NEXT_PUBLIC_API_BASE=http://127.0.0.1:8000
```

Then update lib/api.ts:

```
const API_BASE = process.env.NEXT_PUBLIC_API_BASE || "http://127.0.0.1:8000";
```

Restart Next.js after changing env vars:

```
npm run dev
```

Fix 3: Enable CORS in FastAPI

If the backend is running but the browser still blocks the request, CORS is the likely issue. FastAPI does not allow cross-origin requests by default, so you need CORSMiddleware and must allow your frontend origin explicitly. [\[127\]](#) [\[126\]](#) [\[132\]](#) [\[125\]](#)

main.py

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()

origins = [
    "http://localhost:3000",
    "http://127.0.0.1:3000",
]

app.add_middleware(
    CORSMiddleware,
    allow_origins=origins,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

```
@app.get("/")
def root():
    return {"message": "API is running"}

@app.get("/papers")
def get_papers():
    return []
```

FastAPI CORS middleware must be added on the server side, because the browser requires Access-Control-Allow-Origin from the API response before it will allow the frontend to read the result. [\[126\]](#) [\[125\]](#)

Fix 4: Make sure your routes exist

Your frontend expects these endpoints:

- GET /papers
- GET /papers/{id}
- GET /papers/{id}/extraction
- POST /papers/{id}/search
- POST /papers/{id}/parse
- POST /papers/{id}/extract
- POST /compare [\[133\]](#)

If GET /papers does not exist yet, getPapers() will fail immediately because the dashboard loads it on first render. Your phase-1 UI is designed around uploaded papers, parsing, retrieval, extraction, and comparison, so /papers is the first route you need working. [\[133\]](#)

Temporary frontend-safe API code

Right now your UI crashes visibly because the fetch error bubbles up. Add safer error handling so the app still renders even when the backend is not ready. A fetch failure is not an HTTP error; it means the request failed before the server response was usable. [\[134\]](#) [\[125\]](#)

Replace lib/api.ts with this safer version

```
import { CompareRow, Extraction, Paper, ChunkResult } from "./types";

const API_BASE = process.env.NEXT_PUBLIC_API_BASE || "http://127.0.0.1:8000";

async function api<T>(path: string, options?: RequestInit): Promise<T> {
  try {
    const res = await fetch(`${API_BASE}${path}`, {
      ...options,
      headers: {
        ...(options?.headers || {}),
      },
      cache: "no-store",
    });
  };
```

```

    if (!res.ok) {
      const text = await res.text();
      throw new Error(`API ${res.status}: ${text} || "Request failed"``);
    }

    return res.json();
  } catch (error) {
    console.error("Fetch error:", error);
    throw new Error(
      "Could not connect to backend. Check FastAPI server, API URL, and CORS settings"
    );
  }
}

export const getPapers = () => api<Paper[]>("/papers");
export const getPaper = (id: string) => api<Paper>(`/papers/${id}`);
export const getExtraction = (id: string) => api<Extraction>(`/papers/${id}/extractions`);

export const searchPaper = (id: string, query: string) =>
  api<ChunkResult[]>(`/papers/${id}/search`, {
    method: "POST",
    body: JSON.stringify({ query }),
    headers: { "Content-Type": "application/json" },
  });

export const comparePapers = (paperIds: string[]) =>
  api<CompareRow[]>("/compare", {
    method: "POST",
    body: JSON.stringify({ paper_ids: paperIds }),
    headers: { "Content-Type": "application/json" },
  });

export async function uploadPaper(file: File) {
  const formData = new FormData();
  const res = await fetch(`${API_BASE}/papers/upload`, {
    method: "POST",
    body: formData,
  });

  if (!res.ok) throw new Error("Upload failed");
  return res.json();
}

export const parsePaper = (id: string) =>
  api(`/papers/${id}/parse`, { method: "POST" });

export const extractPaper = (id: string) =>
  api(`/papers/${id}/extract`, { method: "POST" });

```

Minimal FastAPI starter backend

If you just want the frontend to stop failing tonight, use this basic backend first.

main.py

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel

app = FastAPI()

app.add_middleware(
    CORSMiddleware,
    allow_origins=[
        "http://localhost:3000",
        "http://127.0.0.1:3000",
    ],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

class SearchRequest(BaseModel):
    query: str

class CompareRequest(BaseModel):
    paper_ids: list[str]

papers = [
    {
        "id": "1",
        "title": "ResearchOS Demo Paper",
        "filename": "demo-paper.pdf",
        "authors": ["Vaishnavi Balodhi"],
        "year": 2026,
        "abstract": "Demo abstract for testing frontend integration.",
        "status": "uploaded",
    }
]

@app.get("/")
def root():
    return {"message": "Backend running"}

@app.get("/papers")
def get_papers():
    return papers

@app.get("/papers/{paper_id}")
def get_paper(paper_id: str):
    for p in papers:
        if p["id"] == paper_id:
            return p
    return {"id": paper_id, "title": "Untitled paper", "filename": "unknown.pdf", "s
```

```

@app.get("/papers/{paper_id}/extraction")
def get_extraction(paper_id: str):
    return {
        "objective": "Assist researchers with literature analysis.",
        "methodology": "RAG + semantic search + structured extraction.",
        "dataset": "Uploaded research corpus",
        "evaluation_metric": "User relevance and extraction accuracy",
        "limitations": "Prototype stage, limited corpus",
        "future_work": "Add reviewer simulation and LaTeX export",
    }

@app.post("/papers/{paper_id}/search")
def search_paper(paper_id: str, body: SearchRequest):
    return [
        {
            "id": "chunk-1",
            "section_title": "Methodology",
            "page": 3,
            "text": f"Result for query: {body.query}",
            "score": 0.93,
        }
    ]

@app.post("/papers/{paper_id}/parse")
def parse_paper(paper_id: str):
    return {"message": f"Paper {paper_id} parsed successfully"}

@app.post("/papers/{paper_id}/extract")
def extract_paper(paper_id: str):
    return {"message": f"Paper {paper_id} extracted successfully"}

@app.post("/compare")
def compare(body: CompareRequest):
    return [
        {
            "field": "Methodology",
            "values": {pid: "RAG-based pipeline" for pid in body.paper_ids},
        },
        {
            "field": "Dataset",
            "values": {pid: "Uploaded PDF corpus" for pid in body.paper_ids},
        },
        {
            "field": "Limitation",
            "values": {pid: "Prototype stage" for pid in body.paper_ids},
        },
    ]

```

Run it:

```

pip install fastapi uvicorn
uvicorn main:app --reload --port 8000

```

Best debugging order

Do these in order:

1. Open `http://127.0.0.1:8000/papers` in the browser. If it does not open, backend is not running. [\[125\]](#) [\[128\]](#)
2. If it opens, but frontend still fails, it is probably CORS. [\[124\]](#) [\[126\]](#)
3. Set `NEXT_PUBLIC_API_BASE=http://127.0.0.1:8000` in `.env.local`. [\[135\]](#) [\[131\]](#)
4. Restart both backend and frontend after changes. [\[135\]](#)
5. Check browser DevTools Network tab to see whether the request is blocked, refused, or missing. Reading the exact failing request is a recommended debugging step. [\[129\]](#) [\[125\]](#)

What this means for your project

This error is not about your UI design; it is about the frontend-backend connection. Since your phase-1 ResearchOS demo depends on paper upload, parsing, retrieval, extraction, and comparison, getting the `/papers` and related routes stable is the first thing you need before the polished frontend can work properly. [\[133\]](#)

Send me your current `main.py` and `lib/api.ts`, and I'll patch them line by line.

✱

1. Synopsis-content.pdf
2. Synopsis-content.pdf
3. Synopsis-content.pdf
4. Synopsis-content.pdf
5. Synopsis-content.pdf
6. Synopsis-content.pdf
7. Synopsis-content.pdf
8. Synopsis-content.pdf
9. Synopsis-content.pdf
10. Synopsis-content.pdf
11. Synopsis-content.pdf
12. Synopsis-content.pdf
13. Synopsis-content.pdf
14. Synopsis-content.pdf
15. Synopsis-content.pdf
16. Synopsis-content.pdf
17. image.jpg
18. Synopsis-content.pdf
19. image.jpg
20. <https://fastapi.tiangolo.com/tutorial/response-model/>

21. <https://pub.towardsai.net/ii-type-hints-pydantic-fastapis-foundation-dcccba337eaf>
22. <https://codingls.com/python/8633/>
23. <https://www.getorchestra.io/guides/pydantic-json-schema-a-comprehensive-guide-for-fastapi-users>
24. Synopsis-content.pdf
25. https://huggingface.co/koichi12/llm_tutorial/raw/7d4012e61f079d6181ebf25ee757481f705277ff/.venv/lib/python3.11/site-packages/fastapi/utils.py
26. <https://github.com/fastapi/fastapi/discussions/9709>
27. https://gdevops.frama.io/python/versions/3.9.0/pep_0585/pep_0585.html
28. <https://discuss.huggingface.co/t/fastapi-gradio-versioning-fieldinfo-object-has-no-attribute-in/48140>
29. <https://github.com/fastapi/fastapi/discussions/14264>
30. <https://python3.info/fastapi/pydantic/schema.html>
31. <https://python-data-science.readthedocs.io/en/latest/fastapi.html>
32. https://github.com/bmaltais/kohya_ss/issues/1253
33. <https://stackoverflow.com/questions/73957175/why-do-i-get-attribution-error-in-fast-api-attributeerror-type-object-has-no-a>
34. <https://fastapi.tiangolo.com/reference/openapi/models/>
35. <https://stackoverflow.com/questions/73664830/pydantic-object-has-no-attribute-fields-set-error>
36. Synopsis-content.pdf
37. image.jpg
38. image-2.jpg
39. Synopsis-content.pdf
40. <https://www.perplexity.ai/search/6b513a5f-2d48-4cab-bf5f-8c9f608c43a0>
41. Synopsis-content.pdf
42. <https://www.perplexity.ai/search/5aff92ed-d85c-4378-9fd6-3426d8071895>
43. <https://www.perplexity.ai/search/23b82362-acae-4d0a-ac3d-32bfd287686c>
44. <https://www.perplexity.ai/search/93de3d45-a22b-437e-a8f6-cca3d6e2f6f4>
45. Synopsis-content.pdf
46. <https://www.perplexity.ai/search/23b82362-acae-4d0a-ac3d-32bfd287686c>
47. <https://www.perplexity.ai/search/5aff92ed-d85c-4378-9fd6-3426d8071895>
48. Synopsis-content.pdf
49. <https://www.perplexity.ai/search/23b82362-acae-4d0a-ac3d-32bfd287686c>
50. <https://www.perplexity.ai/search/ad51b890-de3d-4066-becd-260b6c551881>
51. <https://www.perplexity.ai/search/2429ac22-087f-4f6c-8bfb-d61385859720>
52. Synopsis-content.pdf
53. <https://www.perplexity.ai/search/ba7e5460-05bf-425e-8819-1fa8947ca6da>
54. <https://nextjs.org/docs/app/getting-started>
55. <https://nextjs.org/docs/app/api-reference/cli/create-next-app>
56. <https://v3.shadcn.com/docs/installation/next>

57. Synopsis-content.pdf

58. <https://nextjs.org/docs/app/getting-started/installation>

59. <https://ui.shadcn.com/docs/installation/next>

60. <https://ui.shadcn.com/docs/react-19>

61. <https://www.geeksforgeeks.org/reactjs/next-js-13-app-router/>

62. <https://www.youtube.com/watch?v=bTMJgsYhKNw>

63. <https://nextjs.org/docs/app>

64. <https://medium.com/@kulsumansari4/next-js-app-router-routing-8d795dbe324c>

65. <https://nextjs.org/learn/pages-router/create-nextjs-app-setup>

66. <https://medium.com/@npguapo/installation-of-next-js-app-router-app-7bc1a66d9476>

67. <https://www.shadcn.io/ui/installation/nextjs>

68. <https://dev.to/lorenzokrl/nextjs-app-router-4l7e>

69. <https://dev.to/hassanazhar/routing-in-nextjs-how-to-use-app-router-in-your-next-apps-1iea>

70. <https://nextjs.org/docs/app/guides/tailwind-v3-css>

71. <https://nextjs-ko.org/docs/app/building-your-application/styling/tailwind-css>

72. <https://openillumi.com/en/en-nextjs-app-router-tailwind-css-fix/>

73. <https://www.softpplx.com/category/next-js/error/tailwind-not-working-nextjs-app-router>

74. <https://diko-dev99.medium.com/how-to-set-up-tailwind-css-in-next-js-13-app-pages-router-e0cfabc6c1f4>

75. <https://medium.com/@pranitb10/next-js-tailwind-css-a-perfect-recipe-for-developing-magical-and-blazing-websites-92ef08d3b5a2>

76. <https://www.youtube.com/watch?v=mE25ssUPFe0>

77. <https://stackoverflow.com/questions/74403355/unable-to-import-the-global-css-file-to-the-root-layout-in-next-js-13>

78. Synopsis-content.pdf

79. image.jpg

80. <https://schemalispurple.github.io/nextjs/tailwindcss-globalcss/>

81. https://dev.to/sudiip_17/how-to-set-up-tailwind-css-in-nextjs-complete-guide-for-2025-2232

82. <https://github.com/vercel/next.js/discussions/25845>

83. <https://webkul.com/blog/how-to-use-tailwind-css-in-next-js/>

84. https://medium.com/@kemaltf_/next-js-14-styling-458c1735d966

85. <https://qiita.com/taiyo914/items/ad9835c81150997e7f86>

86. <https://stackoverflow.com/questions/76600778/tailwind-not-applying-styles-to-pages-in-nextjs-but-applies-to-index-page>

87. <https://medium.com/@sorayacantos/efficient-font-loading-with-next-js-and-tailwindcss-3a8163f2d511>

88. <https://nextjs.org/docs/pages/getting-started/fonts>

89. <https://www.buildwithmatija.com/blog/nextjs-16-google-fonts-inter-exo-2>

90. Synopsis-content.pdf

91. <https://nextjs.org/docs/app/guides/tailwind-v3-css>

92. <https://www.youtube.com/watch?v=mE25ssUPFe0>
93. <https://openillumi.com/en/en-nextjs-app-router-tailwind-css-fix/>
94. <https://www.softplix.com/category/next-js/error/tailwind-not-working-nextjs-app-router>
95. <https://huggingface.co/spaces/rotoaurio/candymodel/raw/5e3b91cec87bdb57e17f4330e0ca4d35b319f993/next-fonts-fix.md>
96. <https://medium.com/@divineosehotue/using-custom-fonts-in-next-js-tailwind-css-v4-a37057b18f7f>
97. <https://dev.to/adhi28snsct/i-built-a-modern-saas-dashboard-ui-with-nextjs-tailwind-3129>
98. <https://saas-factory.ai/docs/font-optimisation-perf-03>
99. <https://github.com/tailwindlabs/tailwindcss/discussions/15923>
100. <https://dev.to/mikeesto/how-to-use-nextfont-globally-4img>
101. <https://www.youtube.com/watch?v=HAc5fRSPsXk>
102. <https://github.com/nnerb/dashboard-nextjs-tailwind>
103. <https://tailadmin.com/nextjs>
104. <https://colorlib.com/wp/nextjs-admin-dashboard-templates/>
105. <https://www.youtube.com/watch?v=rQZU-q25dIY>
106. <https://stackoverflow.com/questions/79329935/i-need-help-importing-my-global-css-file-in-next-js>
107. image.jpg
108. <https://nextjs.org/docs/app/api-reference/file-conventions/dynamic-routes>
109. <https://dev.to/nik-bogachenkov/dive-into-nextjs-app-router-building-dynamic-nested-and-static-pages-4e22>
110. <https://dev.to/khanrabiul/dynamic-routes-in-nextjs-4o74>
111. Synopsis-content.pdf
112. <https://medium.com/@1206411001/fetching-data-from-app-router-client-component-in-nextjs-d592120b6b08>
113. <https://stackoverflow.com/questions/77412027/using-next-13-5-6-app-router-how-to-get-params-of-dynamic-route>
114. https://www.reddit.com/r/nextjs/comments/175620x/fetching_data_from_app_router_client_component/
115. <https://medium.com/@sehouli.hamza/creating-dynamic-routes-in-nextjs-15-with-app-router-e8cc15e401d0>
116. <https://oneuptime.com/blog/post/2026-01-24-nextjs-dynamic-routing/view>
117. <https://dev.to/fujiwaracj/nextjs-13-data-fetching-with-app-router-289h>
118. <https://stackoverflow.com/questions/79594120/best-pattern-for-fetching-both-from-server-and-client-in-next-js-app-router-han>
119. <https://www.youtube.com/watch?v=7Kz4--kCBP0>
120. <https://www.youtube.com/watch?v=BukWELRZSIg>
121. <https://stackoverflow.com/questions/76110930/read-url-dynamic-parameter-slug-with-next-js-in-the-app-router>
122. <https://www.youtube.com/watch?v=b9lWYqhbHzg>
123. <https://medium.com/@snehalmehra017/routing-in-next-js-using-the-app-router-1c03ce9225c8>
124. https://www.linkedin.com/posts/ramavala_fastapi-nextjs-python-activity-7458375903996465152-5fkH
125. <https://jsondevtools.org/errors/failed-to-fetch.html>
126. <https://httpfixer.dev/blog/cors/fix-cors-fastapi/>

- 127. <https://stackoverflow.com/questions/63984555/fetch-in-javascript-for-fastapi-in-localhost>
- 128. <https://www.w3tutorials.net/blog/next-js-fetch-request-gives-error-typeerror-fetch-failed/>
- 129. <https://stackoverflow.com/questions/71480475/javascript-es6-fastapi-fetch-results-in-type-error>
- 130. <https://stackoverflow.com/questions/79415742/my-fastapi-service-is-giving-cors-errors-from-my-website-running-on-localhost-wh>
- 131. <https://stackoverflow.com/questions/74165121/next-js-fetch-request-gives-error-typeerror-fetch-failed>
- 132. <https://community.appybuilder.com/blog/fastapi-next-js-solving-cors-issues-1764806068>
- 133. Synopsis-content.pdf
- 134. <https://www.devsky.net/2025/03/fix-nextjs-error-fetch-failed-due-to.html>
- 135. <https://www.xjavascript.com/blog/fetch-error-when-building-next-js-static-website-in-production/>
- 136. image.jpg
- 137. <https://stackoverflow.com/questions/42754388/uncaught-in-promise-typeerror-failed-to-fetch-and-cors-error>
- 138. <https://sentry.io/answers/how-do-i-fix-the-typeerror-failed-to-fetch-error-in-javascript/>
- 139. https://www.reddit.com/r/nextjs/comments/1kvr7al/type_error_failed_to_fetch_how_to_debug/
- 140. <https://teratail.com/questions/en7yodi2glhbma>